Embedding

류영표 강사
ryp1662@gmail.com



Youngpyo Ryu. All rights reserved.
Unauthorized reproduction or distribution is prohibited.

류 영 표

AI & Applied Mathematics Researcher · Lecturer

ryp1662@gmail.com



학력

- 동국대학교 일반대학원 응용수학 석사 수료 (2016-2019)
- 동국대학교 수학과 졸업 (2010-2016)

심사 및 전문가 자문 활동

- 인공지능 학습용 데이터 구축 사업 품질검증 전문가 자문위원 (2023)
- 2023 신뢰할 수 있는 인공지능 개발 안내서(안)_고도화 전문위원
- 2023 자율주행/의료/공공사회 분야 신뢰할 수 있는 인공지능 개발 안내서(안) 자문위원

연구 및 프로젝트

- 부산대학교 병원: 개인 맞춤형 당뇨병 예방·관리 인공지능 시스템 외주 프로젝트 (2022)
- 연세대학교 병원: 영상의학과 FLASK 고도화 연구 (2023, 2024)
- 산업수학 스터디 그룹 (국가수리과학연구소): 피부암/유방암 분류, 유전자 정보 분석
- 폐플라스틱 이미지 객체 검출 경진대회 3위 (2021)
- 동국대학교 수학과 연구원(2025.10 ~ / 양자 AI, XAI 연구 진행 중)

논문

- Quantum computing for the optimization of CT image reconstruction, IEEE ICTC 2022

㈜ 헬로월드랩스 전담교육팀 과장

강의 및 교육 활동

대학 강의

- 고려대학교, 인천대학교, 동양미래대학교, 목포대학교
- 이젠 아카데미: 파이썬/ADSP 과정

기업 교육

- 현대자동차 연구원
- 한화, 하나금융, 한전 KDN, IBK 기업은행, 아가방앤컴퍼니, IM증권

공공/국비 과정

- [KT Cloud] 생성형 AI 2기
- [Est Camp] AI 퀀트 캠프 1기
- 보건·바이오 AI 활용 전문가 양성과정 (1-3기)
- 공공데이터 청년 인턴 멘토
- 충청 ICT 취창업 역량강화 프로그램 (SQLD)

아카데미 & 부트캠프

- 모두의연구소 Aiffel 퍼실리테이터 (1기)
- Upstage AI Boostcamp 멘토 (1-6기)
- 패스트캠퍼스/멀티캠퍼스 조교 및 멘토
- 공개SW개발자대회 멘토

자연어 임베딩



❶ 비정형 데이터란? - 비정형 데이터와 표현학습 개요, 임베딩이 필요한 이유

비정형 데이터는 정해진 행과 열의 구조(테이블)가 없고 사람의 언어나 감각을 통해 이해하는
텍스트, 이미지, 오디오, 영상과 같은 데이터를 의미합니다.

비정형 데이터의 예시

텍스트

배송이 정말 빨랐어요!
제품도 만족스럽습니다.
… …

고객 리뷰, 뉴스 기사, 이메일, SNS 게시글 등

이미지



사진, 의료 영상, CCTV, 도면, 그림 등

오디오



음성, 음악, 효과음, 인터뷰 녹음 등

영상



동영상, 강의 영상, 실시간 스트리밍 등

이러한 데이터는 사람이 이해하기는 쉽지만, 컴퓨터(기계)는 그대로 처리하기 어렵습니다.
따라서 AI가 이해할 수 있도록 **숫자 백터로 변환하는 과정이** 필요합니다.

임베딩(Embedding)이 필요한 이유



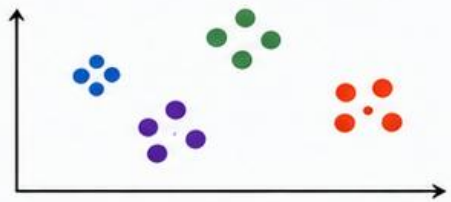
텍스트, 이미지, 소리를
그대로는 이해할 수 없어요!

컴퓨터는 숫자(벡터)만 이해하고 계산합니다.
010101 101010 110011 100101 …

1. 비정형 데이터

- 텍스트
- 이미지
- 오디오
- 영상
- …

2. 임베딩(백터 변환)



각 데이터를 의미를 가진
숫자 백터로 변환하여
백터 공간에 배치합니다.

3. AI 활용

- 유사도 계산 / 검색
- 추천 (상품, 콘텐츠, 사용자)
- 분류 / 예측
- 클러스터링 / 패턴 분석
- …

핵심 메시지: 임베딩은 비정형 데이터를 AI가 이해하고 활용할 수 있도록 숫자 백터로 변환하는 핵심 과정입니다.

2. 표현학습(Representation Learning)과 임베딩(Embedding)이란?

표현학습은 데이터 안에 숨겨진 의미 있는 특징(표현)을 스스로 학습하는 방법입니다.
임베딩(Embedding)은 데이터를 의미를 담은 숫자 벡터로 변환한 결과입니다.

1) 다양한 비정형 데이터

텍스트
고객 리뷰, 뉴스 기사,
이메일, SNS 게시글 등

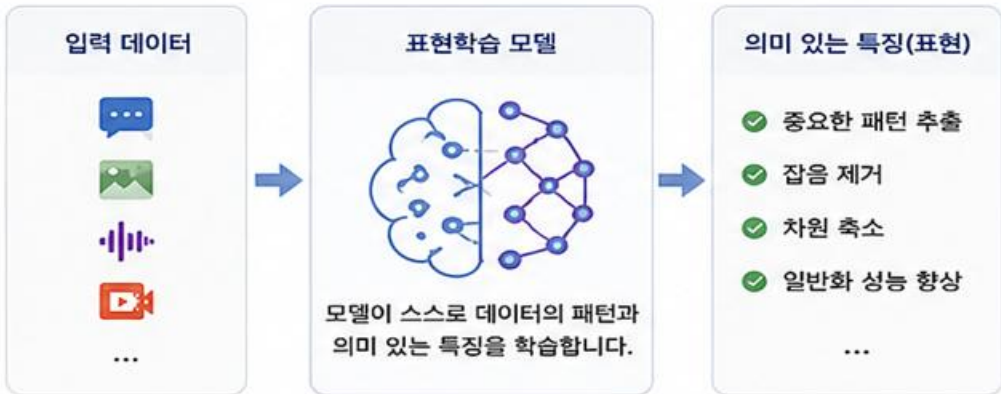
이미지
사진, 의료 영상, 도면,
CCTV 영상 등

오디오
음성, 음악, 효과음,
인터뷰 녹음 등

영상
동영상, 강의 영상,
실시간 스트리밍 등

형태는 다르지만, 컴퓨터가 이해하려면
모두 숫자 표현으로 변환해야 합니다.

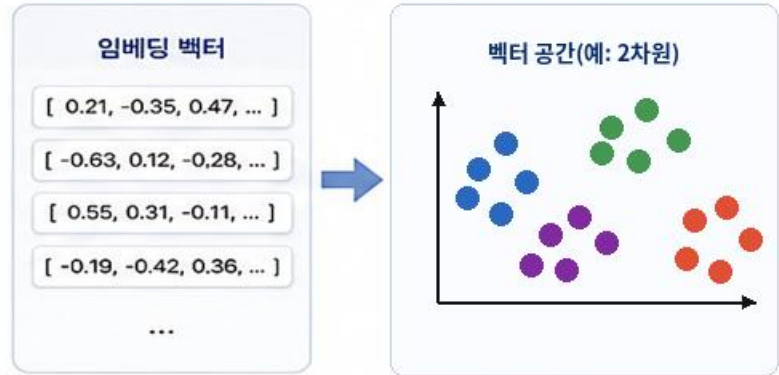
2) 표현학습 과정



사람이 일일이 특징을 설계하지 않아도,
모델이 데이터로부터 유용한 표현을 자동으로 학습합니다.

3) 임베딩(Embedding)이란?

학습된 표현을 의미 있는 숫자 벡터로 변환한 결과입니다.



벡터 공간에서는 의미가 비슷한 데이터가 가까운 위치에 모이고,
의미가 다른 데이터는 서로 먼 위치에 배치됩니다.

임베딩의 핵심 포인트

임베딩은 데이터의
의미를 보존하면서
계산 가능한 숫자 벡터로
바꾸는 과정입니다.

의미 비교 가능
벡터 간 거리/유사도로
문장, 이미지, 음성의
의미 비교가 가능합니다.
예) 코사인 유사도

검색과 추천 가능
비슷한 문서, 이미지,
상품, 사용자를
찾고 추천할 수 있습니다.
예) 유사 상품 추천

분류 성능 향상
핵심 특징이 압축된 벡터를
사용하여 모델이 더 잘
학습하고 예측합니다.
예) 감정 분류

멀티모달 연결
텍스트·이미지·오디오를
공통 벡터 공간에 연결해
함께 검색·분석할 수 있습니다.
예) 이미지-텍스트 검색

차원 축소 및 효율성
고차원 데이터를 낮은 차원의
벡터로 변환하여 저장·계산
비용을 줄일 수 있습니다.
예) 백터 DB

핵심 메시지 표현학습은 데이터 안의 의미 있는 특징을 학습하는 과정이고,
임베딩은 그 특징을 숫자 벡터로 변환하여 AI가 활용할 수 있게 만드는 과정입니다.



3. 임베딩이 필요한 이유

텍스트, 이미지, 오디오를 숫자 벡터로 변환하면 **AI가 의미를 이해하고** 다양한 작업을 수행할 수 있습니다.

핵심 메시지 **임베딩(Embedding)**은 비정형 데이터를 **의미를 담은 숫자 벡터로 변환하여** AI가 계산하고 학습할 수 있게 해주는 핵심 과정입니다.

1. 임베딩을 사용하면 무엇이 가능할까?

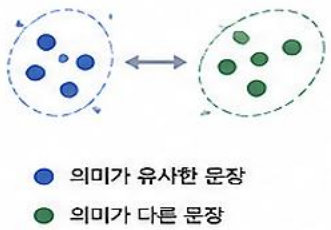
1 AI 모델 입력으로 변환

텍스트, 이미지, 오디오를
모델이 계산 가능한
숫자 벡터로 변환합니다.



2 의미 비교 가능

내용은 달라도 의미가
비슷하면 벡터 공간에서
가깝게 표현됩니다.



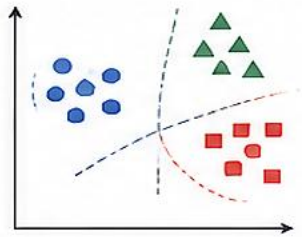
3 검색과 추천 가능

벡터 유사도를 이용해
비슷한 문서, 이미지, 상품,
사용자를 찾을 수 있습니다.



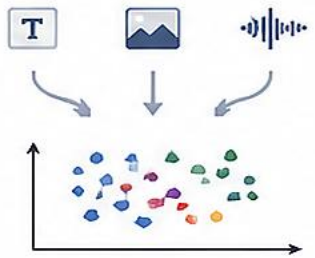
4 분류 성능 향상

데이터의 핵심 특징을
효과적으로 압축하여
분류 모델의 성능을 높입니다.



5 멀티모달 AI 구현

텍스트, 이미지, 오디오를
공통 벡터 공간에 연결하여
통합적으로 이해합니다.



6 다양한 다운스트림 작업 활용

분류, 군집, 생성, 요약,
감정 분석 등 다양한
작업에 활용됩니다.



2. 임베딩 전 vs 임베딩 후 비교 (예시)

임베딩 전 (원분 데이터)

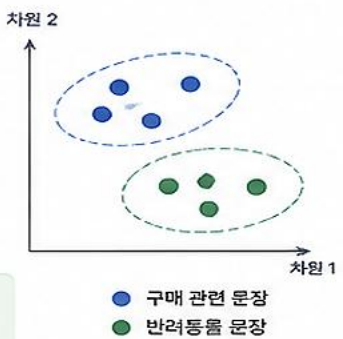
문장	특징
고객이 상품을 구매했다.	공통 단어: 없음 ✕
사용자가 제품을 결제했다.	공통 단어: 없음 ✕
강아지가 공원에서 뛰고 있다.	공통 단어: 없음 ✕

☹ 단이가 다르면 의미가 비슷해도
컴퓨터가 직접 비교하기 어렵습니다.

임베딩 후 (벡터 공간)

문장	벡터 (예시)
고객이 상품을 구매했다.	[0.21, -0.35, 0.47, 0.12, ...]
사용자가 제품을 결제했다.	[0.24, -0.31, 0.49, 0.10, ...]
강아지가 공원에서 뛰고 있다.	2, 0.15, -0.22, 0.31, ...]

☺ 벡터로 표현하면 의미가 비슷한 문장은
가까운 위치에 배치되어 비교가 쉬워집니다.



3. 활용 예시 (실생활/비즈니스)

검색 엔진	사용자의 질문과 가장 관련 있는 문서를 의미 기반으로 검색	예) "신용카드 분실 시 어떻게 하나요?"
상품 추천	사용자의 관심사와 유사한 상품을 추천	예) "이 상품을 본 고객이 함께 본 상품"
챗봇 / QA	질문 의도 파악 및 적절한 답변 검색	예) "환불 기간은 어떻게 되나요?"
데이터 분석	고객 리뷰 감성 분석, 주제 분류, 트렌드 파악	예) "부정 리뷰의 주요 원인 분석"
음성 인식 / 음악 추천	음성 내용 이해, 유사한 음악 스타일 추천	예) "비슷한 분위기의 음악 추천"
...		

정리 임베딩은 비정형 데이터를 **의미 있는 숫자 벡터로 변환**하여 AI가 이해·비교·검색·예측할 수 있게 하는 출발점입니다.

정적 임베딩 VS 문맥 임베딩

정적 임베딩 (Static)

단어가 항상 동일한 벡터를 가집니다.

예: Word2Vec, GloVe

"bank"는 강둑이든 은행이든 같은 벡터값을 가짐

문맥 임베딩 (Contextual)

문맥에 따라 같은 단어도 다른 벡터를 가집니다.

예: BERT, GPT 계열

주변 단어를 분석하여 "river bank"와 "bank account"를 구분함

정적 임베딩 VS 문맥 임베딩: 단어 의미 표현의 차이

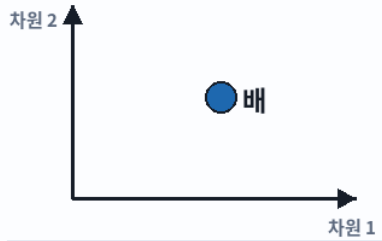
정적 임베딩 Static Embedding

같은 단어는 문맥이 달라도
항상 같은 벡터로 표현됩니다.

예시 문장

1. 맛있는 배 를 먹었습니다.
2. 항구에 커다란 배 가 정박했습니다.

벡터 표현



'배'라는 단어는
과일/운송수단 의미와 관계없이
하나의 고정 벡터를 가짐

대표 모델
Word2Vec, GloVe

VS

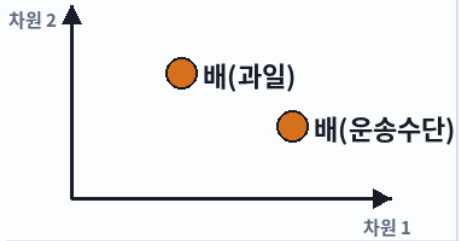
문맥 임베딩 Contextual Embedding

같은 단어도 문맥에 따라
서로 다른 벡터로 표현됩니다.

예시 문장

1. 맛있는 배 를 먹었습니다.
2. 항구에 커다란 배 가 정박했습니다.

벡터 표현



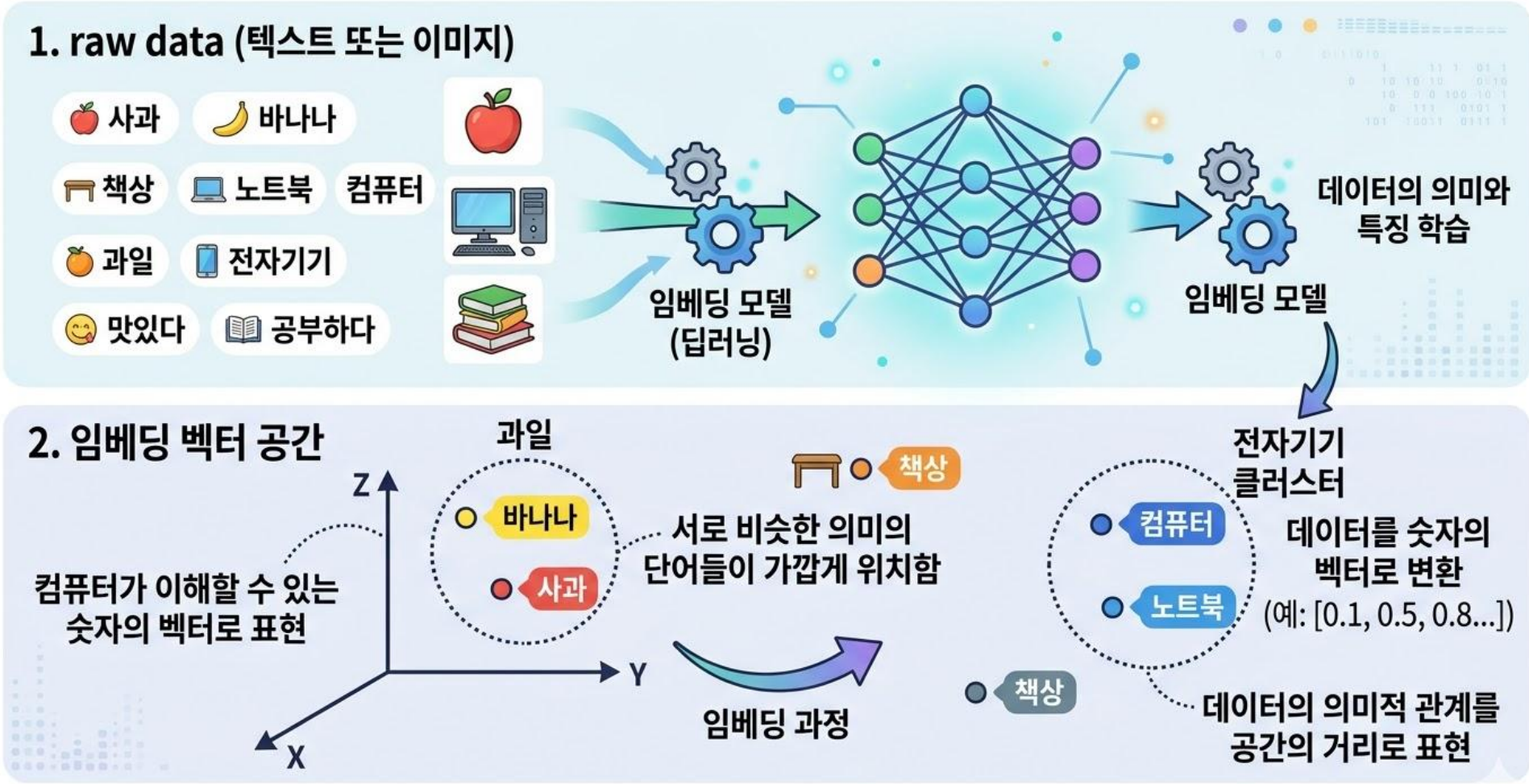
주변 단어를 보고
'먹다' → 과일 의미
'항구' → 운송수단 의미

대표 모델
BERT, ELMo, GPT

핵심: 정적 임베딩은 단어 자체를, 문맥 임베딩은 단어가 쓰인 문맥까지 함께 반영합니다.

임베딩(Embedding): 의미의 수치화

고차원의 복잡한 데이터를 저차원의 연속적인 벡터 공간으로 변환



임베딩: 데이터의 복잡한 특징을 보존하면서 고차원 의미 공간에 벡터로 매핑하는 기술

기계의 딜레마: 문장은 읽지 못하고, 숫자만 계산한다

그래서 텍스트를 AI가 계산할 수 있는 숫자 벡터로 바꿔야 합니다.

사람이 보는 문장

봄비가 내리는 고요한 새벽,
고양이가 창가에 앉아 있다.

사람은 뉘앙스와 문맥을 바로 이해합니다.

임베딩
Embedding

기계가 계산하는 표현

숫자 벡터

[0.21, -0.35, 0.47, 0.12, ...]

또는 0과 1의 배열로 처리

010011	011011	011001	010101	011100
010000	010010	001000	010110	011001

핵심 정리

1. 컴퓨터는 문장의 의미를 직접 이해하지 못합니다.
2. 문장을 숫자 벡터로 바꾸면 계산과 비교가 가능해집니다.
3. 이 변환 과정이 임베딩이며, AI 활용의 출발점입니다.

텍스트 → 임베딩 → 숫자 벡터 → AI 연산

거리로 의미를 표현하는 벡터 공간

단어를 숫자 벡터로 바꾸면, 단어 사이의 의미 관계를 거리와 방향으로 계산할 수 있습니다.

핵심 개념

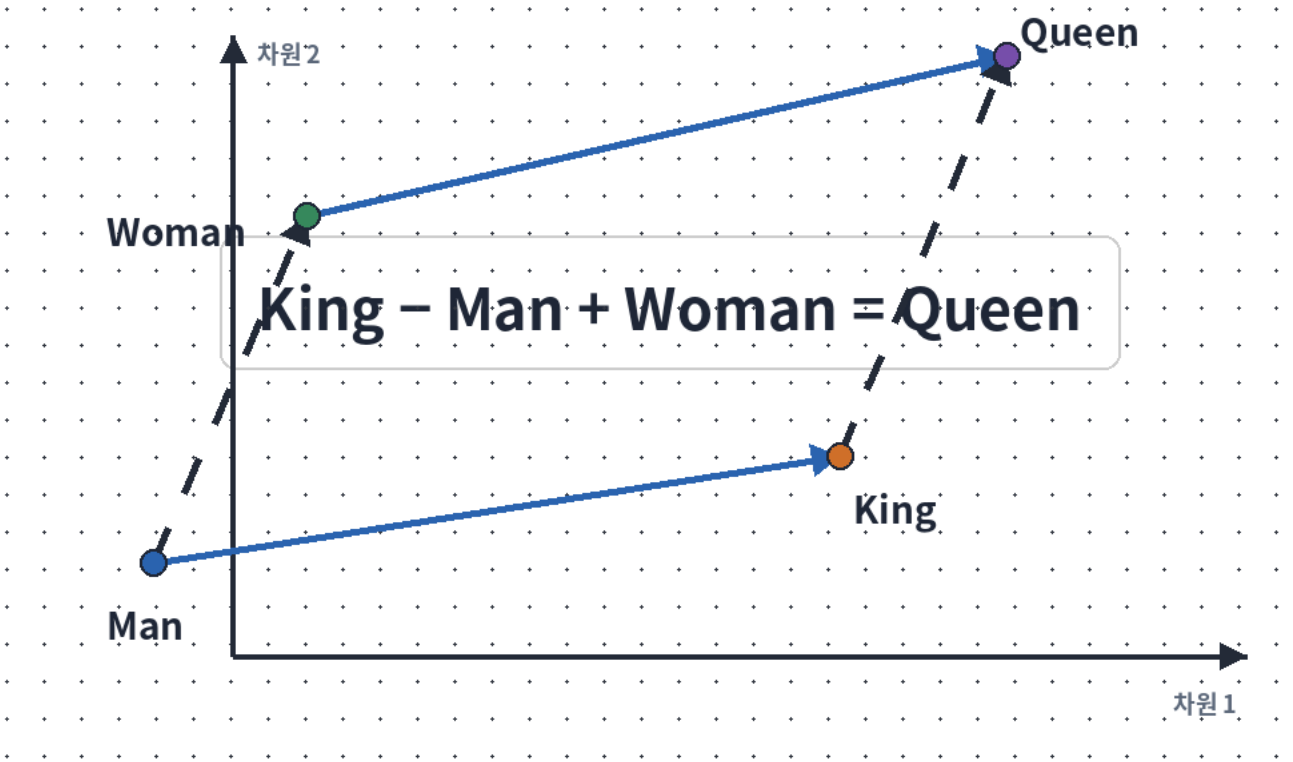
벡터 공간에서의 의미

단어 임베딩의 목표는
단어를 숫자 좌표로 배치하는 것입니다.

의미가 비슷한 단어는 가까운 위치에,
의미가 다른 단어는 먼 위치에 놓입니다.

예시 관계

King − Man + Woman ≈ Queen



핵심: 벡터 공간에서는 단어 사이의 거리와 방향이 의미 관계를 나타내며, AI는 이를 계산에 활용합니다.

단순 숫자 부여의 한계

숫자로 바꾸는 것만으로는 단어의 의미 관계를 표현하기 어렵습니다.

1. Integer Encoding

단어에 숫자를 하나씩 붙이는 방식



문제 **숫자의 차이가 의미의 차이를 뜻하지 않습니다.**

예: 2는 1과 3의 중간이지만,
강아지가 고양이와 자동차의 중간 의미는 아닙니다.

2. One-Hot Encoding

단어마다 하나의 위치만 1로 표시

1	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	0	0	0	1	0	0	0
0	0	0	0	0	0	0	0

1. **유사도 반영 불가** — 고양이와 강아지가 가깝다는 정보 없음
2. **차원 증가** — 단어 수가 많아질수록 벡터가 커짐
3. **대부분 0** — 저장·계산 효율이 낮아짐

핵심 정리 단순히 숫자를 붙이는 방식은 **의미 관계**를 보존하지 못합니다. 그래서 **임베딩**처럼 의미가 반영된 벡터 표현이 필요합니다.

단어 → 단순 숫자 ☒ / 의미 벡터 ✓

임베딩이란?: 복잡한 정보를 숫자 좌표로 바꾸기

예: 사람의 성격을 여러 숫자로 표현하면 AI가 비교하고 계산할 수 있습니다.

1 사람의 특징



프로필 카드

2 성격 벡터

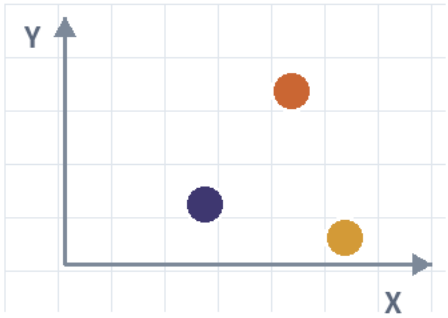
성격을 숫자 배열로 표현

0.78 | 0.62 | 0.72

예: [외향성, 친화성, 개방성]

복잡한 성격 → 계산 가능한 값

3 벡터 공간



가까울수록 비슷한 성향

핵심 정리

① 여러 속성을 숫자로 표현합니다. ② 숫자 배열 하나가 한 대상의 좌표가 됩니다. ③ 벡터 사이의 거리가 가까우면 의미도 비슷합니다.

임베딩 = 복잡한 정보를 의미를 보존한 숫자 벡터로 바꾸는 과정

단어 벡터: 단어를 숫자 패턴으로 표현하기

단어의 의미는 숫자 벡터의 패턴으로 표현되고, 비슷한 의미일수록 패턴도 비슷합니다.

단어
King, Woman, Water

→

숫자 벡터
[0.50, 0.69, ...]

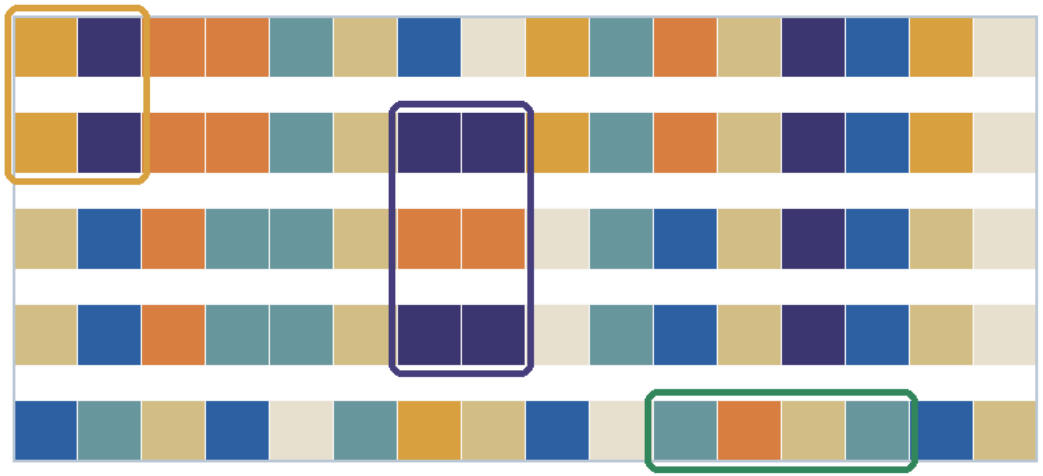
→

패턴 비교
의미 유사도 계산

단어 목록

왕 (King)
여왕 (Queen)
남성 (Man)
여성 (Woman)
물 (Water)

색 막대 = 단어 벡터



패턴이 의미를 담음

왕·여왕
일부 패턴이 비슷함

남성·여성
성별 관련 차원이 다름

물(Water)
사람 관련 단어와 패턴이 다름

핵심 정리

단어 벡터는 단어를 숫자 패턴으로 바꾸는 방법입니다.
패턴이 비슷하면 의미도 비슷하다고 계산할 수 있습니다.

단어 → 숫자 벡터 → 패턴 비교 → 의미 유사도

벡터 연산: 의미 관계를 계산하다

단어가 벡터 공간의 좌표가 되면, 의미 관계를 덧셈과 뺄셈으로 다룰 수 있습니다.



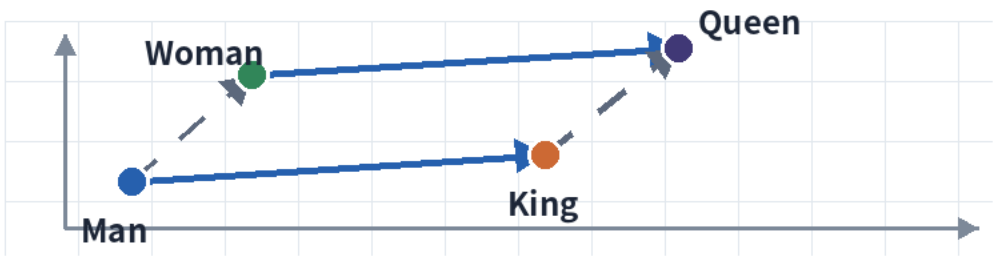
King − Man + Woman ≈ Queen

연산의 의미

King − Man	왕에서 '남성성'을 빼고
+ Woman	'여성성'을 더하면
≈ Queen	여왕과 가까운 벡터가 됩니다.

핵심: 벡터의 방향과 거리가 단어 간 의미 관계를 나타냅니다.

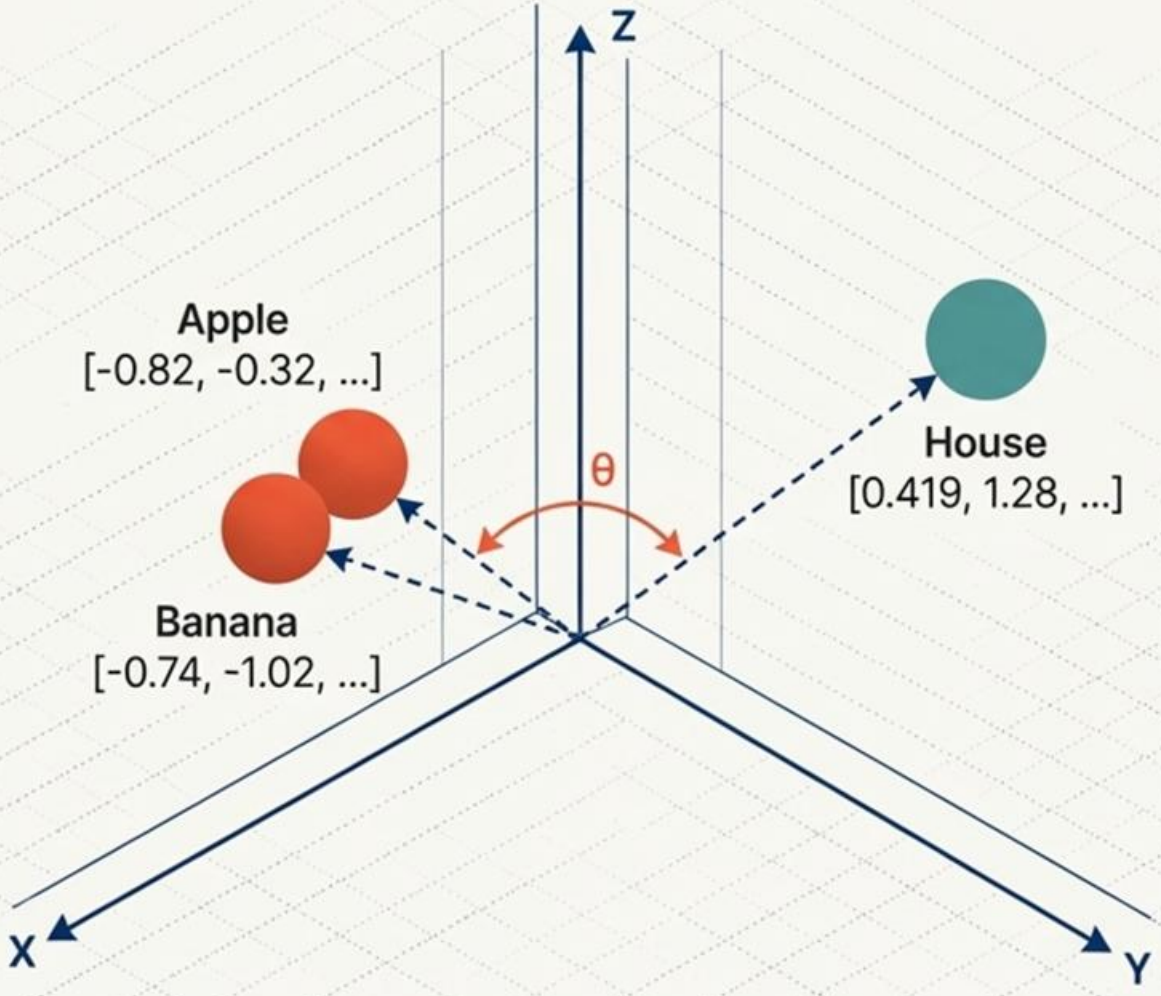
벡터 공간에서 보기



핵심 정리 단어 벡터는 단어를 좌표로 표현하므로, AI는 벡터 연산으로 의미 관계를 추론할 수 있습니다.

엔진 해부: AI는 언어를 어떻게 이해하는가? (임베딩)

고차원의 복잡한 데이터를 저차원의 연속적인 벡터 공간으로 변환



벡터화 (Vectorization)

복잡한 언어를 고차원 숫자 배열로 압축하여 공간의 좌표로 변환합니다.

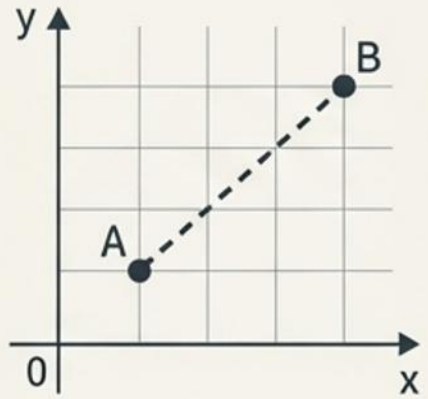
코사인 유사도 (Cosine Similarity)

두 벡터 사이의 각도(θ)가 작을수록
의미가 유사합니다.
벡터의 크기보다 방향성이 핵심입니다.

단어와 문장은 수천 차원의 공간 속 좌표가 되며,
'거리가 가깝다'는 것은 '의미가 같다'는 것을 뜻합니다.

공간의 거리를 측정하는 3가지 공식

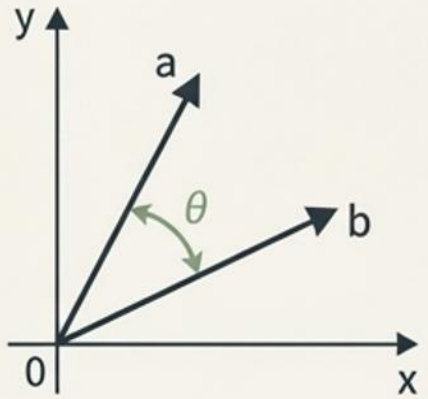
임베딩된 벡터들 사이의 유사도를 측정하여 단어 간의 관계를 수치화합니다.



Euclidean Distance (유클리디안 거리)

$$||a - b||$$

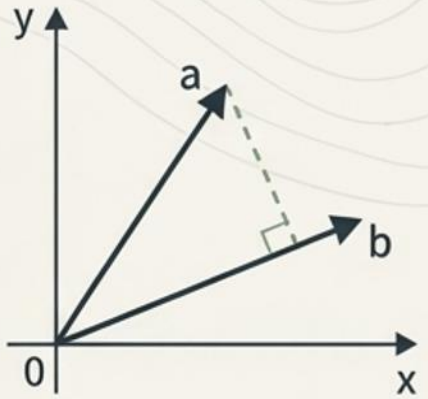
- 의미: 두 벡터의 끝점 사이의 물리적 직선 거리.
- 특징: 차원이 커질수록 거리가 수렴하여 대규모 임베딩에서는 신뢰도가 떨어짐 (음의 상관관계).



Cosine Similarity (코사인 유사도)

$$(a \cdot b) / (||a|| * ||b||)$$

- 의미: 두 벡터가 이루는 각도의 코사인 값.
- 특징: 벡터의 크기(단어의 출현 빈도)를 무시하고 오직 방향(의미)만 측정하므로 편향이 적어 가장 널리 쓰임.



Dot Product (내적)

$$a \cdot b$$

- 의미: 코사인 유사도에 두 벡터의 크기를 곱한 값.
- 특징: 방향뿐만 아니라 크기를 포함하므로, 훈련 데이터에 자주 등장하는(Popular) 단어에 더 높은 가중치를 부여함.

단어의 의미는 '이웃'이 결정한다

"단어의 의미는 그것이 쓰이는
주변 환경(Company)에 의해 알 수 있다."

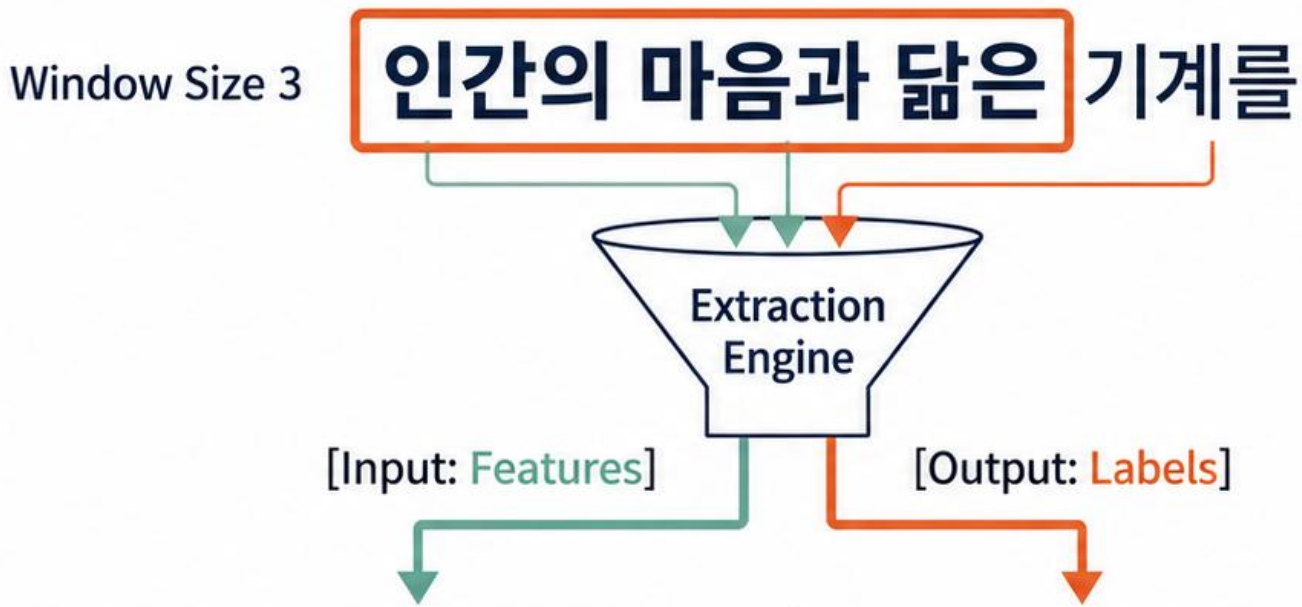
— J.R. Firth (1957)

인간의 [Context: 주변 단어] 마음 과 [Target: 중심 단어] 닮은 [Context: 주변 단어] 기계 를 만들지 말라

기계가 단어의 맥락을 학습하는 방식은 Sliding Window(슬라이딩 윈도우) 기법에 기반합니다.
문장 위를 일정한 크기의 창문이 미끄러지듯 이동하며, 어떤 단어들이 서로 이웃하여 등장하는지 관찰합니다.

비지도 학습을 지도 학습으로: 컨텍스트 추출

언어 모델의 가장 큰 장점은 **인간의 라벨링 노동 없이도**
엄청난 양의 텍스트 데이터에서 **스스로 학습 데이터**를 생성할 수 있다는 것입니다.



Features (주변 단어)	Label (중심 단어)
[인간의, 닮은]	[마음과]
[마음과, 기계를]	[닮은]

개념 설명

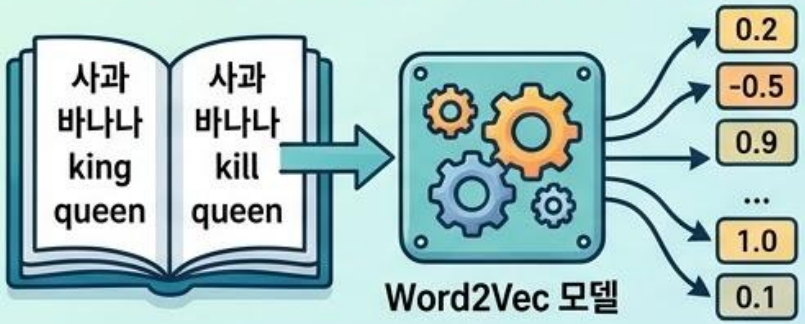
- **원시 텍스트 (Raw Text):**
  인공적인 라벨이 없는 자연어 문장.

- **가상 데이터셋 (Virtual Dataset):**
  원도우가 이동함에 따라, 기계는 앞의 단어들(입력)을 통해 중심 단어(정답)를 맞추는 수천만 개의 퀴즈를 스스로 생성해 냅니다.

쉽게 이해하는 Word2Vec (Word to Vector)

1. 핵심 아이디어: 단어를 숫자로!

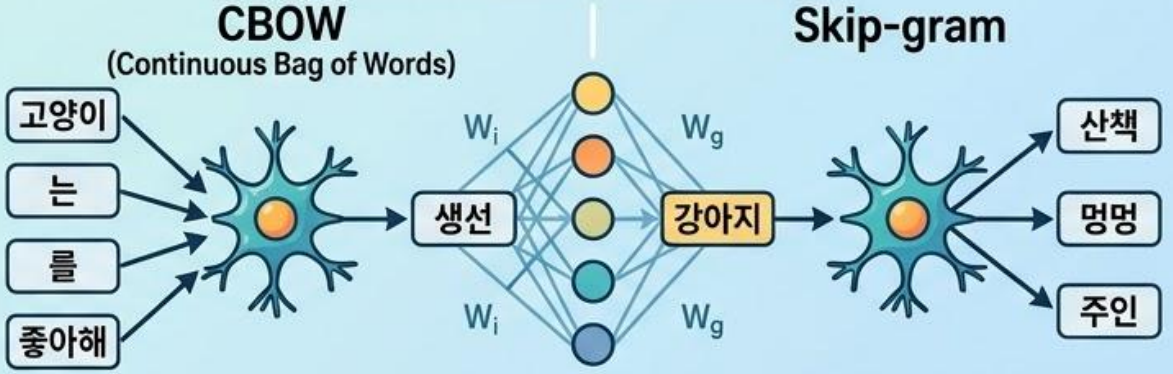
단어의 의미를 '숫자 배열(벡터)'로 표현해요.



단어의 의미를 '숫자 배열(벡터)'로 표현해요.



2. 어떻게 작동할까요?



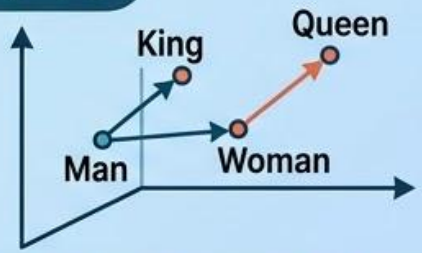
주변 단어들로 중앙의 단어를 예측해요.

중앙의 단어로 주변 단어들을 예측해요.

3. 벡터 공간에서의 특징



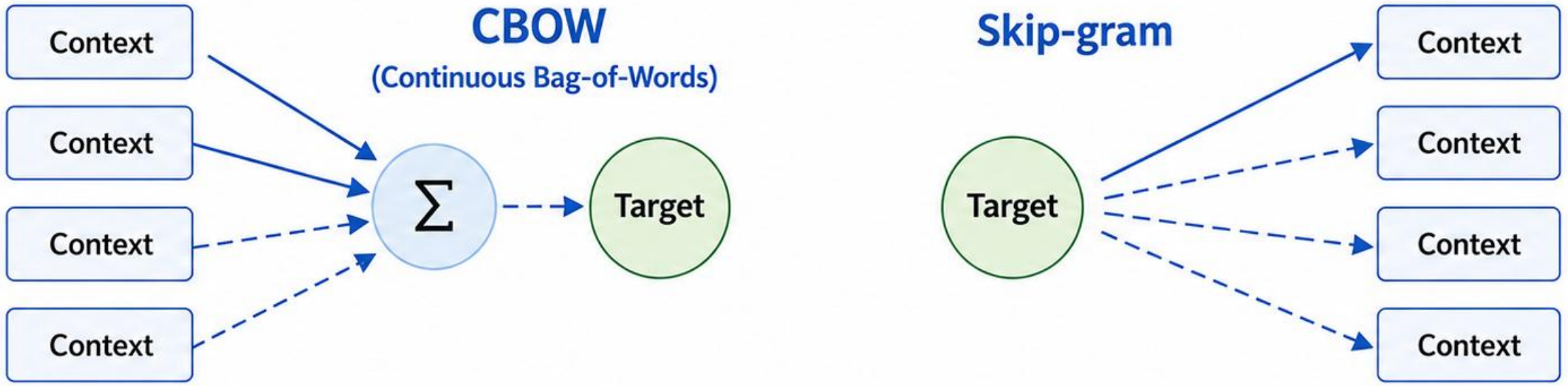
비슷한 의미의 단어는
가까이 위치해요.



'King - Man + Woman = Queen'
같은 관계가 성립해요!

단어를 벡터 공간에 매핑하다: Word Embeddings

의미가 비슷한 단어는 다차원 공간에서 기하학적으로 가까운 곳에 위치합니다 (코사인 유사도).



CBOW (Continuous Bag-of-Words)
주변 문맥 단어들의 벡터 합을 이용해 중심 단어를 예측.
(빠른 학습 속도)

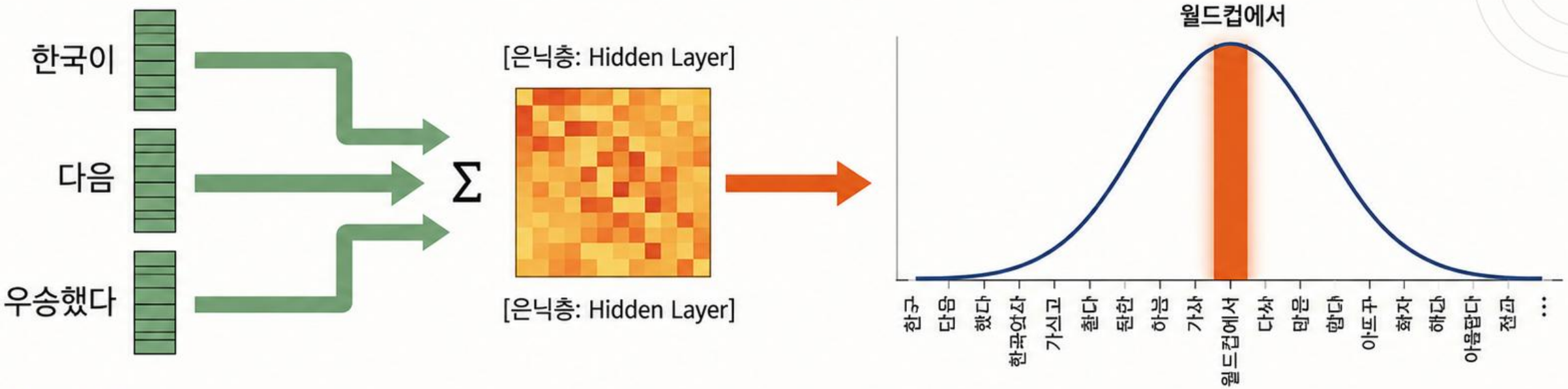
Skip-gram
중심 단어를 통해 주변 문맥 단어들을 예측.
(희귀 단어 처리에 탁월함)

벡터 벡터 연산 보존

King - Man + Woman = Queen

Architecture 1: CBOW (주변이 중심을 결정하다)

주변 단어(Context)를 결합하여 하나의 중심 단어(Target)를 예측합니다.



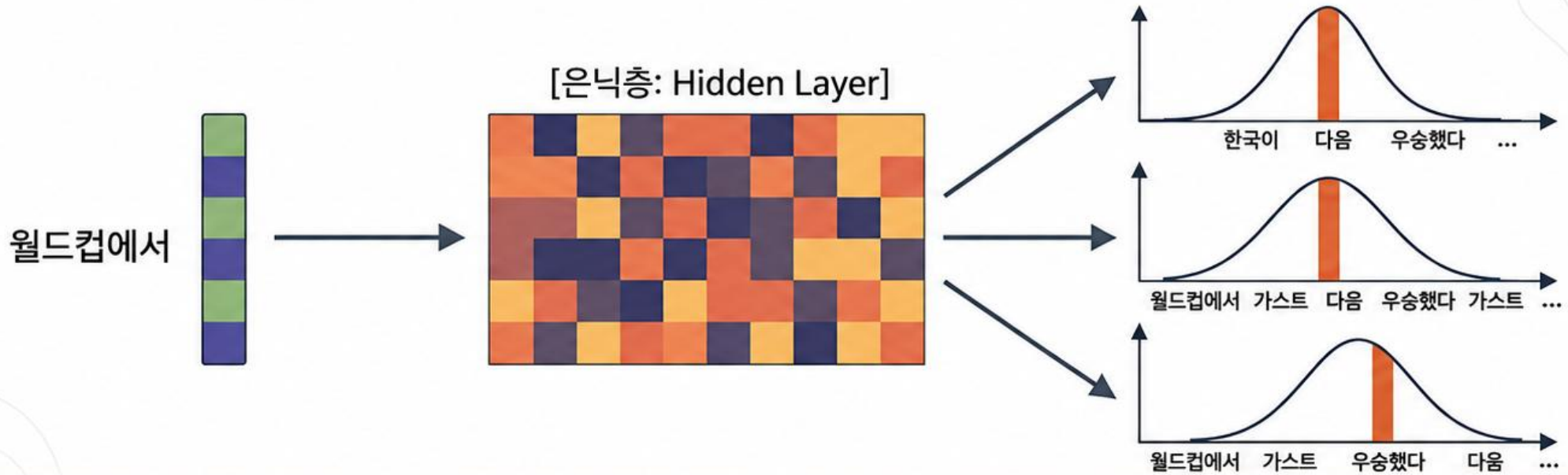
Annotations

- **예시** (Window Size = 3): [‘한국이’, ‘다음’, ‘우승했다’] -> Target: ‘월드컵에서’
- **학습 과정:** 여러 개의 컨텍스트 단어 벡터가 은닉층에서 평균/합산되어 하나의 벡터로 압축된 후, Softmax를 통해 목표 단어의 확률을 예측합니다.
- **권장** Window Size: 5

강점: 한 번의 예측으로 여러 주변 단어를 처리하므로 학습 속도가 빠르며,
자주 동장하는(Frequent) 단어의 구문론적 관계 학습에 유리합니다.

Architecture 2: Skip-Gram (중심이 주변을 예측하다)

하나의 중심 단어(Target)를 바탕으로 여러 개의 주변 단어(Context)를 예측합니다.



- 예시: Target: '월드컵에서' -> Contexts: ['한국이', '다음', '우승했다']
- 학습 과정: 타켓 단어 한 개당 원도우 크기만큼의 여러 독립적인 예측 오류(Error)를 계산하여 모델을 업데이트합니다. 데이터셋이 훨씬 더 많이 생성됩니다.
- 권장 Window Size: 10

강점: 하나의 단어로 여러 번의 가중치 업데이트가 발생하므로, 희귀한(Rare) 단어를 처리하고 깊은 의미(론적(Semantic) 관계를 파악하는 데 압도적으로 우수합니다.

완벽 비교: CBOW vs. Skip-Gram

CBOW		Skip-Gram
다수의 Context로 하나의 Target 예측		하나의 Target으로 다수의 Context 예측
빈출 단어(Frequent words), 구문론적 관계		희귀 단어(Rare words), 의미론적 관계
빠름 (컨텍스트 윈도우당 1회 예측)		느림 (타켓 단어당 다수 예측으로 계산량 증가)
적은 데이터셋에서도 잘 동작함		대규모 데이터셋이 필요하며, 큰 모델 사이즈
보통 크기 (2~5 단어)		넓은 크기 (5~20 단어)

연산의 병목과 해결책: 네거티브 샘플링 (Negative Sampling)

학습 단계에서 수십만 개의 단어 사전에 대한 확률을 매번 계산(Softmax)하는 것은 연산량이 너무 많습니다.

이를 해결하기 위한 혁신적인 트릭이 '네거티브 샘플링'입니다.

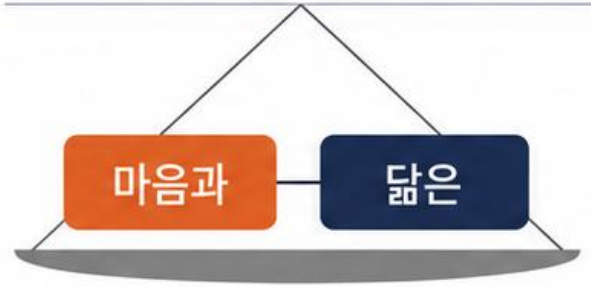


문제의 변환 (The Hack):

'다음 단어는 무엇인가?'라는 다중 클래스 분류를, '이 두 단어가 서로 이웃인가?'라는 단순한 이진 분류 문제로 바꿉니다.

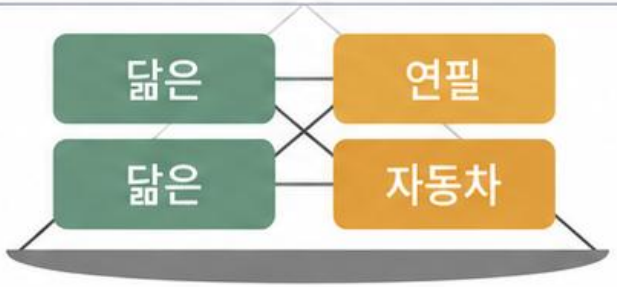
Balancing Scale

Signal (실제 이웃) -> 1



타켓 단어와 실제 주변 단어의 쌍. 모델은 '1'을 출력하도록 학습.

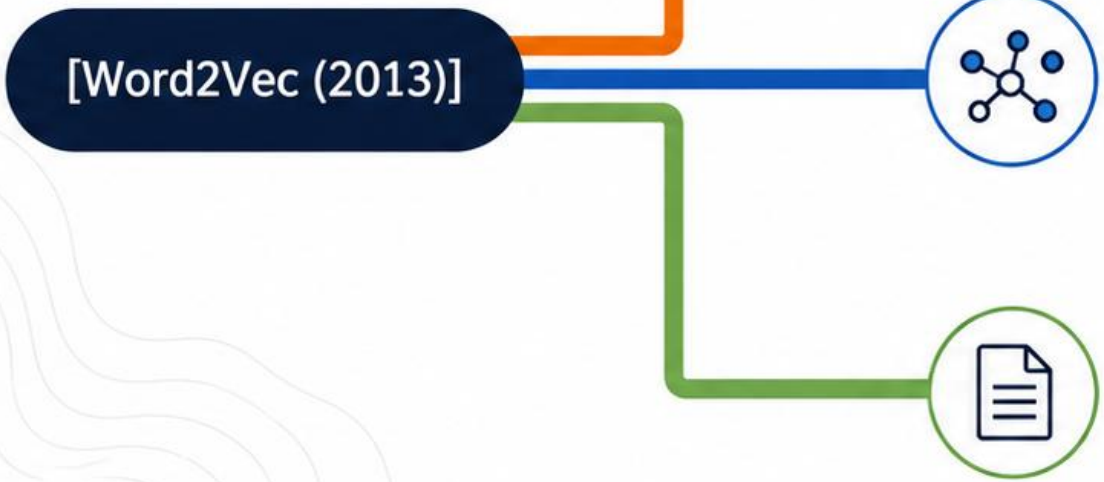
Noise (랜덤 추출) -> 0



사전에 있는 무작위 단어 5~20개를 오답 데이터로 제공. 무조건 '0'을 예측하는 꼼수를 방지합니다.

언어를 넘어선 임베딩 생태계

Word2Vec이 증명한 '연속된 시퀀스에서 이웃을 예측하여 톡징을 추출하는 구조'는
단어를 넘어 다양한 도메인으로 확장되었습니다.



Item2Vec (추천 시스템)

사용자 행동 시퀀스(조회, 클릭, 구매 등)에서 이웃 아이템을
예측하여 아이템 임베딩을 학습, 개인화 추천 성능을 향상시킵니다.

활용 분야: 이커머스, 콘텐츠 추천, 광고 최적화

Graph2Vec (그래프 임베딩)

그래프 구조에서 주변 노드/서브그래프를 예측하여 노드 또는 그래프의
저차원 벡터를 생성, 구조적 패턴을 효과적으로 표현합니다.

활용 분야: 소셜 네트워크 분석, 지식 그래프, 사기 탐지

Seq2Vec (시계열 임베딩)

연속된 시계열 데이터에서 미래 값을 예측하여 시계열의
특징을 벡터로 표현, 다양한 예측 모델의 입력으로 활용됩니다.

활용 분야: 수요 예측, 경제 지표 분석, IoT 이상 탐지

요약: 단순한 예측이 만들어낸 언어의 지형도

Word2Vec의 가장 아름다운 점은 그 설계 철학에 있습니다. 단어 예측이라는 **"가짜 목표"**를 수행하도록 신경망을 강제함으로써, 우리는 언어의 고차원적 지형도를 포착한 **"가치 있는 부산물"**을 얻게 됩니다.



1. 공간적 의미 부여

단어는 다차원 벡터 공간의 좌표가 되며, 거리가 곧 의미의 유사성을 나타냅니다.

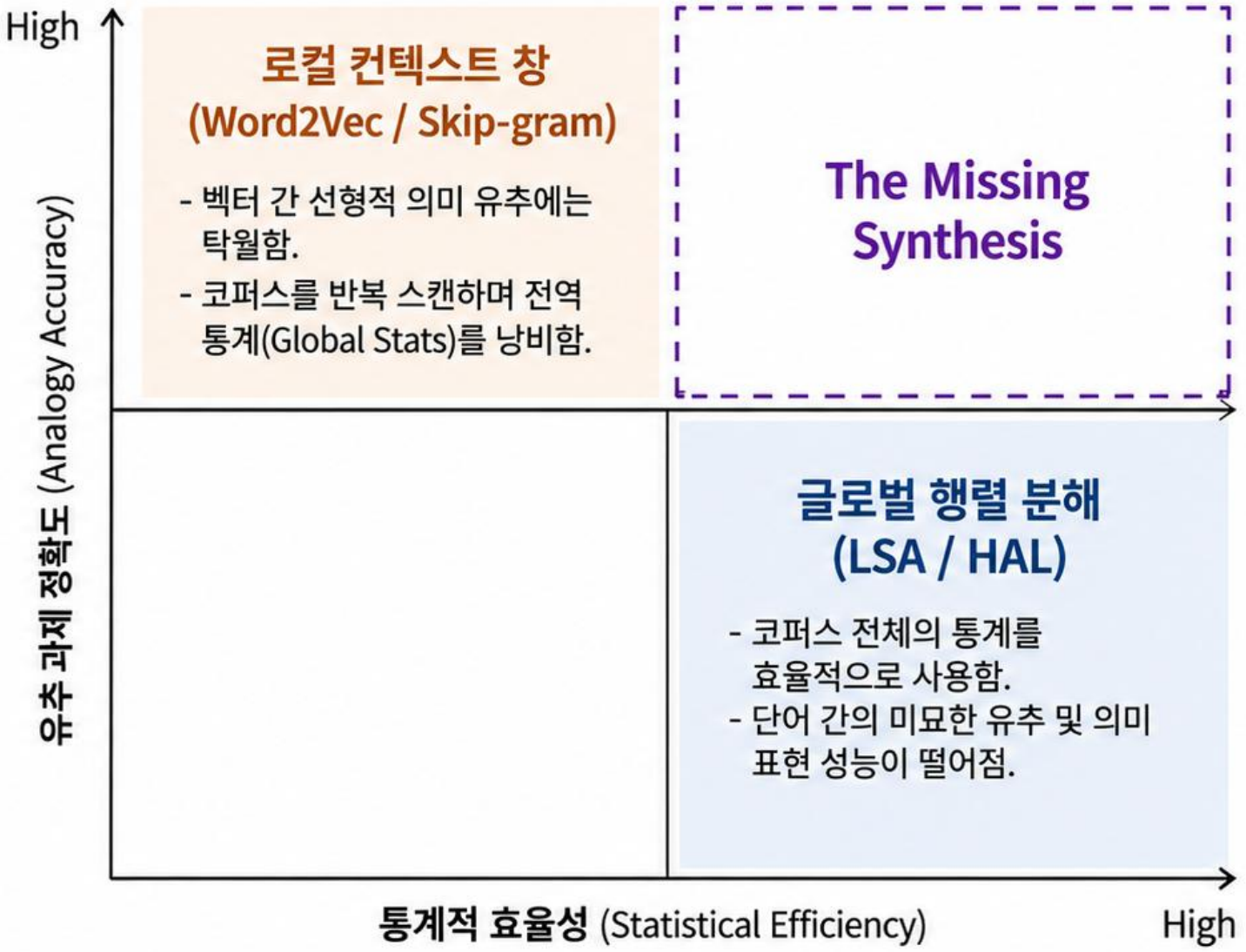
2. 비지도 학습의 마법

슬라이딩 윈도우를 통해 원시 텍스트 스스로가 무한한 학습 데이터를 만들어냅니다.

3. 최적화의 승리

네거티브 샘플링과 단순한 구조를 통해 연산 병목을 돌파하고, 현대 NLP를 탄생시킨 기반이 되었습니다.

단어 임베딩 모델의 딜레마

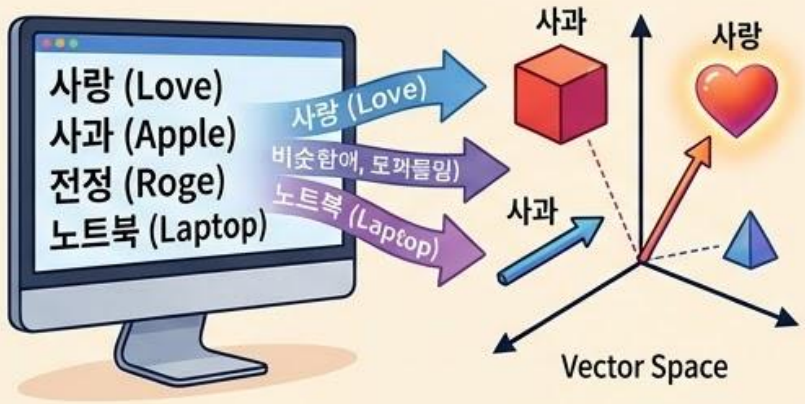


LSA: Latent Semantic Analysis, HAL: Hyperspace Analogue to Language

🧩 GloVe란 무엇인가요? (WHAT IS GloVe?)

GloVe의 전성을 이해하나기

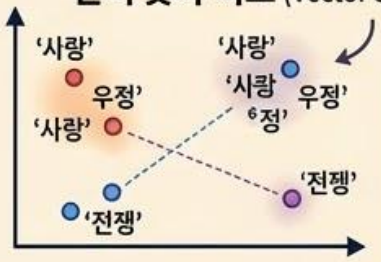
1. 컴퓨터가 단어의 의미를 배우는 방법



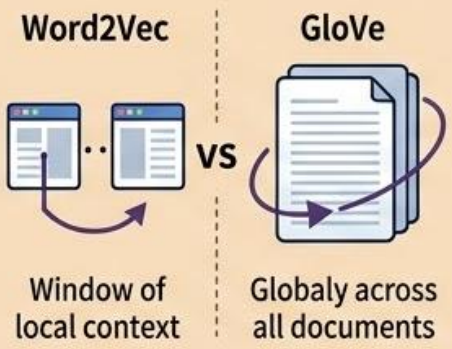
GloVe는 수많은 데이터를 학습하여, 의미가 비슷한 단어들을 서로 가까운 벡터(숫자)로, 의미가 다른 단어들은 먼 벡터로 바꿉니다.

단어 숫자 지도 (Vector Space Map)

단어 임베딩 (Word Embedding)

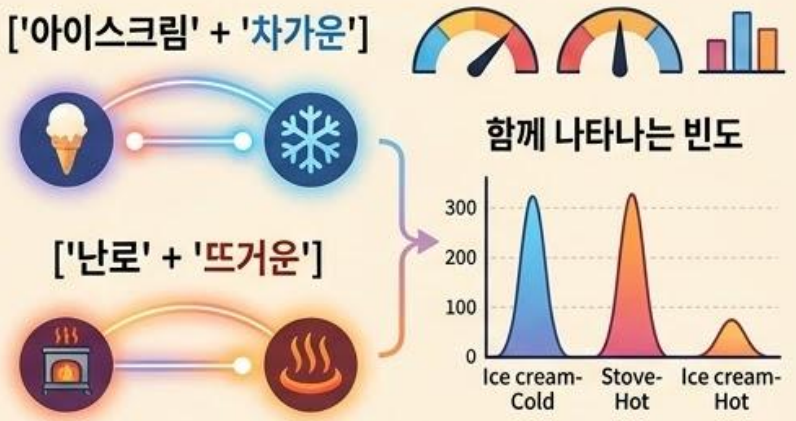


😎 왜 GloVe가 유용한가요?



- **전체적인 시야** (Global Vision)
- **효율적인 학습** (Efficient Learning)
- **더 나은 성능** (Better Performance)

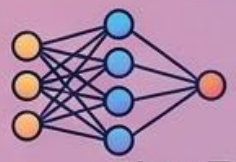
2. 핵심 아이디어: 단어들의 관계를 통계로 배운다





전체 말뭉치를 분석하여 단어 동시 등장 확률을 계산

💖 요약: 그래서 GloVe가 하는 일은?



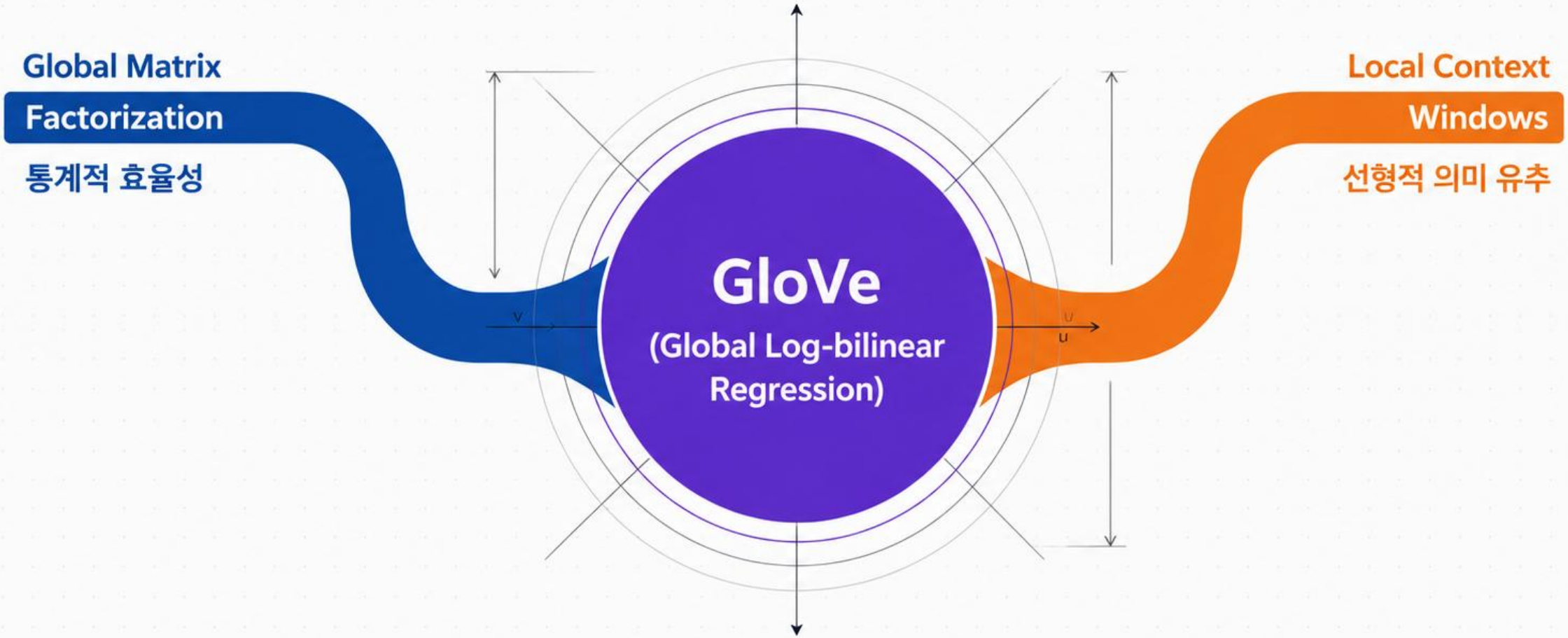
컴퓨터가 '사랑'과 '우정'은 비슷한 숫자로 만들고,
'사랑'과 '전쟁'은 정반대의 숫자로 만들도록 훈련시킵니다.



GloVe로 학습된 '단어 숫자 지도' 덕분에 컴퓨터가 언어를 더 잘 이해하게 됩니다.

GloVe의 동장: 완벽한 합성(Synthesis)

GloVe는 코퍼스 전체의 동시 동장(Co-occurrence) 행렬이 가진 '글로벌 통계'를 활용하면서도,
알은 신경망 모델들이 보여준 '선형적인 의미 방향성'을 동시에 추출해냅니다.



핵심 직관: 의미는 확률의 ‘비율(Ratio)’에서 파생된다

단일 확률값은 노이즈가 많습니다. 하지만 두 단어의 확률을 나누는 순간, 두 단어를 구분 짓는 핵심 속성(solid, gas)은 증폭되고, 공통적이거나 무관한 노이즈(water, fashion)는 1.0에 수렴하며 상쇄됩니다.

Probe Words (k)

	$k = \text{solid}$	$k = \text{gas}$	$k = \text{water}$	$k = \text{fashion}$
$P(k \mid \text{ice})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \mid \text{steam})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
Ratio: $\frac{P(k \mid \text{ice})}{P(k \mid \text{steam})}$	8.9	0.085	1.36	0.96

Ice의 핵심 속성 증폭

Steam의 핵심 속성 증폭

노이즈 상쇄 (~1.0)

직관을 아키텍처로 변환하는 파이프라인

GloVe 모델은 행렬의 빈도 로그값이 두 단어 벡터의 내적과 같아지도록 강제함으로써,
동시 동장 확률의 비율을 벡터 공간의 선형적 차이로 인코딩합니다.

1 동시 동장 행렬(X) 생성



	01	02	03	04	...	V0
01	0.4	0.2	0.3	0.4	...	0.9
02	0.2	0.3	0.4	0.6	...	0.3
03	0.0	0.3		0.4	...	0.5
04	0.3	0.5	0.3		...	0.7
...	...	...	...	...	...	...
V0	0.8	1.0	0.4	0.3	...	

X

- 방대한 텍스트 코퍼스를 스캔하여 전역 통계 행렬을 구축합니다.

2 비율의 로그 변환 및 내적 매핑

글로벌 카운트 로그값

$$w_i^T \tilde{w}_k + b_i + \tilde{b}_k = \log(X_{ik})$$

단어 및 컨텍스트 벡터

편향 (Symmetry 보존)

- 로그 비율을 내적과 편향의 합으로 매핑하여 비율 정보를 벡터 공간에 선형적으로 표현합니다.

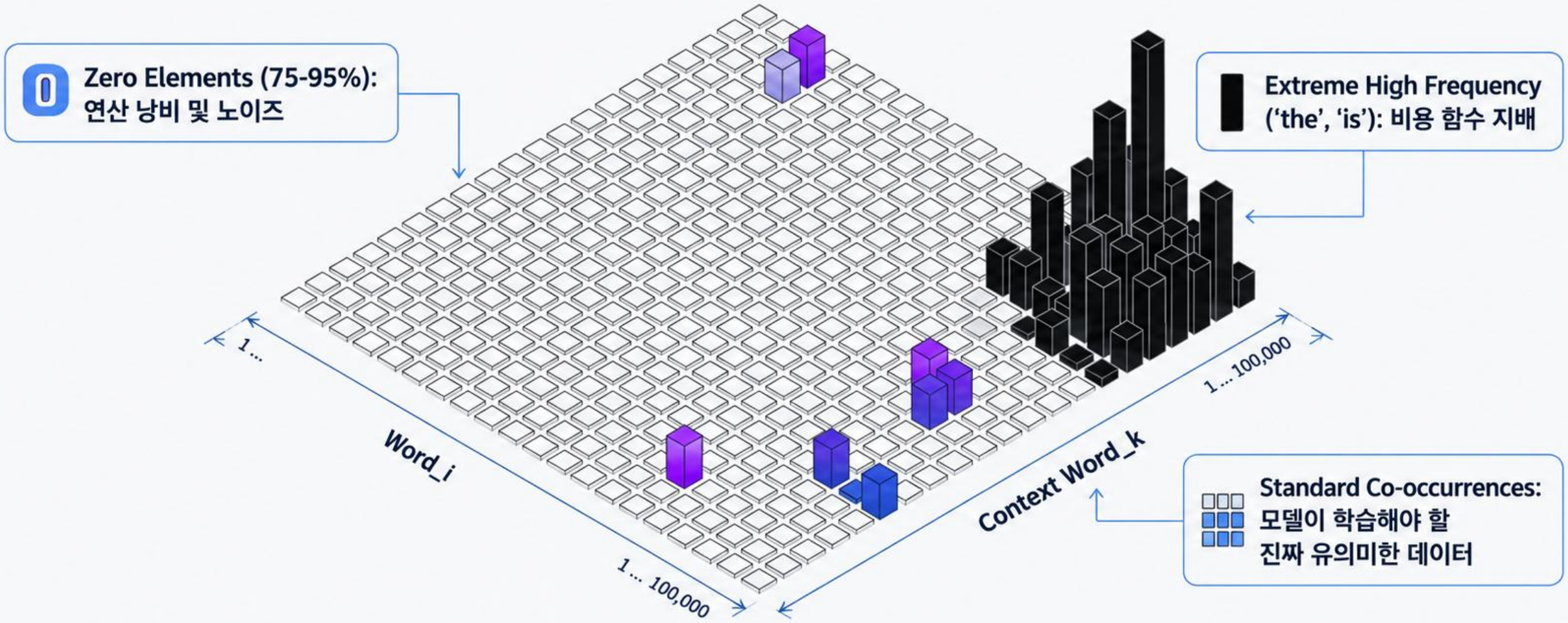
3 가중 최소 제곱법 (Weighted Least Squares)



- 희소성과 노이즈 문제를 해결하는 가중치 함수를 적용하여 비용 함수(Cost Function)를 최적화합니다.

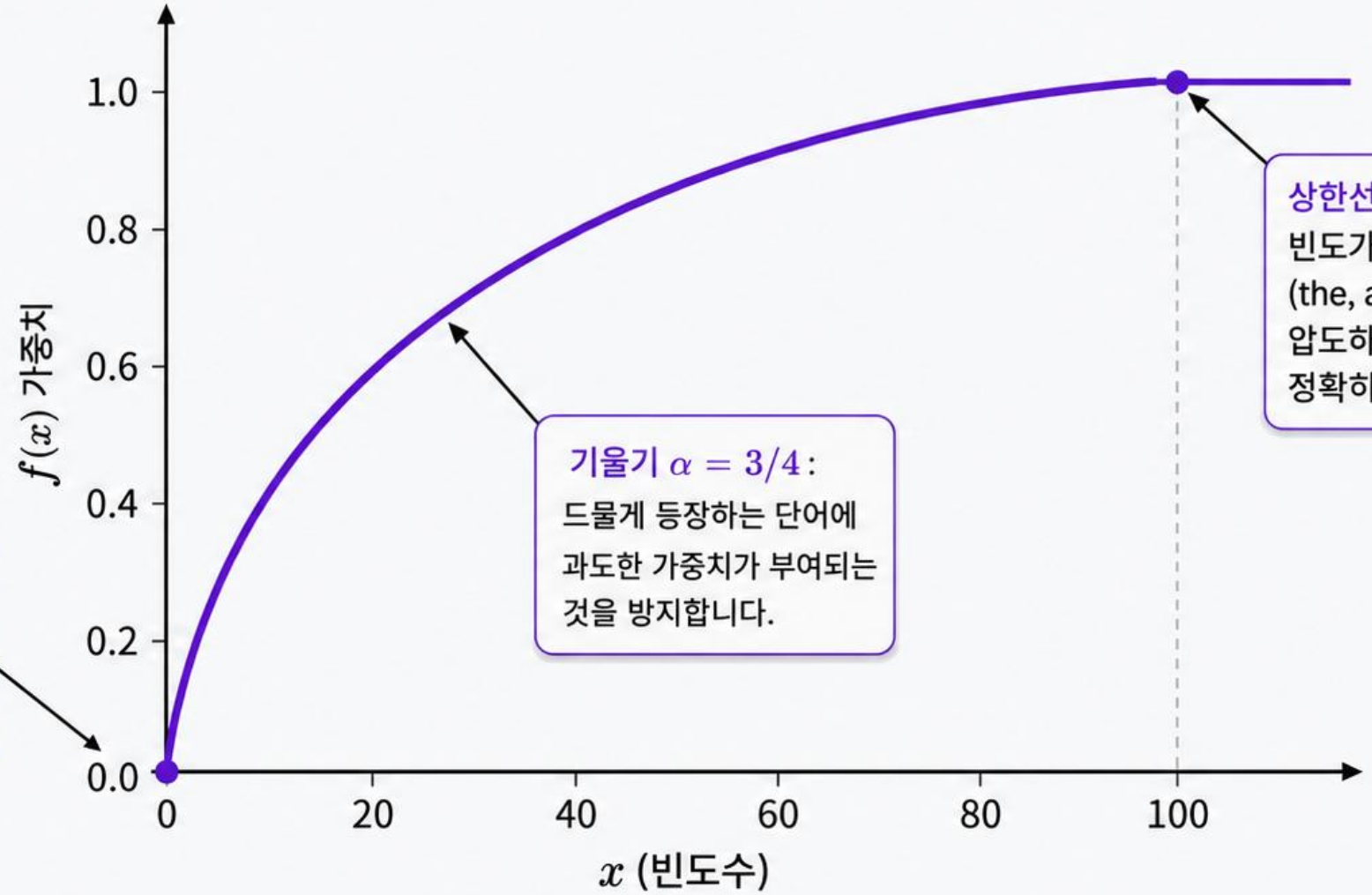
데이터 희소성과 노이즈의 함정

기존의 로그 모델(LSA 등)은 치명적인 결함이 있습니다.
행렬의 **75~95%**를 차지하는 **'0'의** 빈도를 과도하게 반영하며,
극도로 자주 동장하는 관사나 조사에 의해 비용 함수가
지배당합니다. 모든 빈도를 동일하게 취급할 수는 없습니다.



GloVe의 엔진: 가중치 함수 $f(x)$

GloVe는 빈도에 따른 가중치 함수 $f(x)$를 도입하여 극단적인 데이터 불균형 문제를 우아하게 해결합니다.



$f(0) = 0$:
절대 등장하지 않는 0의 값을 완전히 무시하여 방대한 연산량을 절약합니다.

기울기 $\alpha = 3/4$:
드물게 등장하는 단어에 과도한 가중치가 부여되는 것을 방지합니다.

상한선 컷오프 ($x_{max} = 100$) :
빈도가 지나치게 높은 단어 (the, and) 가 모델 전체를 압도하지 못하도록 영향력을 정확히 1.0으로 제한합니다.

계산의 우아함: 압축된 스케일링

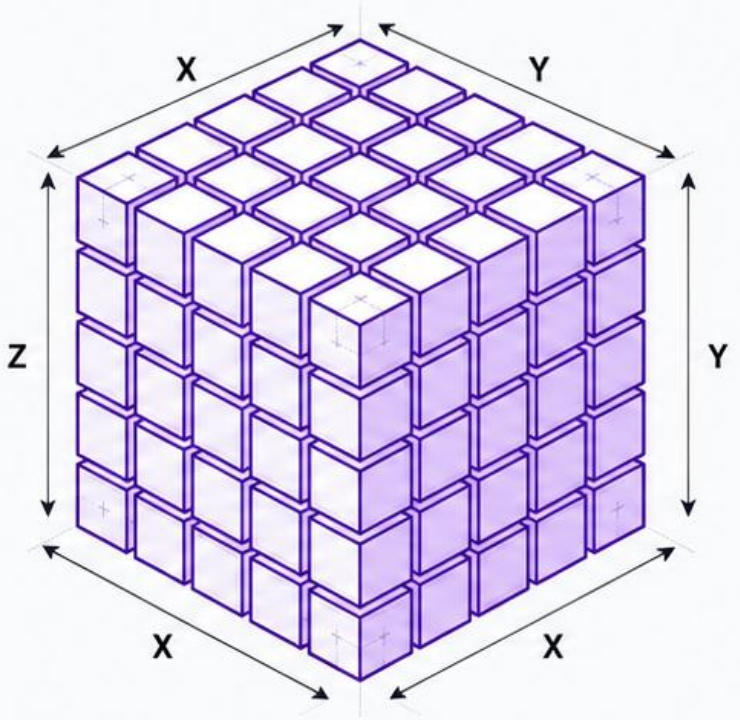
로컬 창 스캔 (Local Window Scanning)



복잡도: $O(|C|)$

Word2Vec과 같은 얕은 신경망은 전체 인터넷 텍스트(코퍼스 크기 $|C|$)를 창(Window)으로 처음부터 끝까지 반복 스캔해야 하므로 막대한 연산이 필요합니다.

글로벌 행렬 학습 (Global Matrix Optimization)



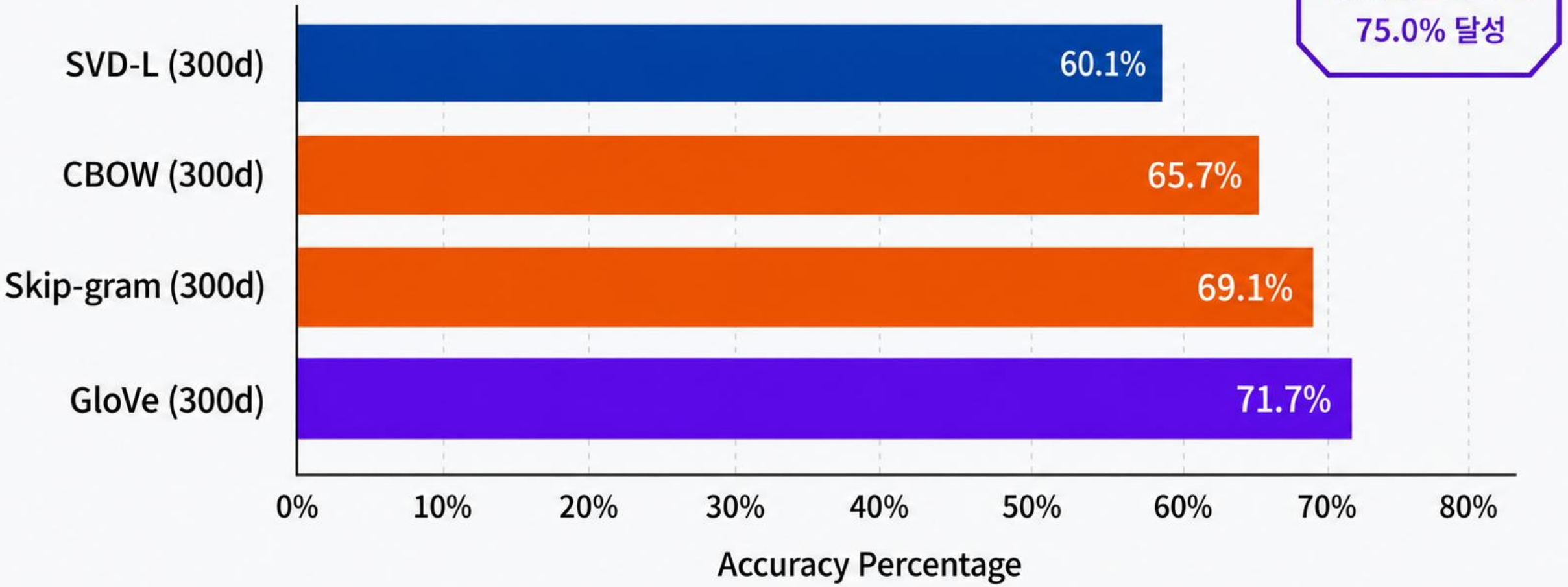
복잡도: $O(|X|) \approx O(|C|^{0.8})$

GloVe는 압축된 글로벌 행렬에서 '0이 아닌 유의미한 행렬 요소($|X|$)'만을 한 번에 학습합니다. 대규모 코퍼스일수록 연산 속도가 본질적으로 훨씬 빠릅니다.

성능 증명 1: 단어 유추 정확도

동일한 60억(6B) 토큰 코퍼스에서 학습했을 때, GloVe는 기존 행렬 분해(SVD) 방식과 Word2Vec(CBOW, SG)을 모두 압도하는 전체 유추 정확도를 기록했습니다.

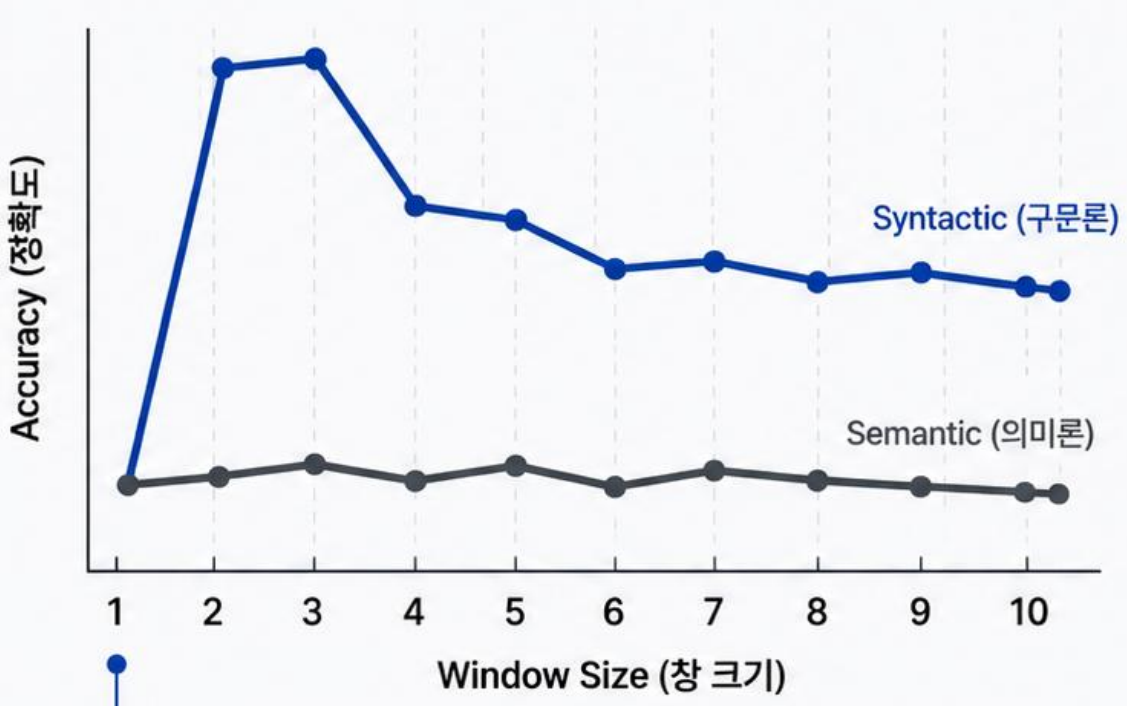
최대 스케일업:
42B 토큰 학습 시
75.0% 달성



컨텍스트의 뉘앙스: 구문론 vs 의미론

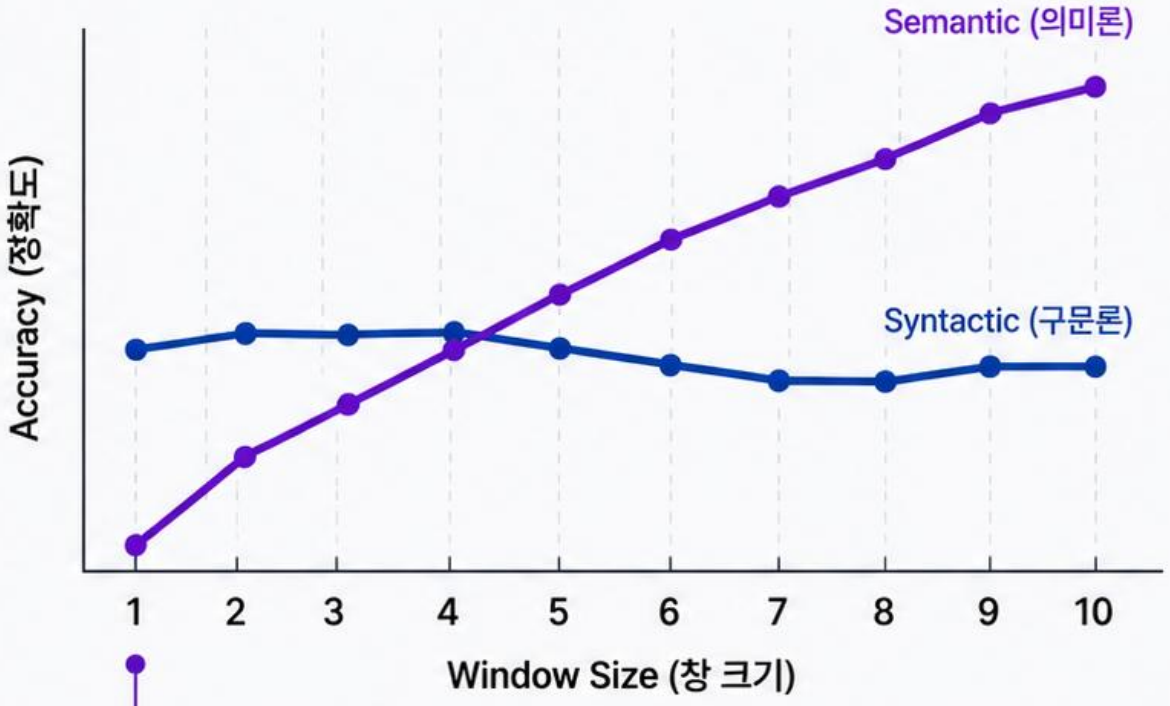
시각화된 데이터를 통해 언어의 구조를 엿볼 수 있습니다. 문법과 같은 '구문론적 정보'는 비대칭적이고 좁은 문맥에 의존하는 반면, 단어의 깊은 '의미론적 정보'는 넓고 대칭적인 문맥을 필요로 합니다.

비대칭형 작은 창 (Asymmetric / Small Window)



구문론(문법)은 바로 옆에 인접한 단어의 순서에 크게 의존함.

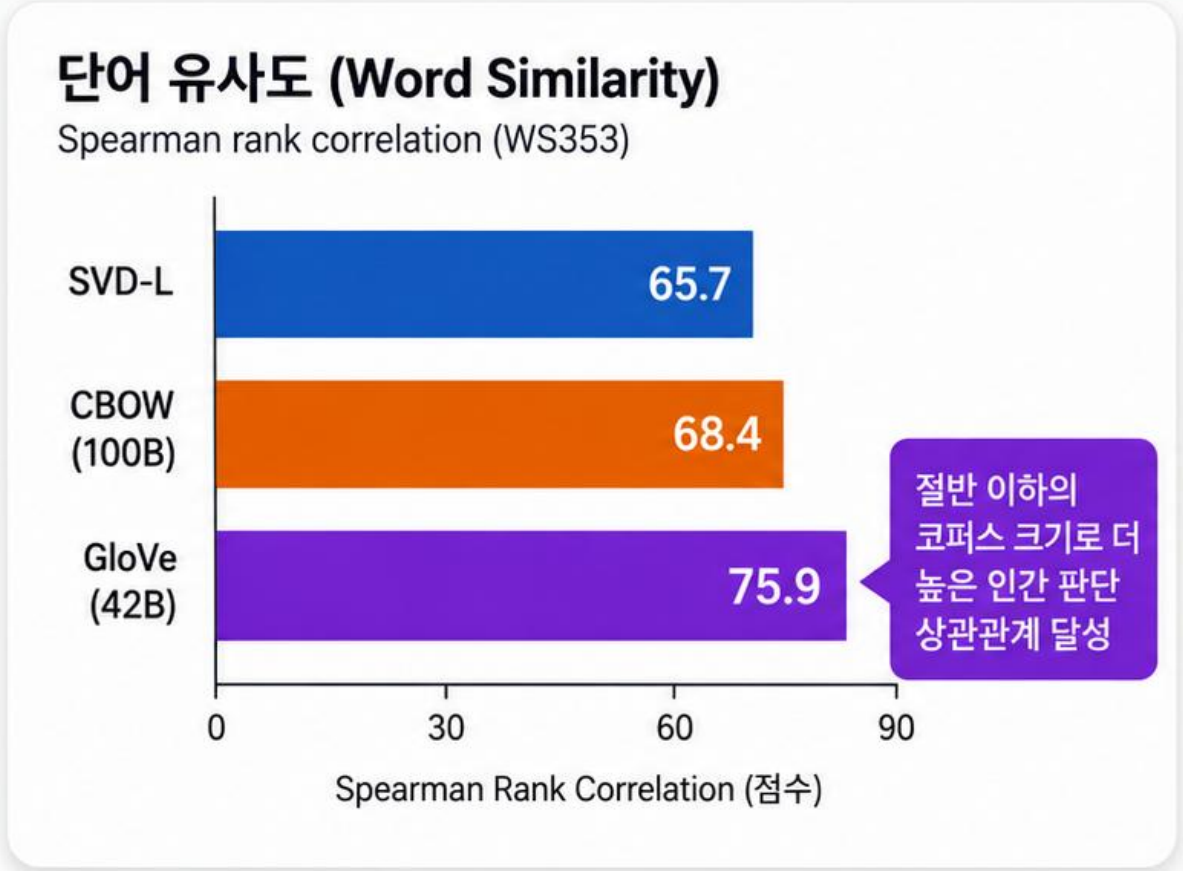
대칭형 큰 창 (Symmetric / Large Window)

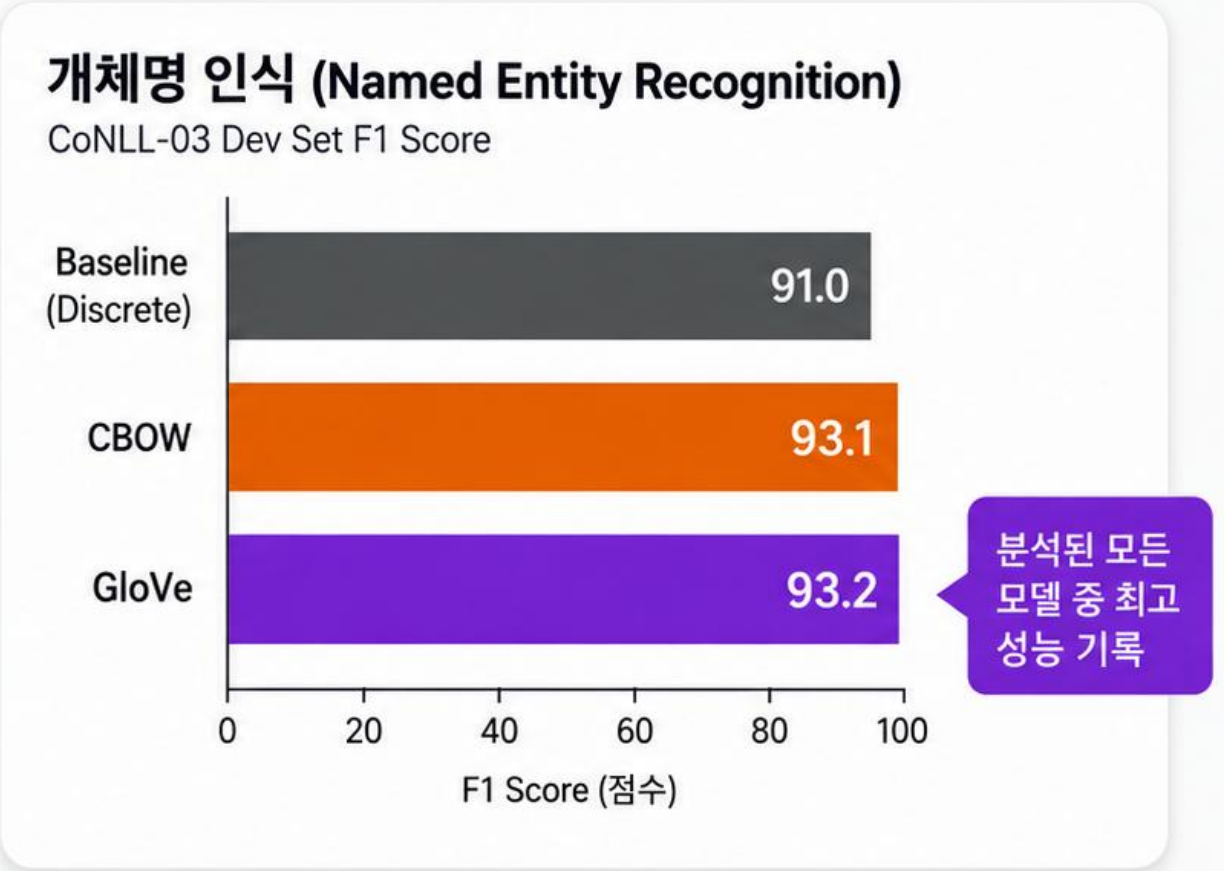


의미론(개념)은 멀리 떨어져 있는 양방향 문맥의 영향을 받아 형성됨.

성능 증명 2: 다운스트림 NLP 태스크

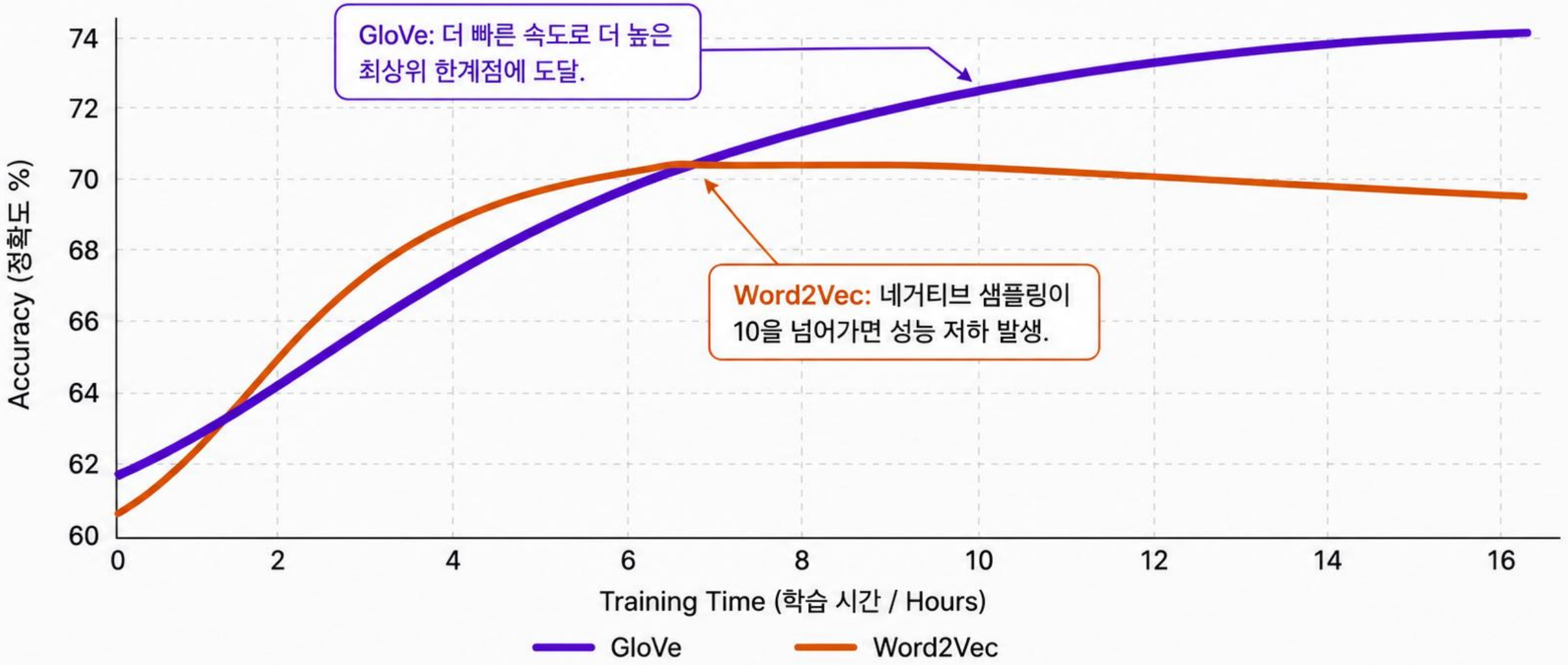
훌륭한 임베딩은 실제 애플리케이션에서 유용해야 합니다. GloVe는 유추 과제를 넘어
단어 유사도 측정과 개체명 인식(NER) 태스크에서도 현존하는 최적의 모델들을 능가합니다.





최후의 대결: GloVe vs Word2Vec

학습 시간 대비 정확도의 정면 대결. 동일한 조건(6B 토큰)에서 두 모델을 비교한 결과,
GloVe가 더 높은 최종 정확도에 도달할 뿐만 아니라 그 도달 속도 자체도 압도적으로 빠릅니다.

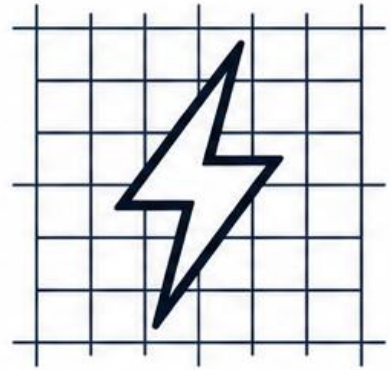


요약 및 핵심 시사점 (The Dialectic Synthesis)



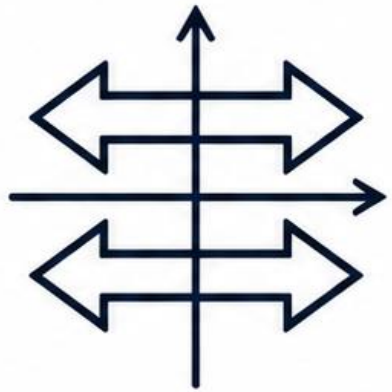
Count vs Predict는 적이 아니다

두 방식 모두 근본적으로 코퍼스의 동시 동장 통계를 탐구하는 다른 렌즈일 뿐입니다. 대립하는 두 진영의 장점을 결합할 수 있습니다.



글로벌 통계의 극대화된 효율성

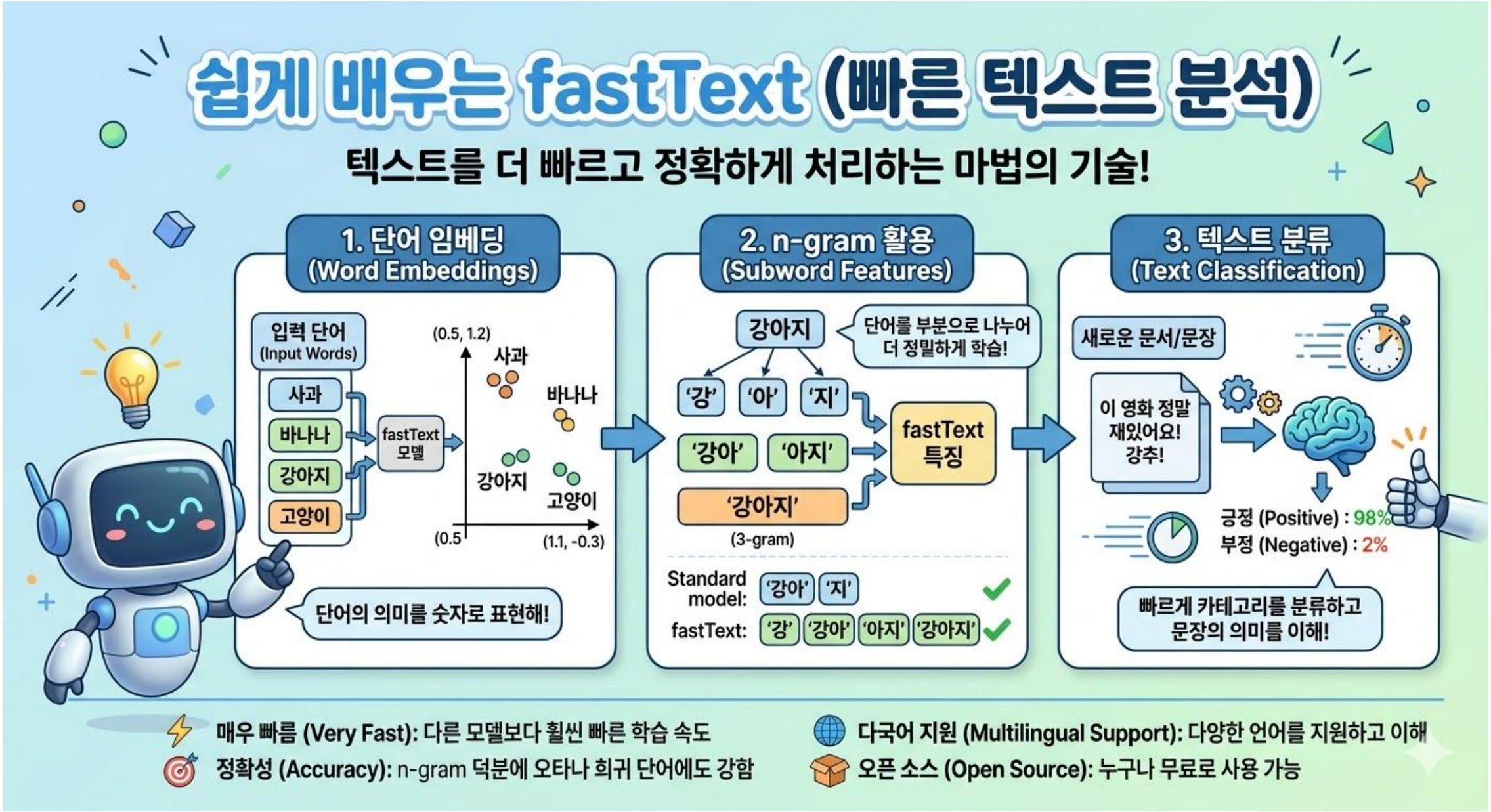
전체 코퍼스를 무한정 스캔하는 대신, 00이 아닌 행렬 값만을 단 한 번 최적화하여 연산 리소스를 극적으로 절약합니다.



정교한 선형적 의미의 포착

확률의 '비율(Ratio)'을 활용하여, 단순한 텍스트 덩어리에서 정교하고 미묘한 인간 언어의 의미적 부분 구조를 완벽하게 추출해냅니다.

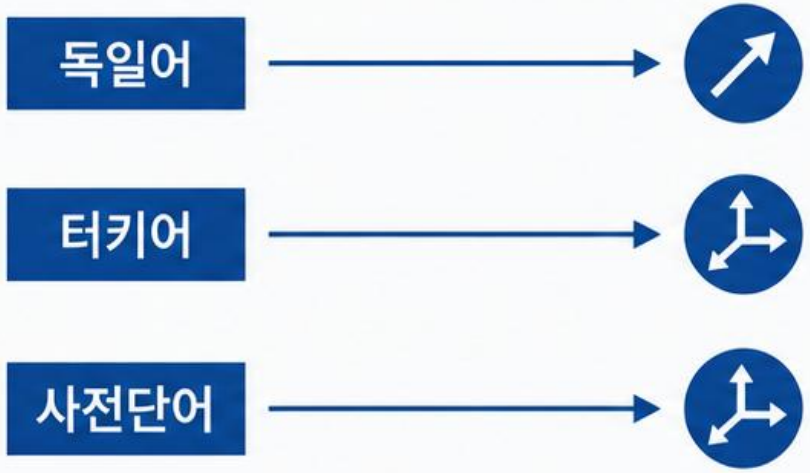
FasText


쉽게 배우는 fastText (빠른 텍스트 분석)
텍스트를 더 빠르고 정확하게 처리하는 마법의 기술!
1. 단어 임베딩 (Word Embeddings)
입력 단어 (Input Words)
사과
바나나
강아지
고양이
fastText 모델
(0.5, 1.2)
사과
바나나
강아지
고양이
(0.5
(1.1, -0.3)
단어의 의미를 숫자로 표현해!
2. n-gram 활용 (Subword Features)
강아지
단어를 부분으로 나누어 더 정밀하게 학습!
'강'
'아'
'지'
'강아'
'아지'
'강아지'
(3-gram)
fastText 특징
Standard model:
'강아'
'지'
fastText:
'강'
'강아'
'아지'
'강아지'
3. 텍스트 분류 (Text Classification)
새로운 문서/문장
이 영화 정말 재있어요! 강추!
긍정 (Positive) : 98%
부정 (Negative) : 2%
빠르게 카테고리를 분류하고 문장의 의미를 이해!
매우 빠름 (Very Fast): 다른 모델보다 훨씬 빠른 학습 속도
정확성 (Accuracy): n-gram 덕분에 오타나 희귀 단어에도 강함
다국어 지원 (Multilingual Support): 다양한 언어를 지원하고 이해
오픈 소스 (Open Source): 누구나 무료로 사용 가능

기존 임베딩 모델의 구조적 한계: OOV 문제

Word2Vec과 GloVe는 강력하지만 근본적인 한계를 지니고 있습니다.
각 단어를 분할할 수 없는 하나의 단위로 취급하여 고유한 벡터를 할당합니다.

단어 대 벡터 1:1 매핑 (정상)





형태소의 무시

형태론적으로 풍부한 언어의 내부 구조를 반영하지 못함.

OOV 실패

훈련 데이터에 없는 새로운 단어 등장 시 에러 발생.

비효율적 확장성

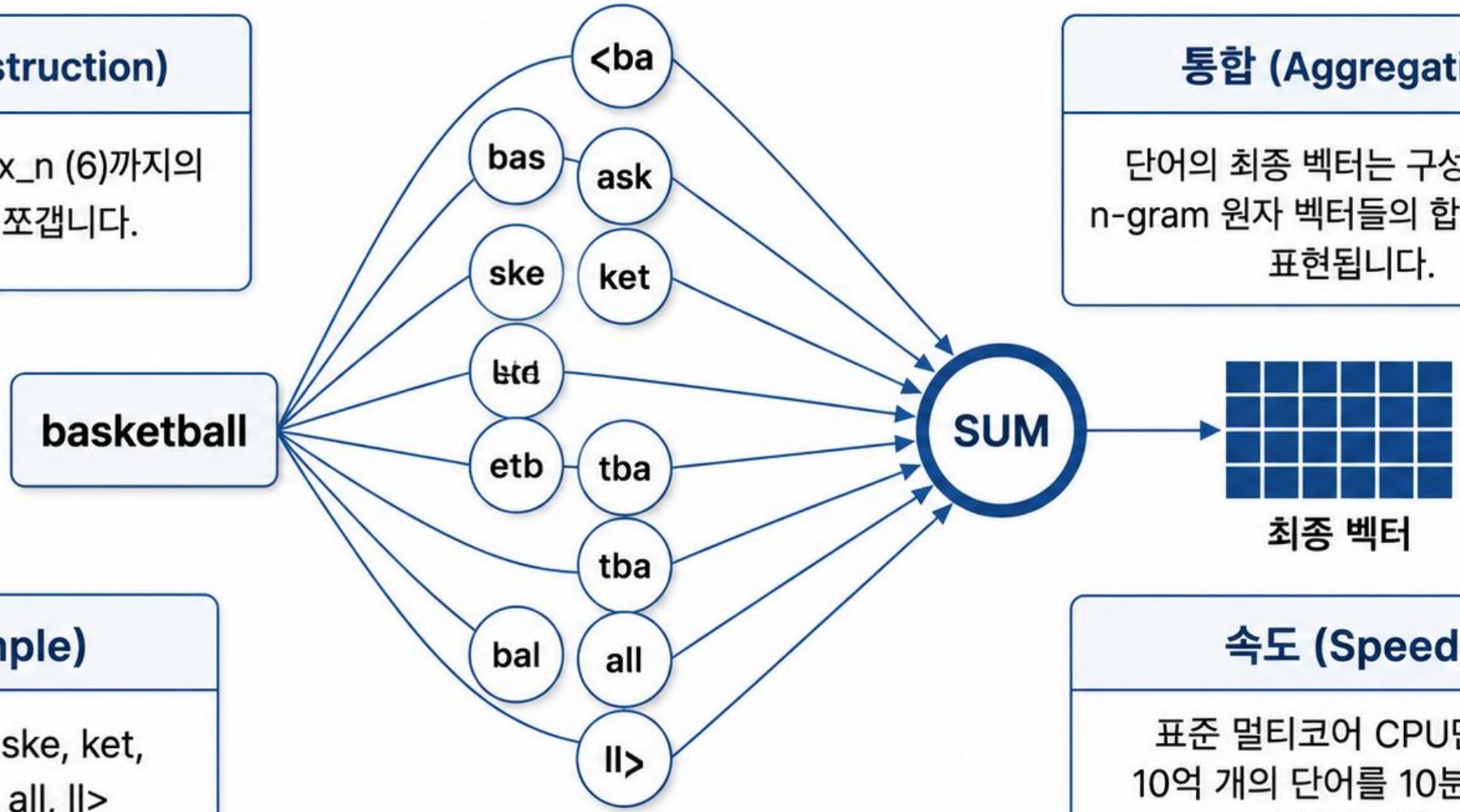
희귀 단어(Rare words)에 대한 정확한 임베딩 학습 불가.

fastText의 혁신: 하위 단어 N-그램 (Subword N-Grams)

fastText는 단어를 가장 작은 원자 단위인 문자 n-gram의 합으로 취급합니다.

분해 (Deconstruction)

min_n (3)부터 max_n (6)까지의 문자로 단어를 쪼갭니다.



통합 (Aggregation)

단어의 최종 벡터는 구성된 모든 n-gram 원자 벡터들의 합(Sum)으로 표현됩니다.

예시 (Example)

<ba, bas, ask, ske, ket, etb, tba, bal, all, ll>

속도 (Speed)

표준 멀티코어 CPU만으로 10억 개의 단어를 10분 이내에 훈련합니다.

OOV(미등록어) 처리 메커니즘

한 번도 본 적 없는 단어라도 훈련 데이터에 최소 하나의 n-gram이 존재한다면
즉시 유효한 벡터를 합성해냅니다.

1. 미등록어 입력

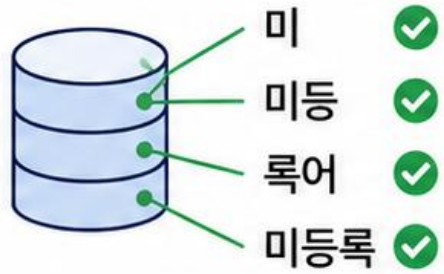
새로운 파생어/오탈자

2. 원자 단위 스캔



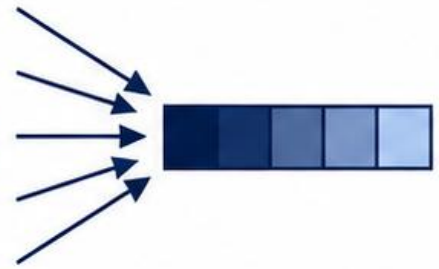
단어를 n-gram으로 분해

3. 패턴 매칭



사전에 학습된
하위 단어 벡터 추출

4. 벡터 합성

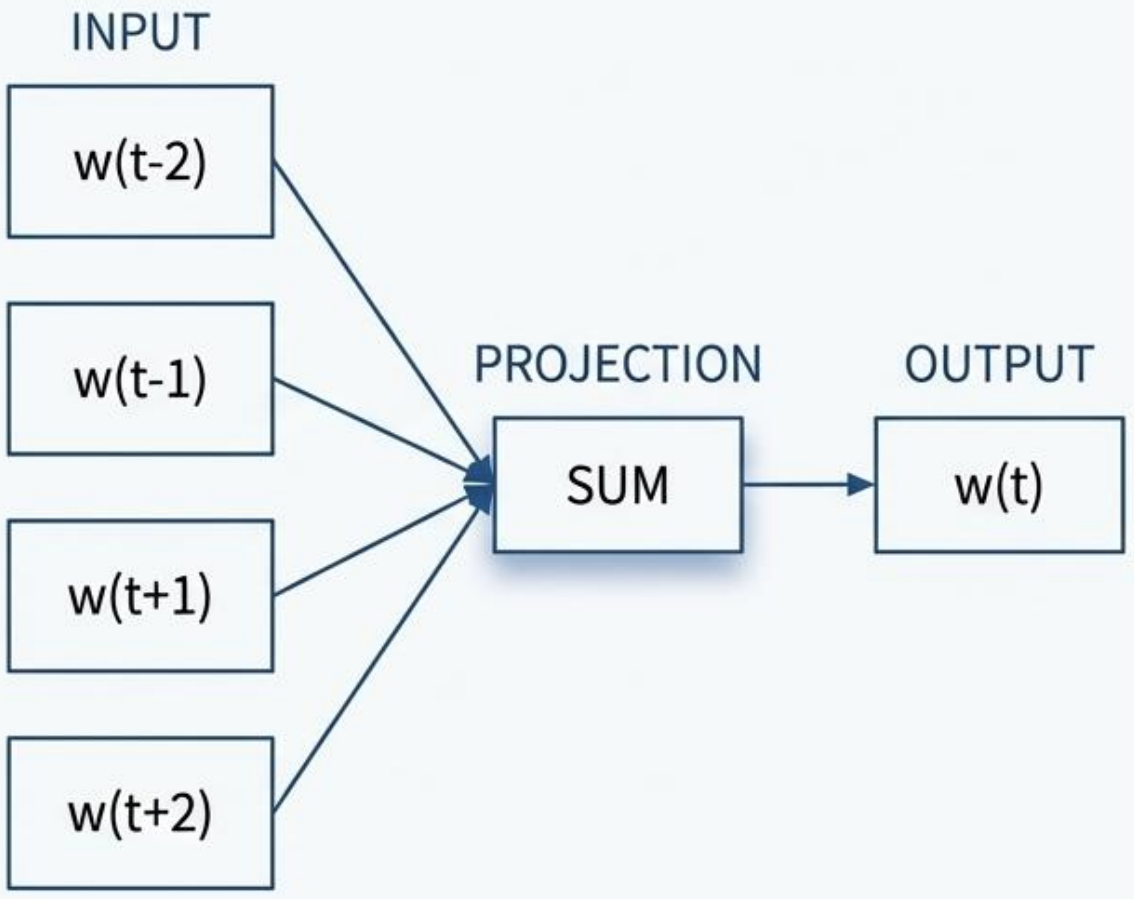


추출된 벡터 합산으로
새로운 벡터 생성

결과: 구문론적 유사도(Syntactic similarity)에서 Word2Vec을 압도하는 성능 발휘

아키텍처 I: CBOW (Continuous Bag of Words)

주변 문맥(Context)을 통해 중심 단어(Target word)를 예측하는 구조입니다.



양방향 윈도우

타겟 단어 양방향의 지정된 window 내의 문맥을 사용하며 단어 순서는 무시됩니다.

투영층 합산 (Projection Sum)

입력층과 투영층 사이의 가중치 행렬을 공유하며 벡터를 합산합니다.

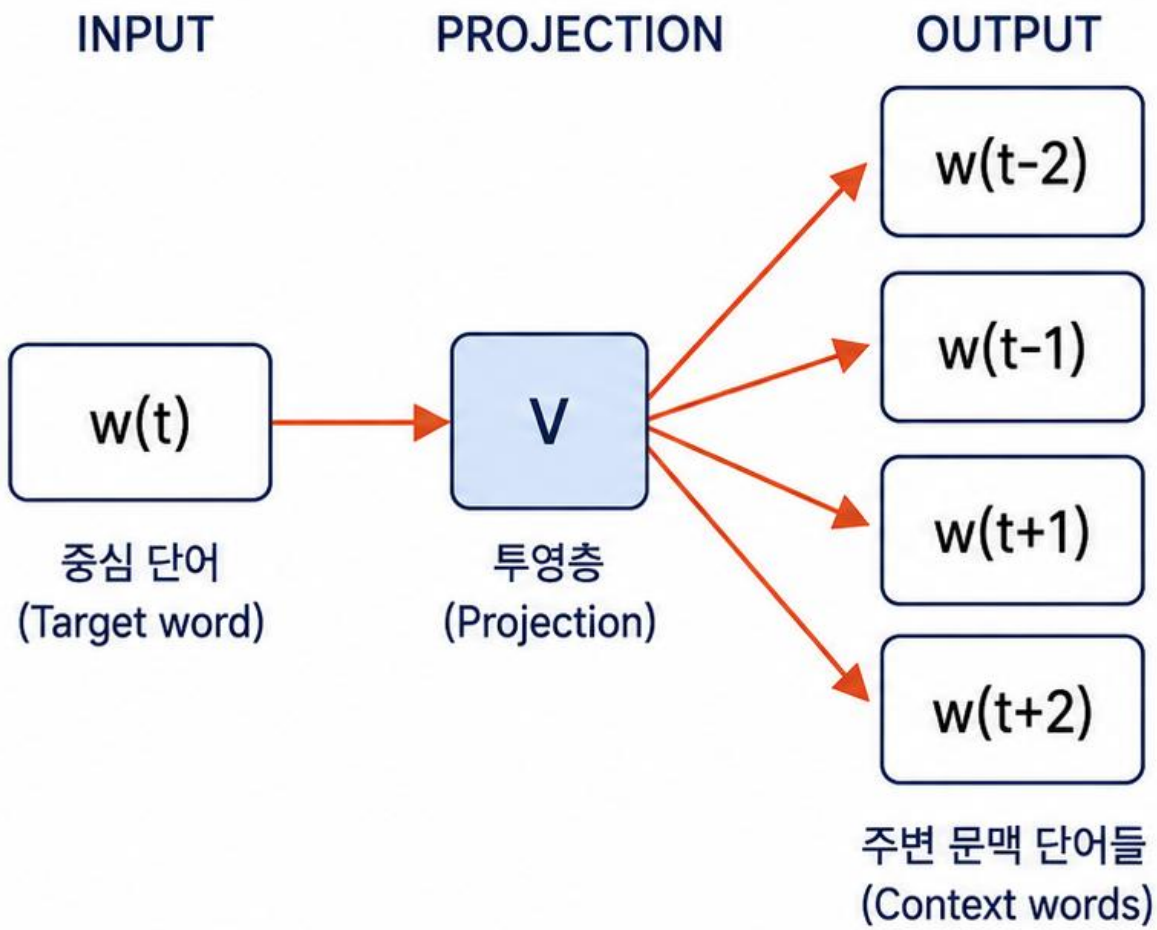
성능 특징

학습 속도가 매우 빠르며, 자주 등장하는 단어(Frequent words)에 대해 약간 더 높은 정확도를 보여줍니다.

NotebookLM

아키텍처 II: Skip-gram

중심 단어(Target word)를 기반으로 주변 문맥(Context words) 를 예측하는 구조입니다.



독립적 보존

각 n-gram 내의 단어 순서는 보존되지만,
원도우 내 n-gram들의 순서는 보존되지 않습니다.

학습 원리

주어진 시퀀스에서 로그 확률(Log probability)의 평균을 최대화하도록 학습합니다.

비교 (vs CBOW)

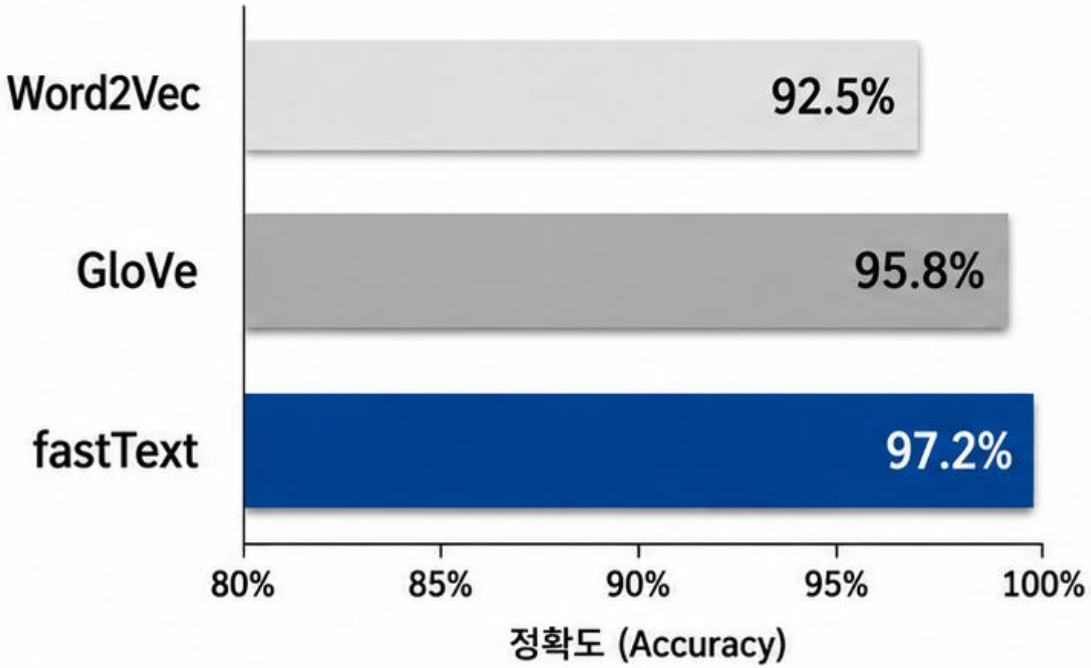
훈련 속도는 상대적으로 느리지만, 드물게 등장하는 희귀 단어(Infrequent words)의 의미를 포착하는 데 더 뛰어난 성능을 발휘합니다.

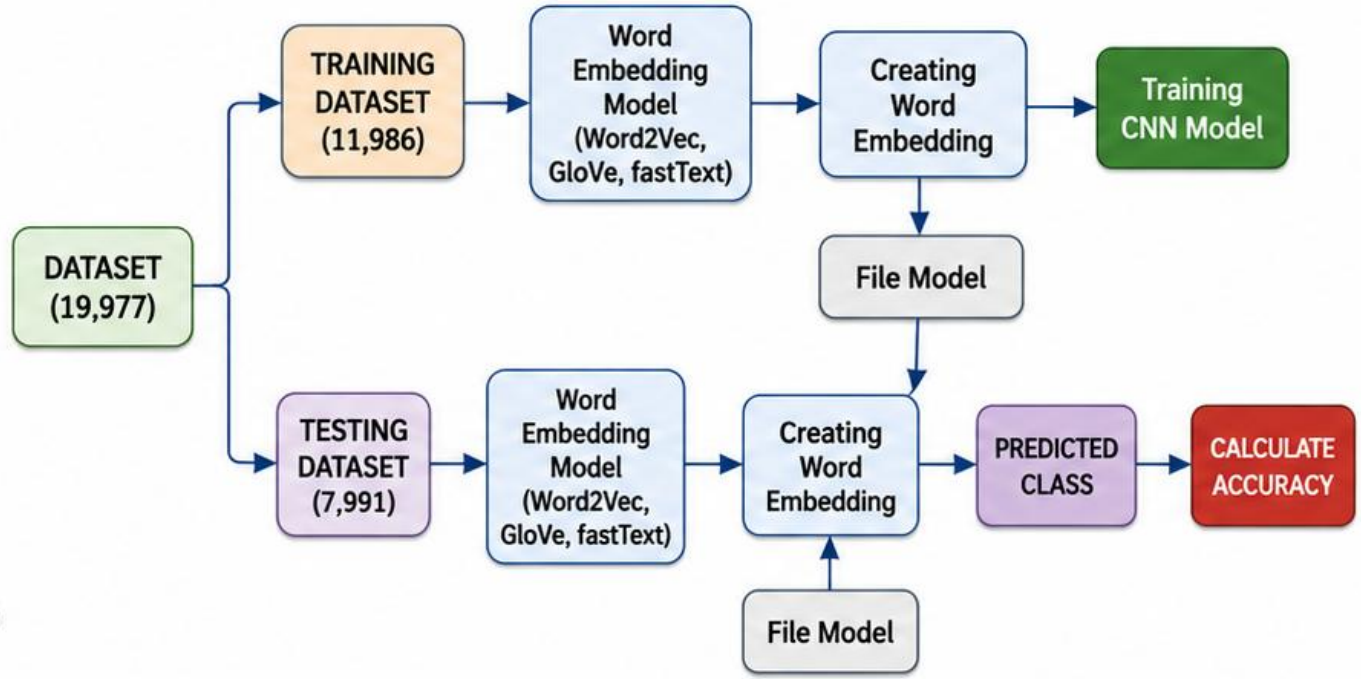
임베딩 모델 비교 매트릭스: Word2Vec vs GloVe vs fastText

특징 (Feature)	Word2Vec	GloVe	fastText
기본 단위 (Base Unit)	단어 (Word)	단어 (Word)	하위 단어 (n-gram)
컨텍스트 (Context)	로컬 윈도우 (Local)	로컬 + 글로벌 동시발생 (Co-occurrence)	로컬 윈도우 + 내부 구조
강점 (Strength)	의미론적(Semantic) 관계	단어 유추 (Word Analogy)	구문론적(Syntactic) 관계 & 복잡한 언어
OOV 처리 역량	불가 (오류 발생)	불가 (오류 발생)	가능 (n-gram 합성) ✅
연산 속도	빠름	보통	매우 빠름 (계층적 소프트맥스 활용시)

벤치마크 검증: CNN 텍스트 분류 연구 (JATIT)

20 뉴스그룹 데이터셋(19,977개 기사)을 활용한 1D-CNN 텍스트 분류 정확도 테스트 결과,
fastText가 가장 우수한 성능을 입증했습니다.





fastText가 1위를 차지한 핵심 이유는 훈련 데이터에 나타나지 않은 미등록어(OOV)를 처리하는 능력에 있습니다. 단어 단위 임베딩의 한계를 넘어서기 때문에 노이즈가 많은 많은 실제 텍스트 데이터에서 압도적으로 유리합니다.

실무 파이프라인: 지도 학습 분류기 구축

명령어 몇 줄만으로 고성능 텍스트 분류 모델(model_cooking.bin)을 구축할 수 있습니다.

1 데이터 준비



레이블(__label__)과 텍스트가 결합된 텍스트 파일 준비.

2 전처리 (Preprocessing)



특수문자 분리, 소문자화(sed, tr)를 통한 어휘 크기 최적화.

3 훈련 (train_supervised)



input, lr, epoch, wordNgrams 파라미터를 설정하여 모델 학습.

4 평가 & 예측 (test / predict)



검증 데이터셋을 활용해 정밀도(Precision@1) 및 재현을 측정.

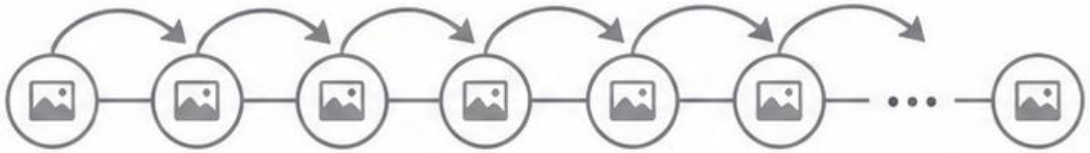


fastText가 1위를 차지한 핵심 이유는 훈련 데이터에 나타나지 않은 미등록어(OOV)를 처리하는 능력에 있습니다. 단어 단위 임베딩의 한계를 넘어서기 때문에 노이즈가 많은 많은 실제 텍스트 데이터에서 압도적으로 유리합니다.

극한의 속도 확장: 계층적 소프트맥스 (Hierarchical Softmax)

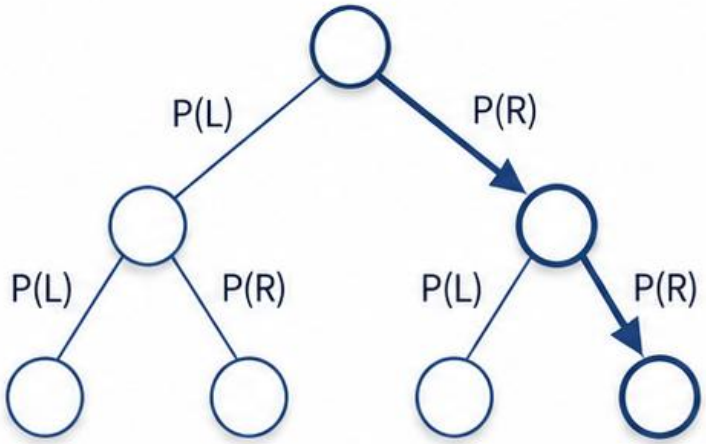
거대한 데어터셋과 다중 클래스 환경에서 학습 속도가 느려지는 문제를 해결하는 핵심 기술입니다 (-loss hs).

기존 방식 (Standard Softmax) - O(N) 병목 발생



모든 클래스의 확률을
선형적으로 순차 계산합니다.

허프만 트리 도입 (Huffman Tree) - O(log N) 획기적 단축

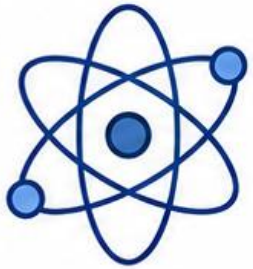


자주 동장하는 출력값을 트리 상단(Root)에 배치.
중간 노드 확률만 곱하여 연산 시간을
기하급수적으로 단축합니다.

효율성:

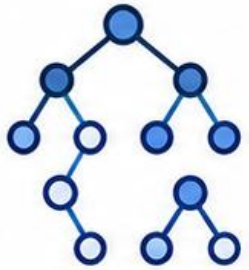
- 예축 시 루트에서 리프까지 경로의 중간 노드 확률만 곱하여 연산 시간을 획기적으로 단축합니다.
- 다중 레이블 분류의 경우 독립적 이진 분류기(-loss ova)를 사용합니다.

fastText: 핵심 요약 치트 시트 (Executive Summary)



[Q1] 핵심 혁신 (Core Innovation)

- 단어를 n-gram의 합으로 처리.
- OOV(미등록어) 처리 가능.
- 형태론적으로 복잡한 언어 처리에 최적화.



[Q2] 아키텍처 (Architecture)

- 확장된 CBOW & Skip-gram 모델 활용.
- 내부 순서 정보 보존 및 계층적 소프트맥스(Huffman Tree)를 통한 연산 속도 O(log N) 최적화.



[Q3] 벤치마크 (Empirical Edge)

- 구문론적(Syntactic) 작업에서 Word2Vec 압도.
- JATIT 분류 태스트 결과 97.2% 정확도로 1위 달성.

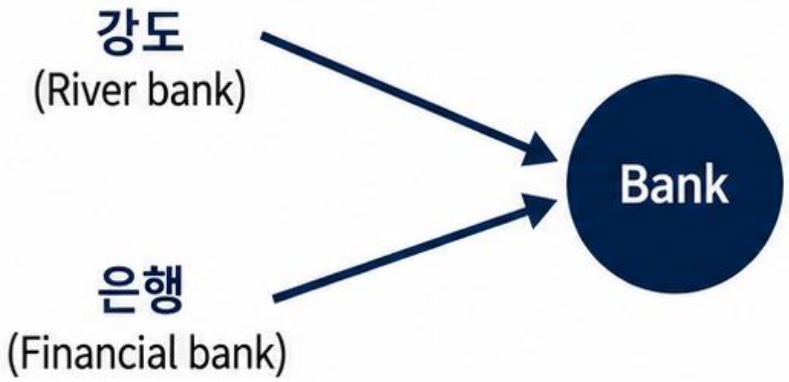


[Q4] 구현 및 최적화 (Implementation)

- 성능 튜닝: epoch, lr, wordNgrams 집증 조정.
- 메모리 제약 발생 시 quantize, compress-fasttext 또는 .vec 변환을 통한 풋프린트 축소.

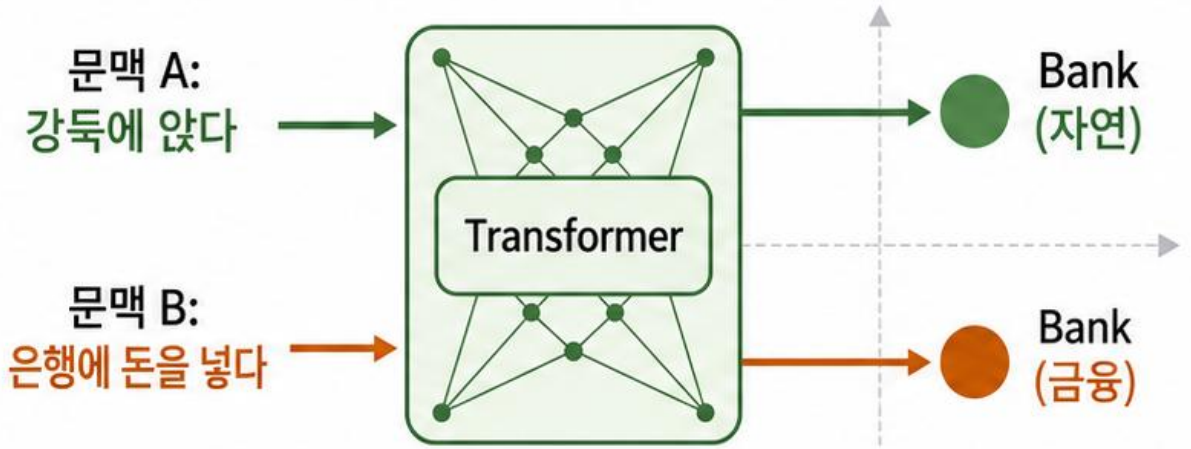
다의어의 난제와 문맥적 임베딩의 진화

Word2vec (정적 임베딩)



초기 한계: 의미가 여러 개인 다의어도 단 하나의 고정된 벡터로 융합(Conflation)됩니다.

BERT / ELMo (문맥적 임베딩)



현대 AI 해법: 문장 전체를 양방향으로 읽어 문맥에 따라 동일한 단어도 매번 다른 토큰 수준(Token-level)의 동적 벡터로 분리합니다.

핵심 요약

정적 임베딩은 단어의 다양한 의미를 구분하지 못하지만,
문맥적 임베딩은 주변 문맥을 반영하여 다의어의 의미를 효과적으로 분리합니다.

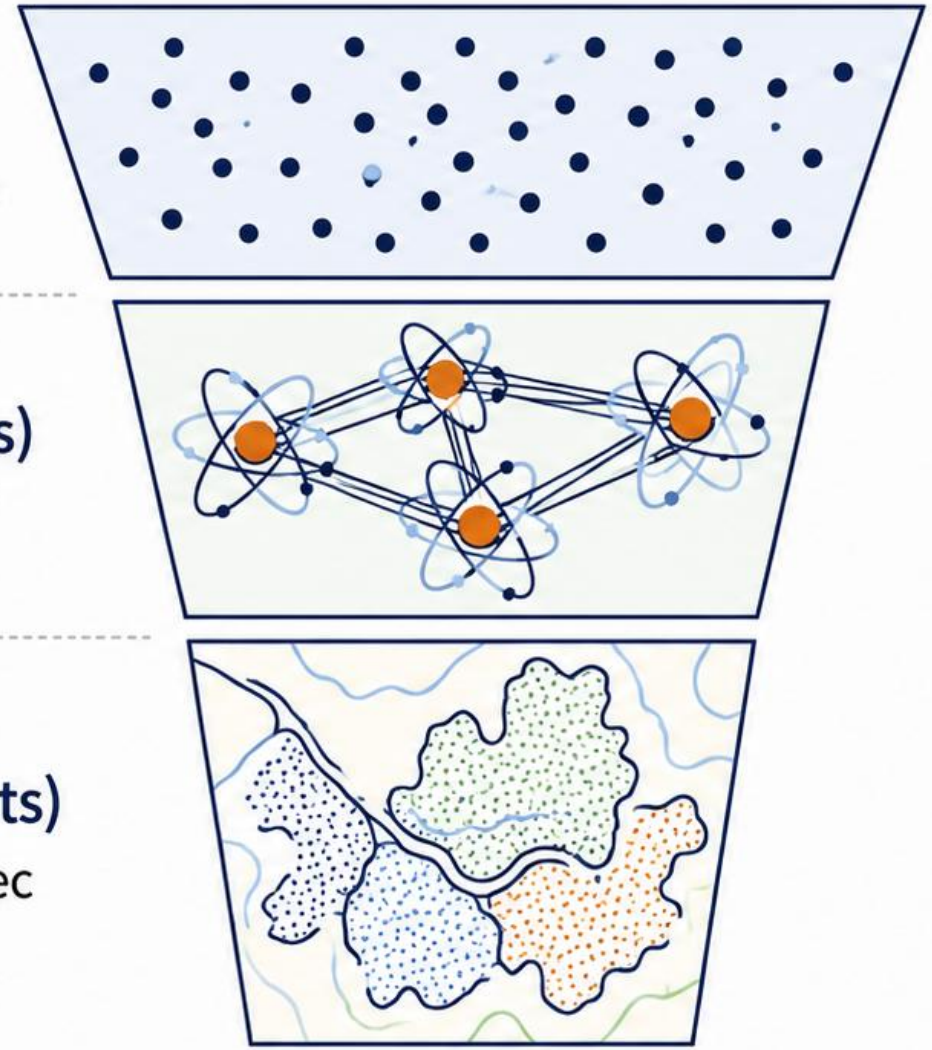
확장: 단어를 넘어 문장과 문서의 공간으로

- 임베딩의 원리는 단어에만 국한되지 않습니다.
- 가변 길이의 텍스트(문장, 단락, 전체 문서)를 단일한 고정 차원 벡터로 압축해 냅니다.
- 이를 통해 문서 분류, 토픽 모델링, 의미론적 검색 엔진의 기반이 마련됩니다.

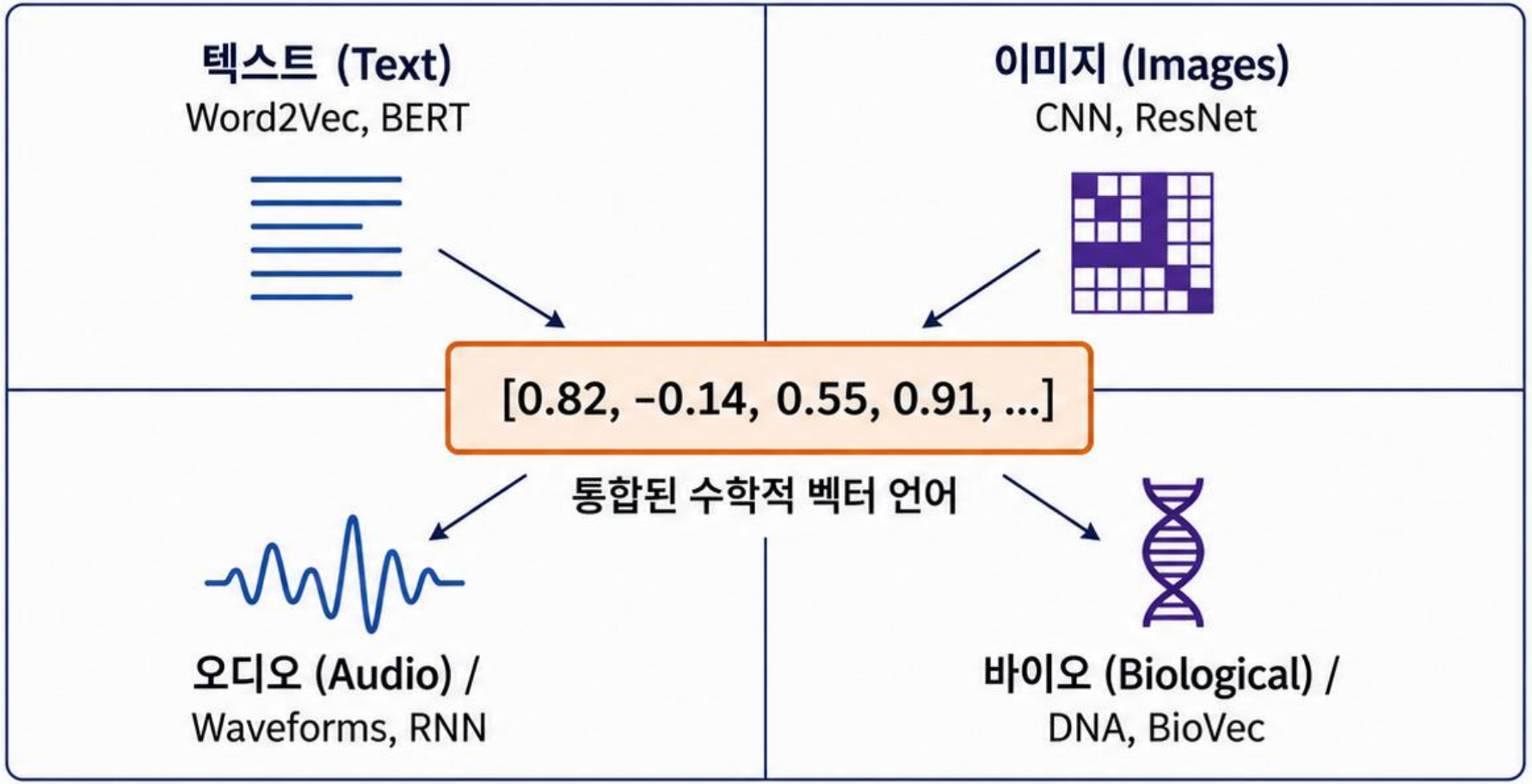
단어 (Words)
Word2vec, GloVe

문장 (Sentences)
Sentence-BERT

문서 (Documents)
Doc2vec, Top2vec

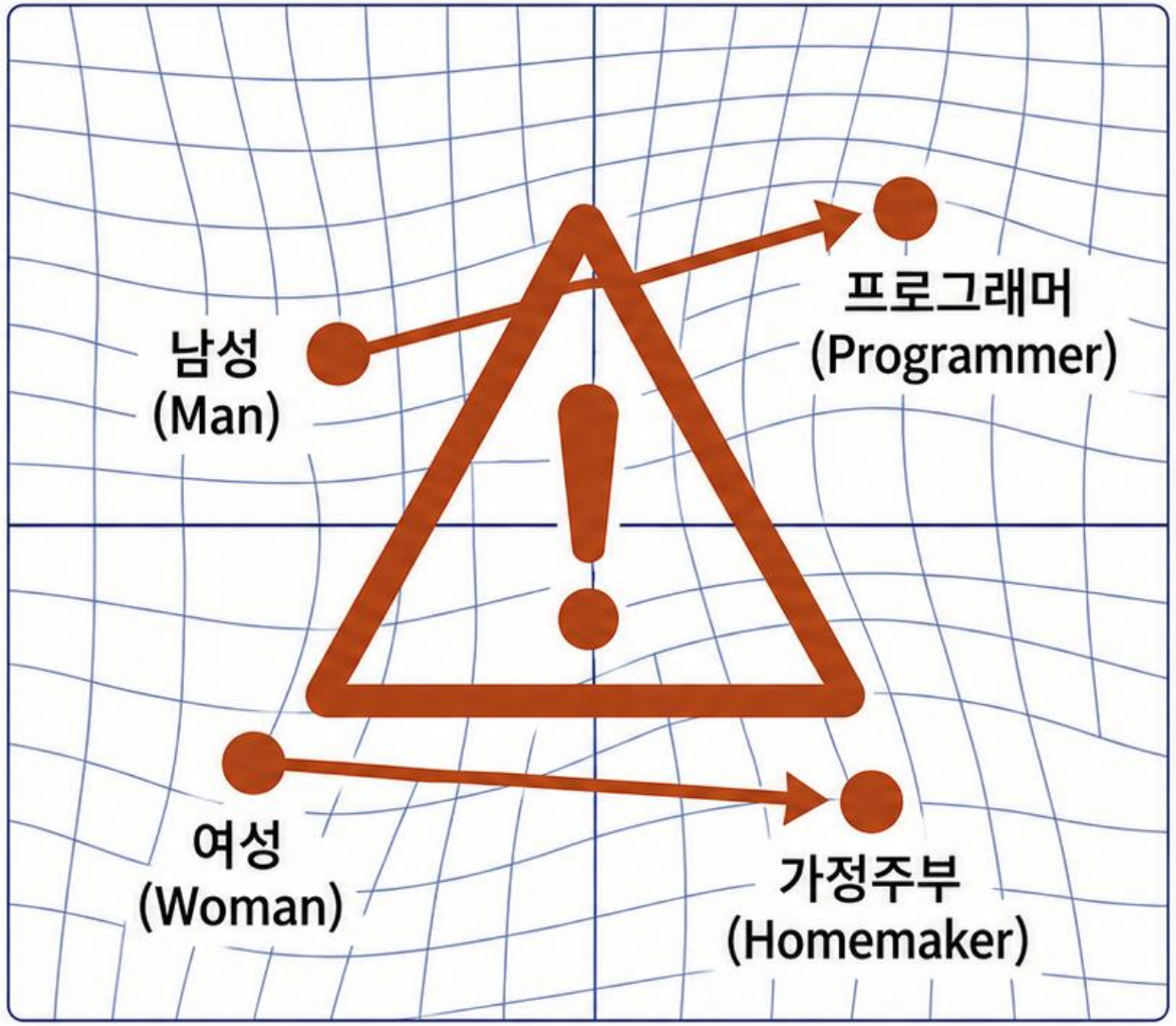


멀티모달 임베딩 (Multimodal Embeddings): 모든 데이터의 벡터화



이미지, 오디오, 심지어 그래프 구조와 유전자 염기서열까지, 컴퓨터가 인식할 수 있는 모든 형태의 데이터는 **임베딩**을 통해 **동일한 수학적 벡터 언어**로 통일될 수 있습니다.

숨겨진 위험: 내재된 편향성 (Embedded Bias)



> "Man is to Computer Programmer as Woman is to *Homemaker?*"
>
> – (Bolukbasi et al., 2016)

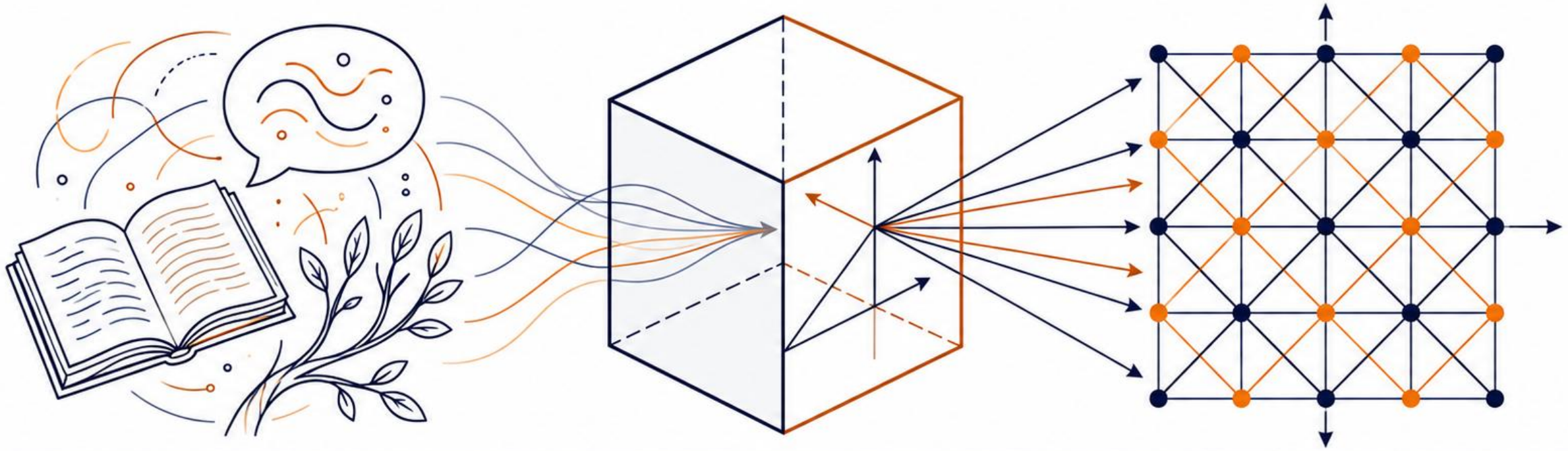
- 임베딩 모델은 인간이 작성한 방대한 뉴스 기사와 웹 문서를 여과 없이 학습합니다.
- 결과적으로 인종, 성별, 직업에 대한 역사적 선입견과 사회적 편향성이 수학적 구조 안에 그대로 복제되며, 이를 증폭시키는 위험을 내포하고 있습니다.
- 세심한 감독과 편향 제거(Debiasing) 기술이 필수적입니다.

종합: AI 시대의 만능 번역기 (The Universal Translator)

인간의 의미 세계
혼돈과 뉘앙스

임베딩 (Embedding)
기하학적 번역

기계의 연산 세계
측정 가능한 거리



워드 임베딩과 **Word2vec**은 단순한 알고리즘이 아닙니다.

그것은 인간의 추상적인 의미 세계와 기계의 논리적 연산 세계를 **연결하는 가장 완벽한 기하학적 다리**입니다.

모든 지식과 형태를 측정 가능한 거리로 변환함으로써, 임베딩은 **생성형 AI**와 **모든 인공지능 혁신**의 근본적인 토대가 되었습니다.

이미지 임베딩



인공지능의 시각 해부학

CNN의 특징 추출 원리:
엣지(Edge)에서 사물(Object)까지

단어의 흐름을 읽는 LLM을 넘어,
픽셀의 패턴을 읽는 컴퓨터 비전의 세계로.



인공지능이 세상을 해독하는 두 가지 방법

텍스트는 단어의 의미 패턴을, 이미지는 시각적 패턴을 벡터(임베딩)로 변환하여 이해합니다.

LLM - Text Processing

1. 텍스트 → 임베딩 벡터

나는 (토큰 1) → 밥을 (토큰 2) → [?] (토큰 n) → 먹는다 (토큰 n+1) → 임베딩 벡터 $\in \mathbb{R}^d$

2. 문장의 패턴 학습 (Gemini, GPT)

수많은 텍스트 데이터를 학습해 단어 간의 순서와
확률적 패턴을 이해합니다.

- 대규모 텍스트 데이터
- 맥락 이해
- 다음 단어 예측
- 문장 생성/요약

CNN - Image Processing

1. 이미지 → 임베딩 벡터



Convolution → ReLU → Pooling → 임베딩 벡터 $\in \mathbb{R}^d$

2. 시각적 패턴 조립 (CNN)

작은 패턴(선, 모서리)을 찾고, 이를 조합하여
큰 패턴(눈, 고양이 전체)을 학습합니다.

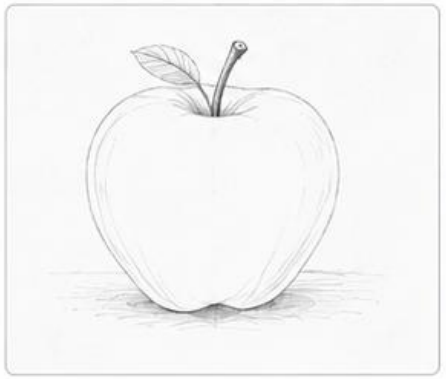
- 대규모 이미지 데이터
- 특징 추출 (선, 모서리, 질감)
- 특징 조합 (부분 → 전체)
- 분류/인식/검출

핵심 아이디어 돌 다 **고차원 임베딩 벡터**로 변환하여 의미 있는 **패턴**을 학습하고, 새로운 데이터를 **이해하고 생성(또는 인식)**합니다.

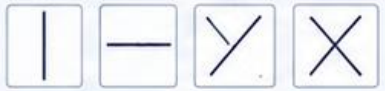
화가의 캔버스와 CNN의 공통점

CNN의 학습 과정은 화가가 그림을 그리는 과정과 정확히 일치합니다.
단순한 스케치에서 시작해 점차 **복잡한 사물**로 나아갑니다.

1. 화가: 윤곽 스케치



CNN: 엣지(Edge) 감지



기본적인 선, 방향, 대비 등
단순한 특징을 감지

2. 화가: 디테일 묘사



CNN: 윤곽 및 질감 추출



형태의 윤곽과 질감 등
부분적 형태를 추출

3. 화가: 채색 및 완성



CNN: 중간 수준 특징 학습



조합된 특징(모양, 패턴, 색의 조합)을
학습하여 더 복잡한 표현을 만듦

완성된 그림



CNN: 사물 인식



고수준 특징을 종합하여
사물을 인식하고 분류

핵심 메시지 | 화가가 점진적으로 디테일을 추가하듯, CNN도 **계층적 구조**를 통해 **단순한 특징에서 복잡한 개념**으로 학습합니다.

1단계: 저차원 특징 (Low-level Feature)

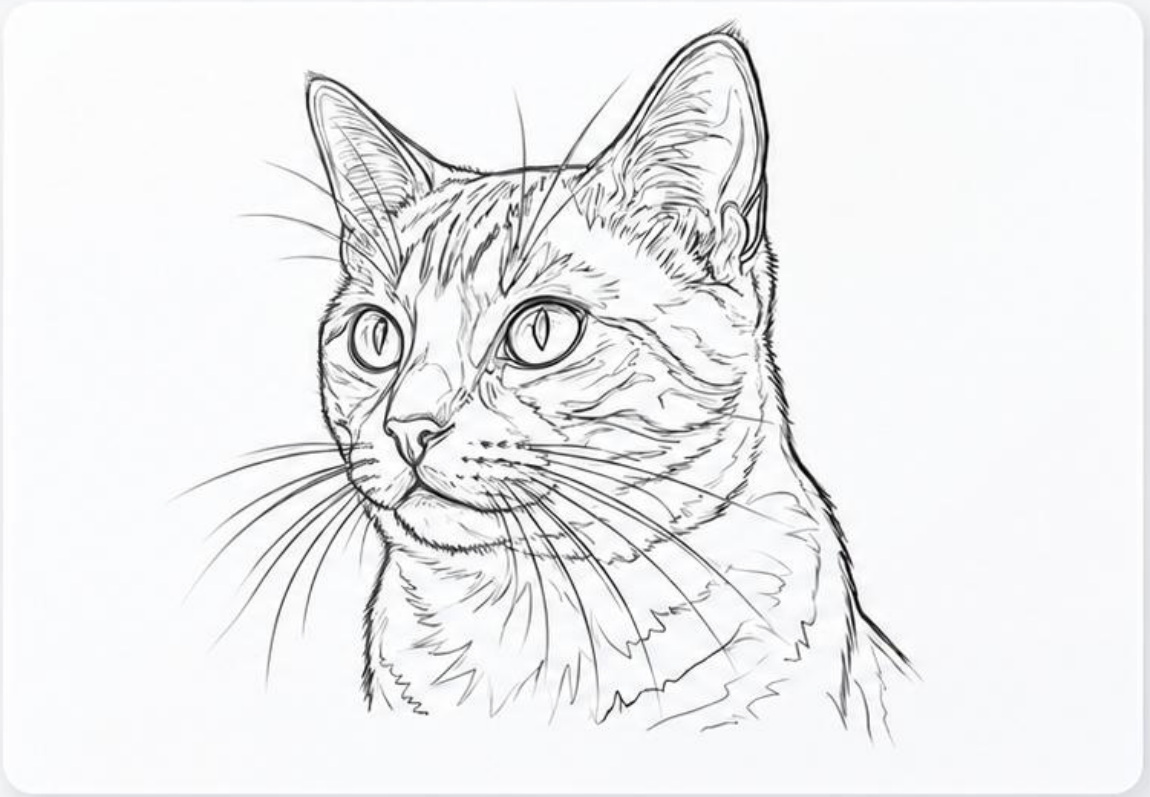
스케치의 시작: **선과 모서리**를 찾다

첫 번째 합성곱(Convolution) 층은
이미지의 가장 기초적인 요소를 감지합니다.

추출 대상: 엣지(Edge), 직선, 단순한 색상 대비

시각적 느낌: 사진을 흑백으로 변환하고
윤곽선만 남긴 형태

역할: 복잡한 형태를 이해하기 위한
기초 특징을 제공합니다.



초기 합성곱 필터가 감지하는 저차원 특징들



2단계: 중간 특징 (Mid-level Feature)

특징: 디테일 요소를 조합하여 의미 있는 패턴을 형성한다

두세 번째 층으로 넘어가면, 단순한 선과 모서리들이 결합하여 의미 있는 조각(특징)을 형성합니다.

형태적 특징

곡선, 윤곽선, 모양, 질감 등
객체의 형태를 나타내는 특징을 학습합니다.

구조적 톡징

여러 형태적 특징이 결합되어
객체의 구조(눈, 귀, 코, 털 패턴 등)를 표현합니다.

의미적 특징

구조적 특징들이 모여 전체적인 의미를 가지는
객체의 부위나 패턴을 인식합니다.

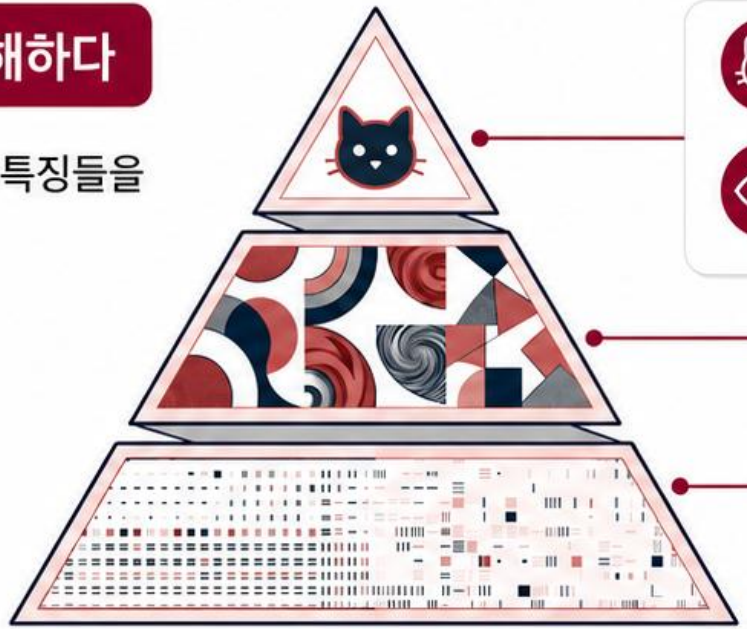


핵심 포인트: 중간 특징은 단순한 요소들의 조합을 통해, 더 복잡하고 추상적인 표현을 가능하게 합니다.

3단계: 고차원 특징 (High-level Feature)

형태의 완성: 사물의 개념을 이해하다

가장 깊은 층에 도달하면, 네트워크는 부분적인 특징들을 조합해 하나의 완성된 개념을 추상화합니다.



추출 대상: 얼굴 전체, 고양이 전체 클래스

시각적 느낌: 단순한 픽셀의 집합이 아닌, 인간이 인식하는 '의미(Semantic)'를 갖춘 상태

단 하나의 추상적 개념 (예: 고양이)
여러 특징이 결합되어 하나의 추상적 개념을 형성

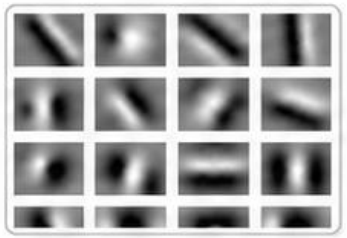
수천 개의 패턴 조각
다양한 질감과 형태의 조합을 통해 더 복잡한 패턴을 표현

수만 개의 픽셀 & 선
픽셀 수준의 정보(엣지, 선, 색상 등) 기분 요소를 추출

입력 이미지

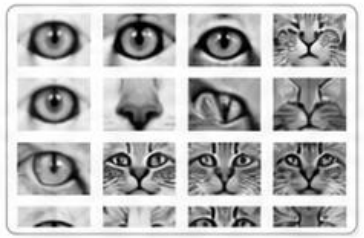


1단계: 저차원 특징 (Low-level Feature)



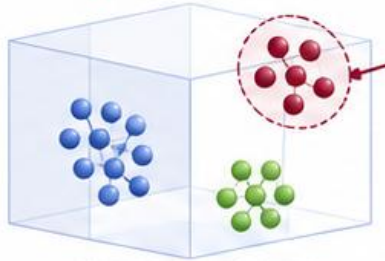
엣지, 선, 색상, 대비 등
기분적인 시각적 특징 감지

2단계: 중간 특징 (Mid-level Feature)



눈, 귀, 코, 수염, 무늬 등
부분적인 조합 특징 학습

3단계: 고차원 특징 (High-level Feature)



여러 특징이 결합되어
'고양이'라는 하나의 개념으로 추상화

고양이 클래스
- 얼굴 전체 형태
- 고유한 패턴
- 자세, 비율
- 전체 개념 표현

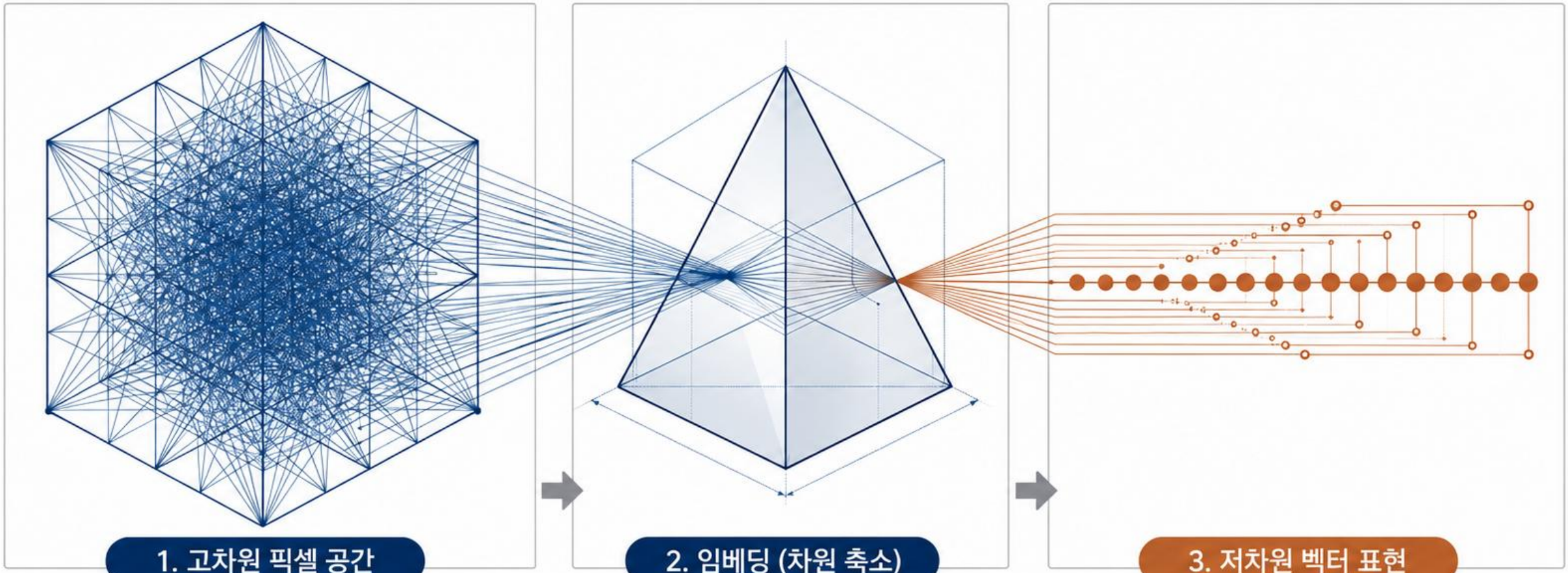
출력: 분류 결과

고양이

핵심 포인트 | 고차원 특징은 네트워크가 축적한 모든 정보를 기반으로, 사물을 사람처럼 '이해'하고 '구분'할 수 있게 해줍니다.

이미지 임베딩: 픽셀에서 벡터로

복잡한 시각 데이터를 저차원 공간의 본질로 압축하는 딥러닝 아키텍처의 이해



1. 고차원 픽셀 공간

이미지는 수많은 픽셀 값으로 구성된
고차원 시각 데이터

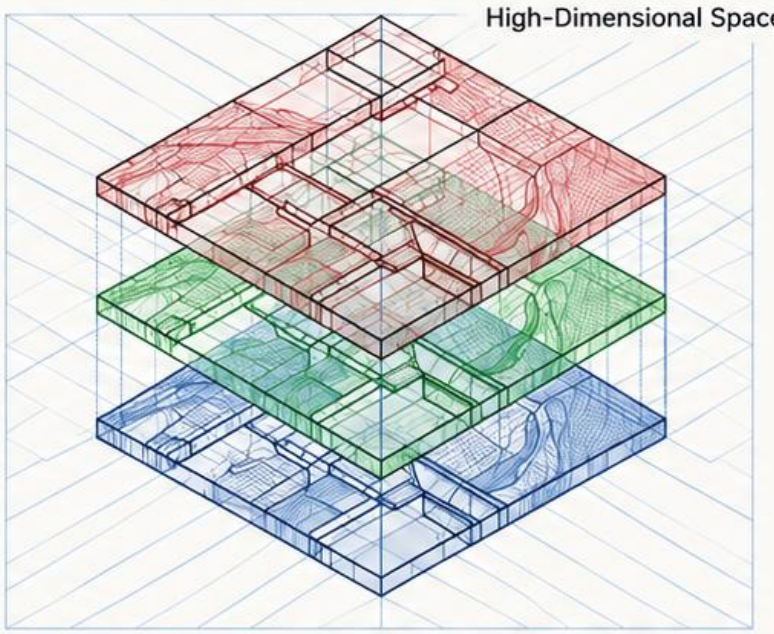
2. 임베딩 (차원 축소)

딥러닝 모델이 핵심 특성을 추출하여
저차원 잠재 공간으로 변환

3. 저차원 벡터 표현

핵심 정보를 담은 벡터로 표현되어
다양한 다운스트림 작업에 활용

임베딩의 본질: 고차원에서 저차원으로



압축 및 변환
(Compression & Transformation)

이미지의 중요한 특징만을 추출하여

Low-Dimensional Space

0.45	-1.23	0.98	2.15	-0.07	1.66	...	0.34	0.22	-0.21	0.98	2.15	1.66	1.36	0.90	-1.27	0.98	3.23	1.81	1.66	-0.07	...

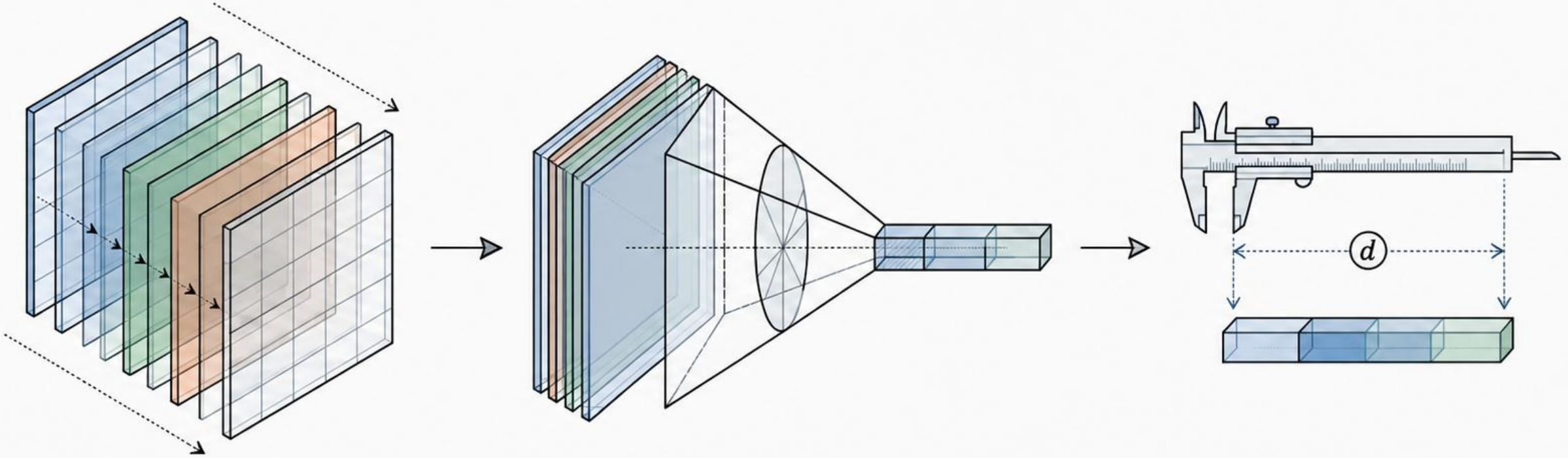
수학적 '벡터(Vector)' 형태로 표현하는 과정

이미지의 복잡한 정보를 수치화하여 벡터로 변환

의미적 유사성을 측정, 분류, 검색,
생성할 수 있는 데어터의 최소 단위.

벡터 간 거리를 통해 유사도 비교 및 다양한 작업 수행

작동 원리: 3단계 파이프라인



특징 추출

CNN 아키텍쳐를 통한 톡징 추출.
저수준(엣지, 텍스쳐)부터 고수준(객체,
장면) 특징까지 단계별 학습.

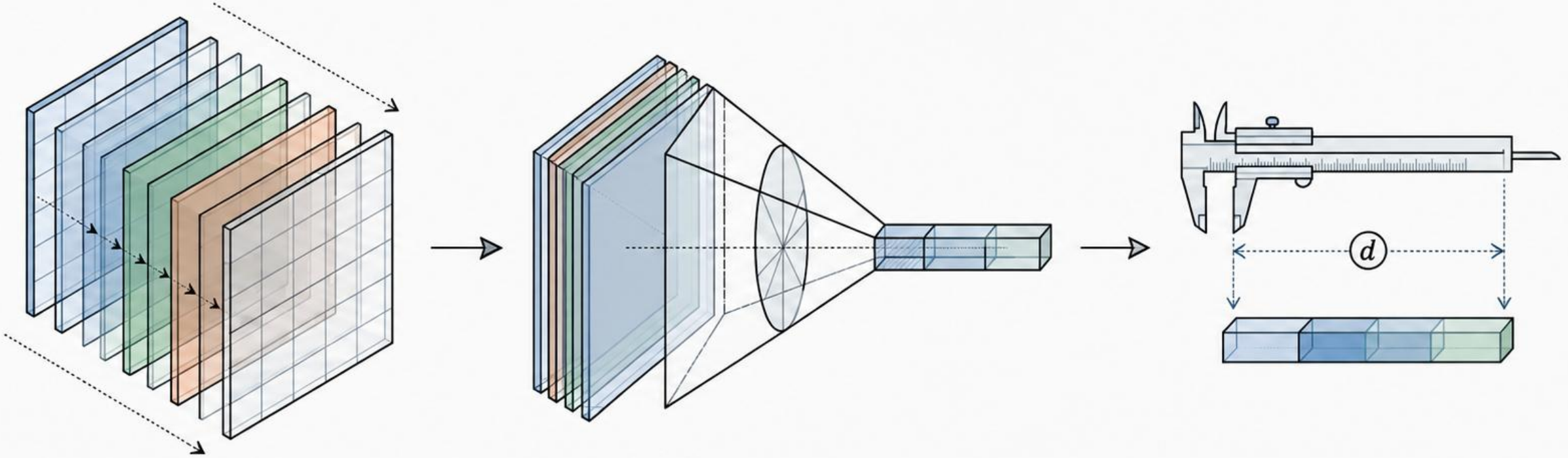
차원 축소

거대한 톡징 맵을
저차원 벡터로 압축.

유사성 측정

생성된 벡터 간의 수학적 거리 계산
(코사인 유사도 또는 유클리드 거리).

작동 원리: 3단계 파이프라인



특징 추출

CNN 아키텍쳐를 통한 톡징 추출.
저수준(엣지, 텍스쳐)부터 고수준(객체,
장면) 특징까지 단계별 학습.

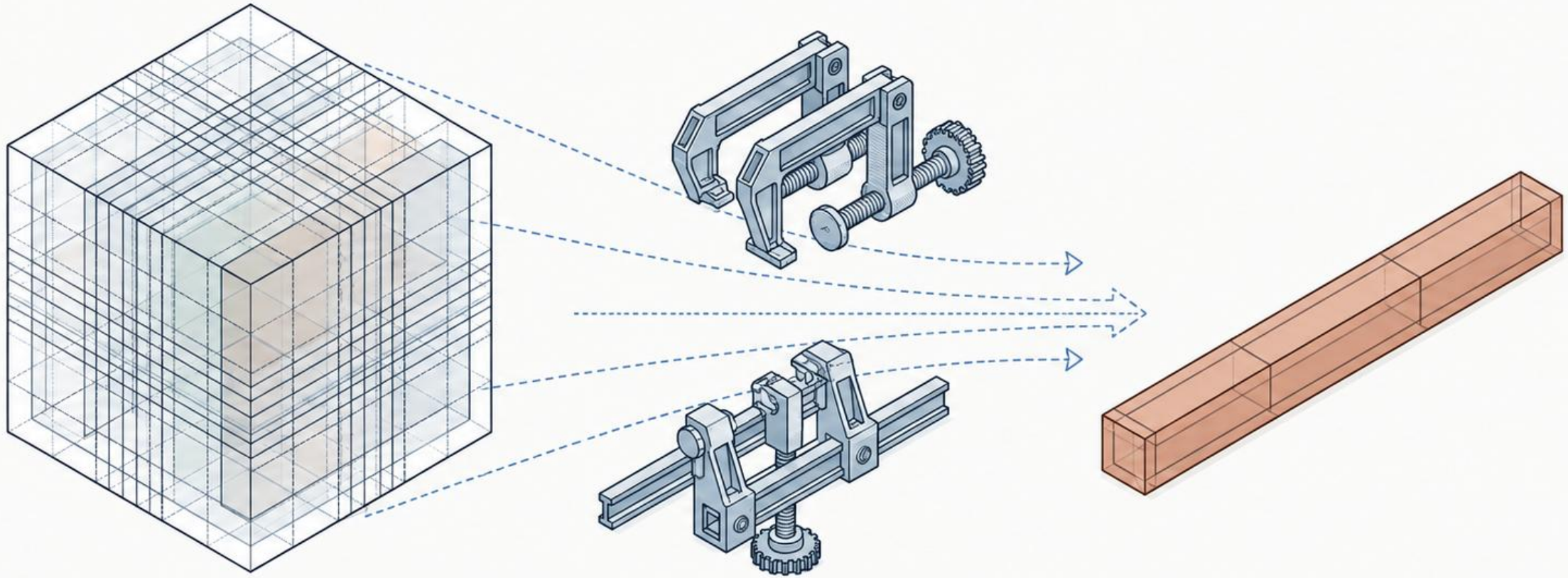
차원 축소

거대한 톡징 맵을
저차원 벡터로 압축.

유사성 측정

생성된 벡터 간의 수학적 거리 계산
(코사인 유사도 또는 유클리드 거리).

차원 축소: 노이즈의 제거와 본질의 압축



1. 특징 추출 맵 (고차원 텐서)

입력 데이터를 CNN 등으로 처리하여
고차원의 특징 맵을 생성합니다.

2. PCA & t-SNE 기법

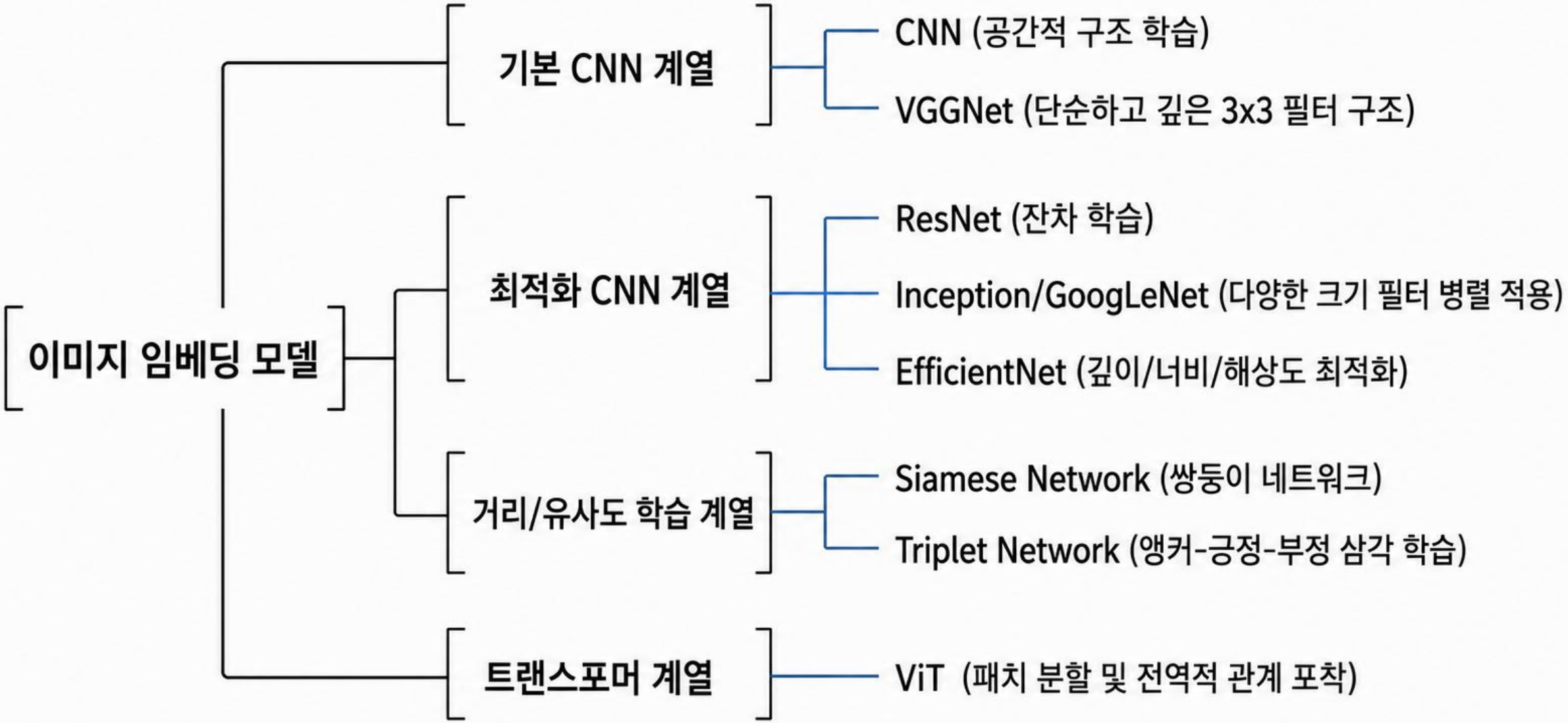
고차원 데이터를 저차원 공간으로
투영하여 노이즈를 제거하고 구조를 보존합니다.

3. 최종 임베딩 벡터 (저차원)

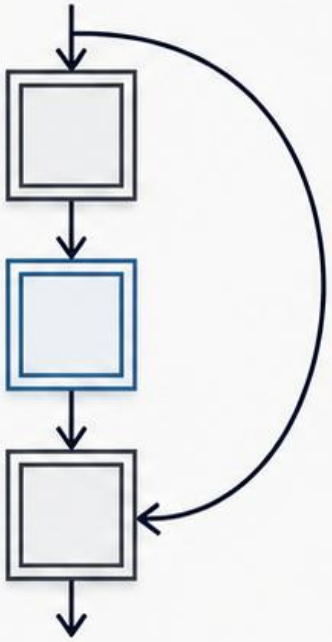
핵심 정보만을 담은 저차원 백터로 변환되어
모델 학습, 시각화, 검색 등에 활용됩니다.

핵심 요약 | 데이터의 분산을 최대한 보존하면서 차원을 축소하여, 노이즈는 제거하고 핵심 특징만 보존합니다.

아키텍처 분류 체계 (Taxonomy of Models)

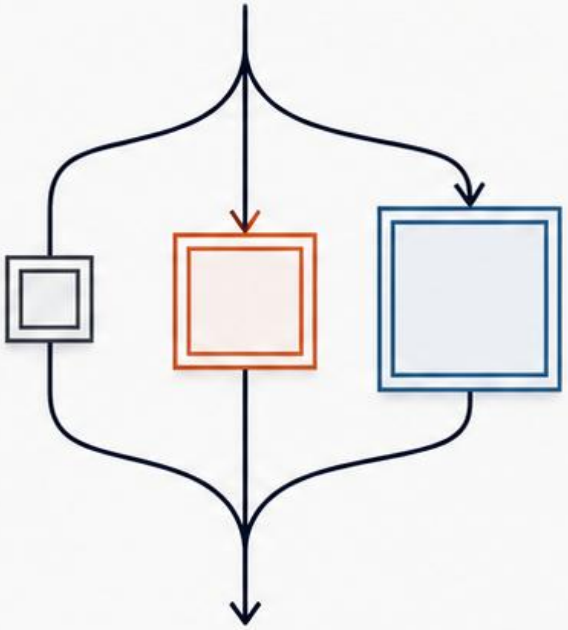


CNN의 진화: 한계를 극복한 최적화 모델들



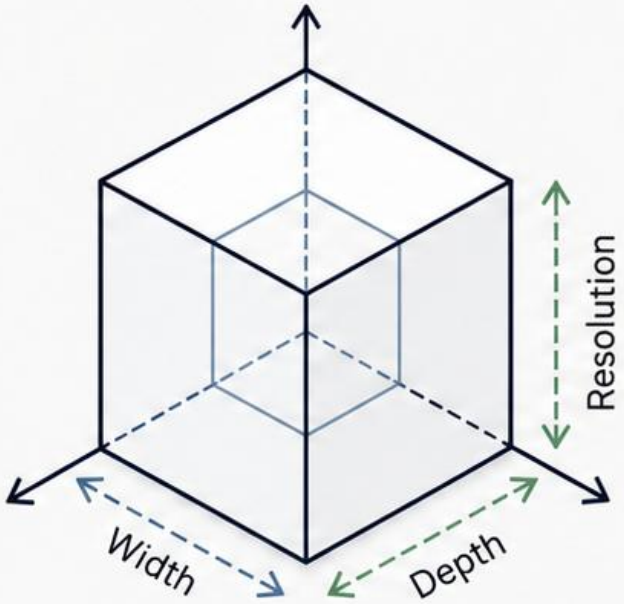
ResNet (Residual Network)

잔차 학습(Residual Learning) 도입. 기울기 소실 없이 매우 깊은 신경망에에서도 강력한 특징 추출 가능.



Inception (GoogLeNet)

다양한 크기의 합성곱 필터를 동시에 적용. 다중 수준의 특징을 효율적인 연산으로 추출.

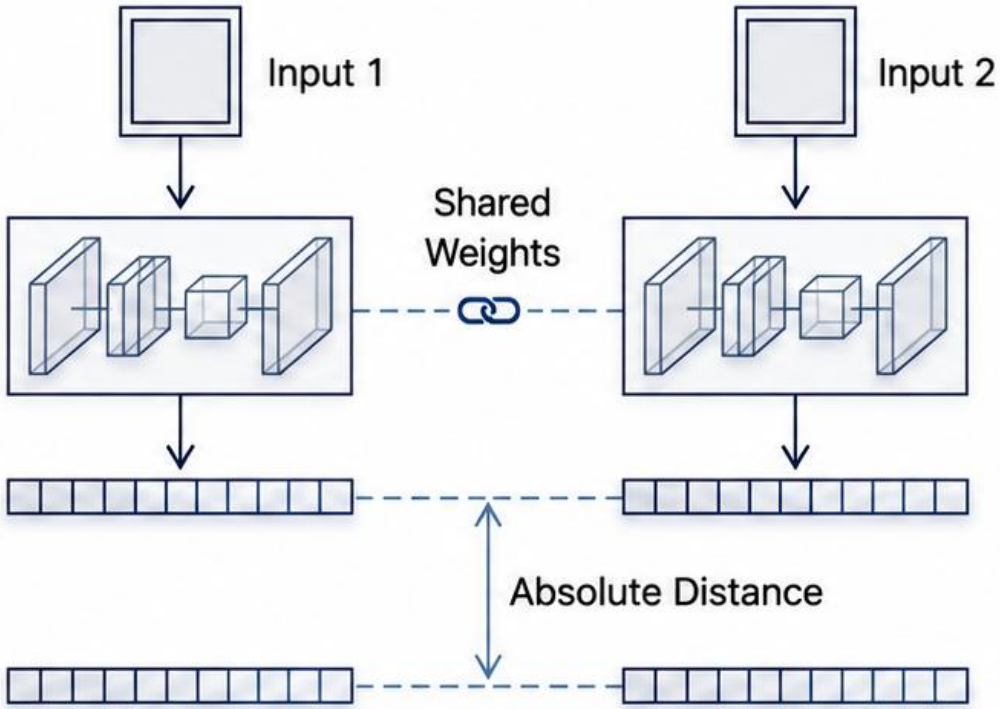


EfficientNet

파라미터 수를 최소화하면서도 성능을 극대화하기 위해 네트워크의 깊이, 너비, 해상도를 황금비율로 조정.

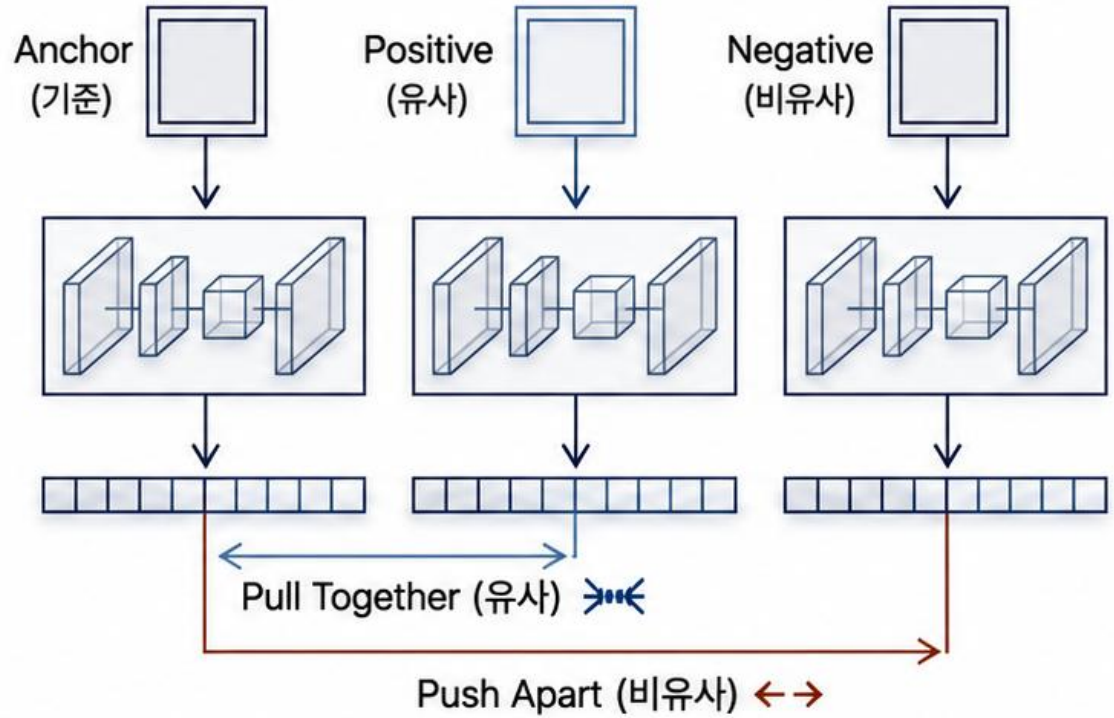
메트릭 러닝 (Metric Learning): 거리와 유사도 학습

Siamese Network (2-Input)



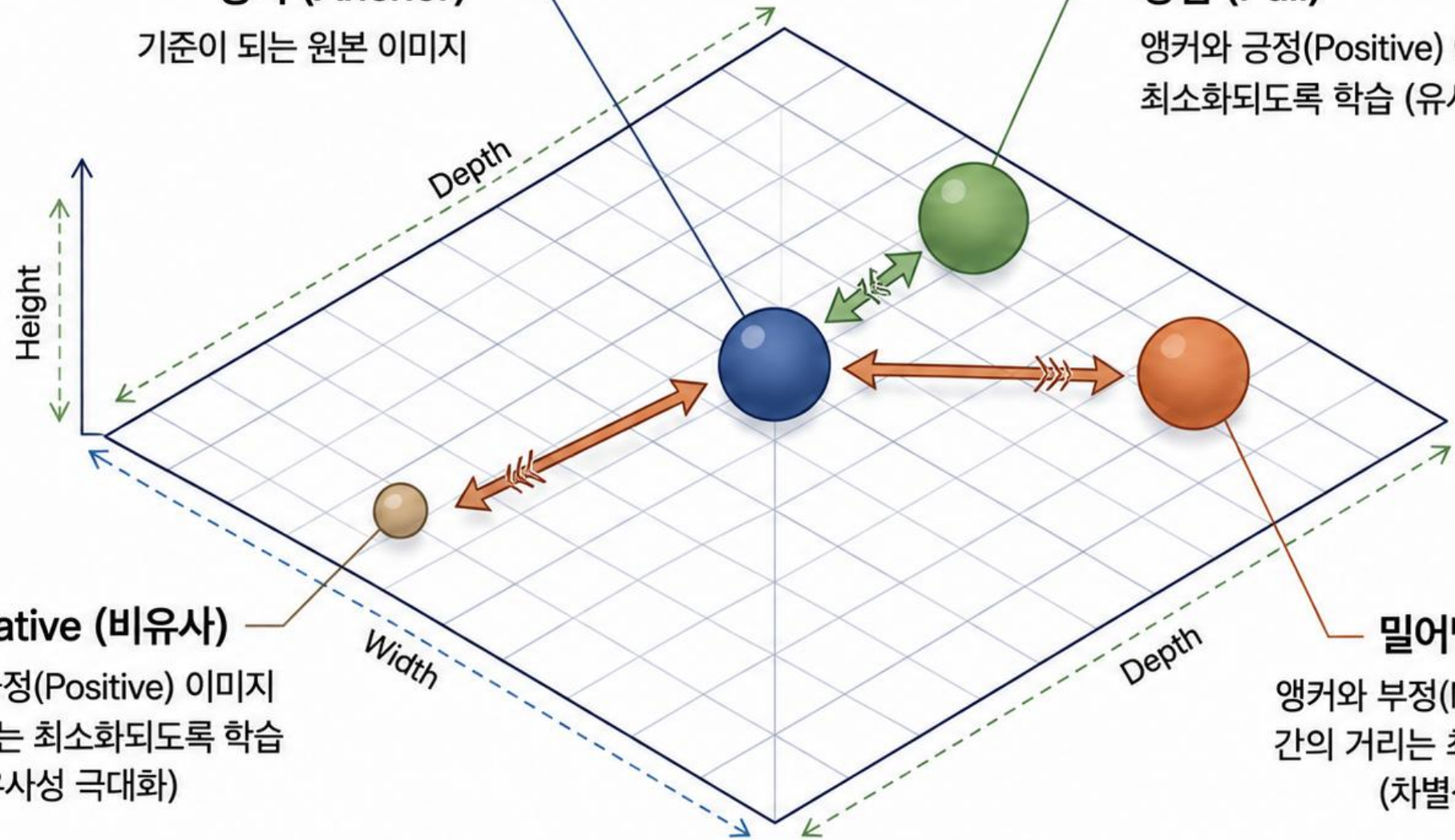
두 개의 동일한 신경망(가중치 공유) 사용.
두 입력 이미지 간의 절대적 거리 및 유사성 측정.
얼굴 인식 등에 활용.

Triplet Network (3-Input)

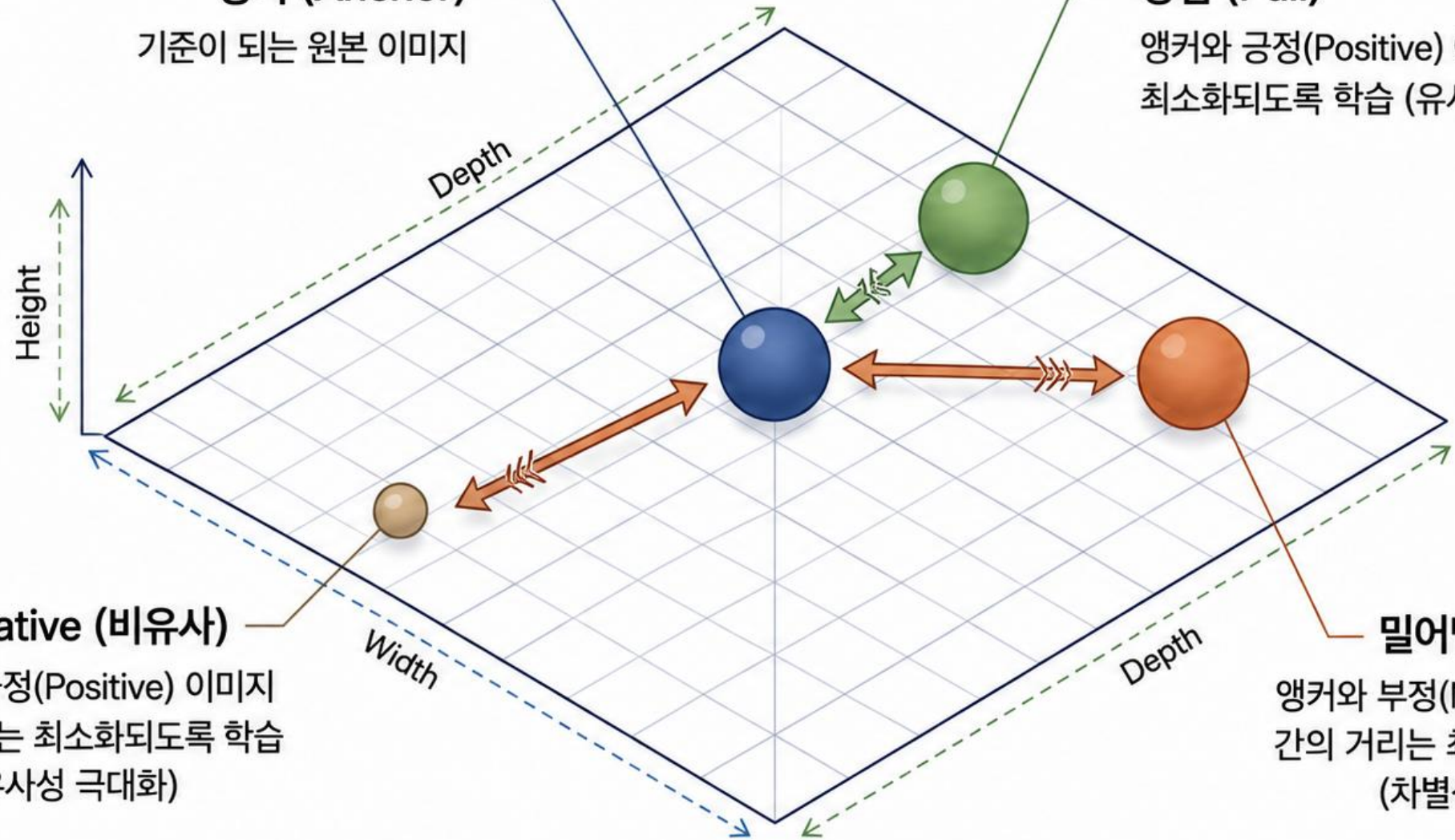


앵커(기준), 긍정(유사), 부정(비유사) 세 가지 입력 사용.
이미지 간의 상대적 관계(거리가 멀고 가까움)를
정밀하게 학습.

Triplet Network의 학습 공간 도식

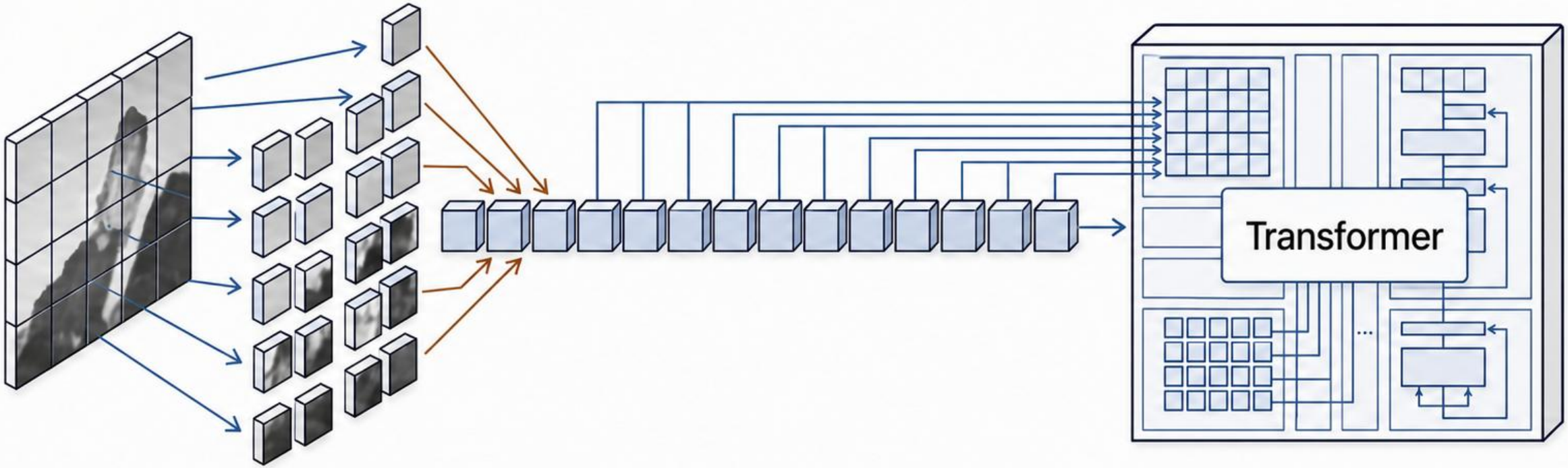


Triplet Network의 학습 공간 도식



패러다임의 전환: ViT (Vision Transformers)

이미지의 시퀀스화 및 전역적 관계 포착



격자 분할

원본 이미지를 균일한 크기의 패치(Patch)로 분할.

시퀀스 변환

분할된 패치들을 마치 텍스트 단어처럼 1차원 시퀀스 데이터로 나열.

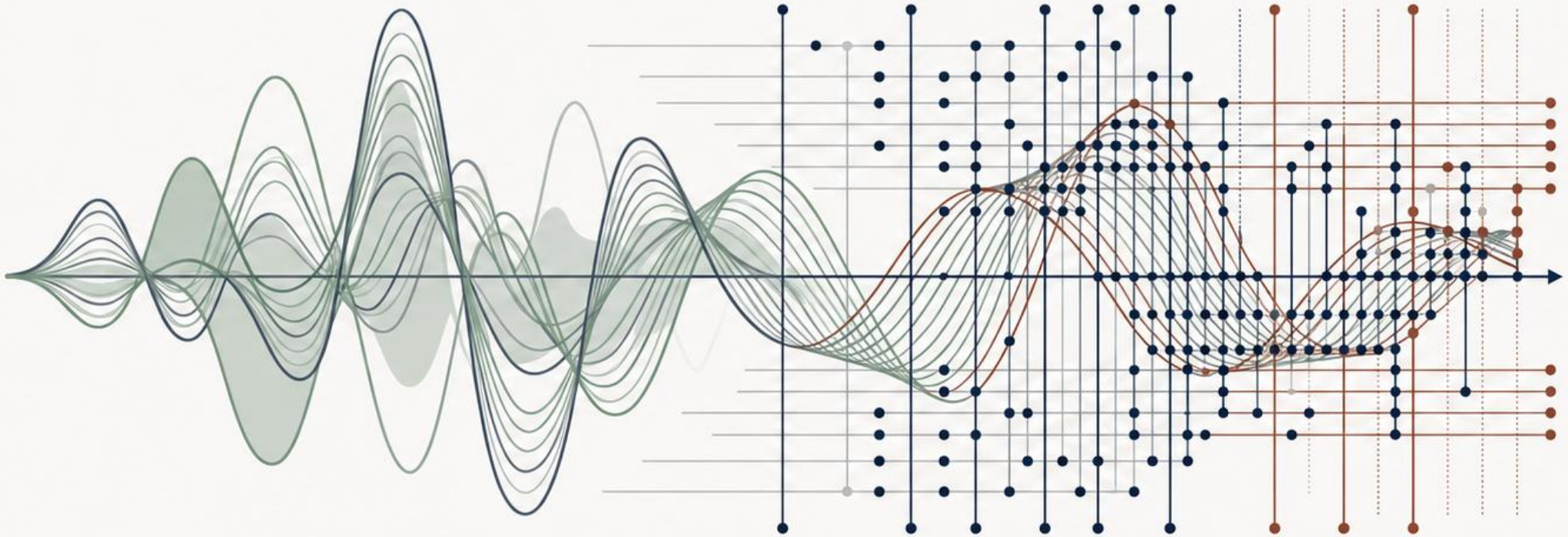
트랜스포머 적용

트랜스포머 아키텍처를 통해 이미지 전체의 전역적(Global) 관계를 종합적으로 포착하여 임베딩.

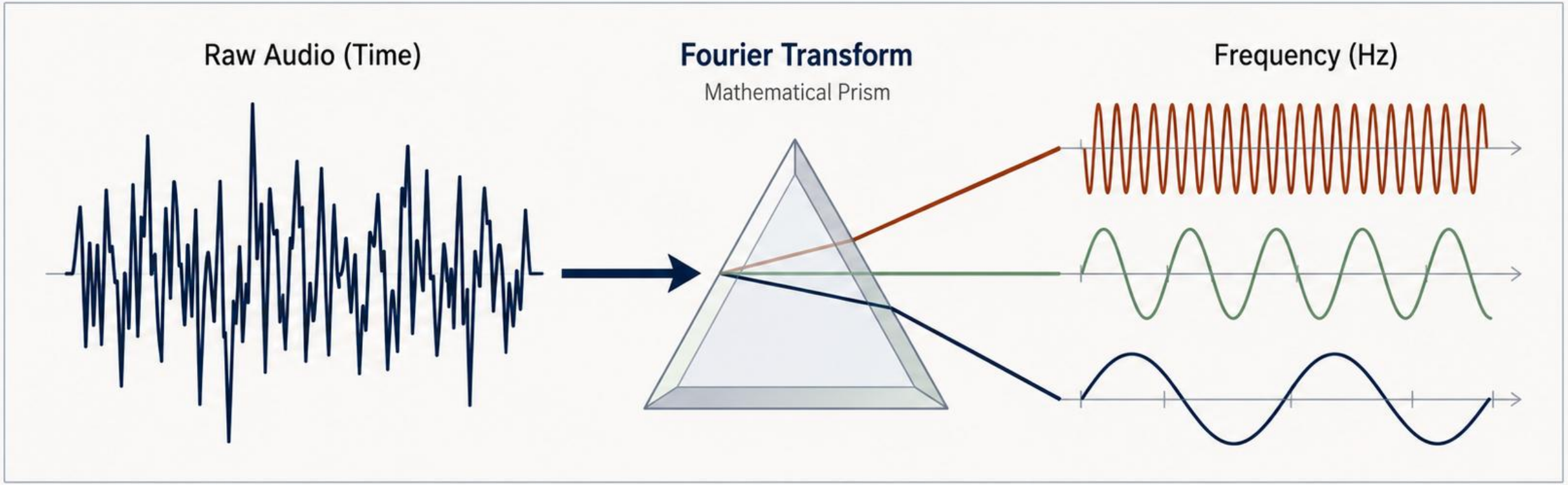
음성 임베딩



음성 신호 처리의 마스터키: Spectrum부터 MFCC까지



소리의 데이터를 해독하기 위한 차원의 전환



원시 신호의 한계 (Time-domain)

음성에 담긴 발음, 성별, 음색 정보는 원시 신호(Time-domain) 자체에서 쉽게 추출할 수 없습니다.

주파수 (Frequency) 정의

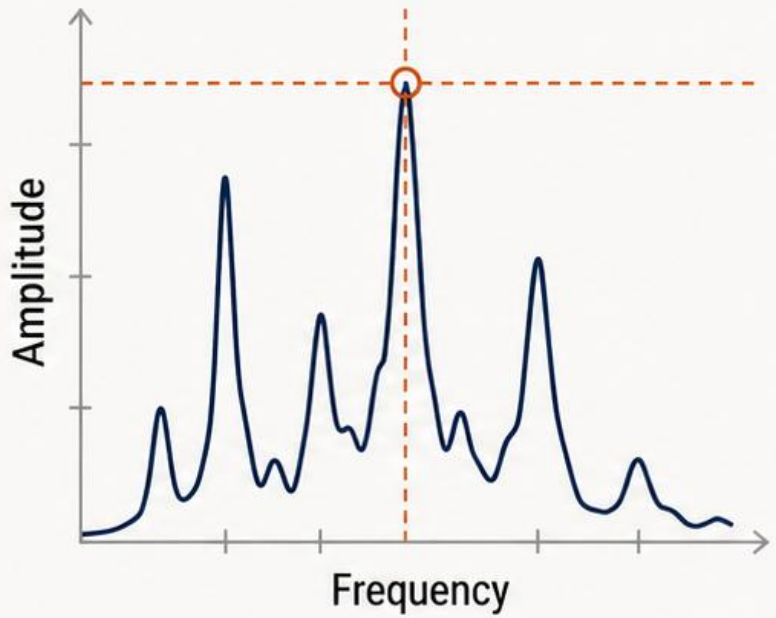
1초에 신호가 몇 번 진동했는지를 나타내는 수치. 주파수가 높을수록 고음, 낮을수록 저음이 됩니다. 자연의 소리는 다양한 주파수 성분의 합입니다.

푸리에 변환 (Fourier Transform) 역할

복잡한 소리를 개별 주파수 성분들로 완벽하게 분해하는 수학적 프리즘.

시간과 주파수를 결합한 소리의 완전한 시각화

Spectrum

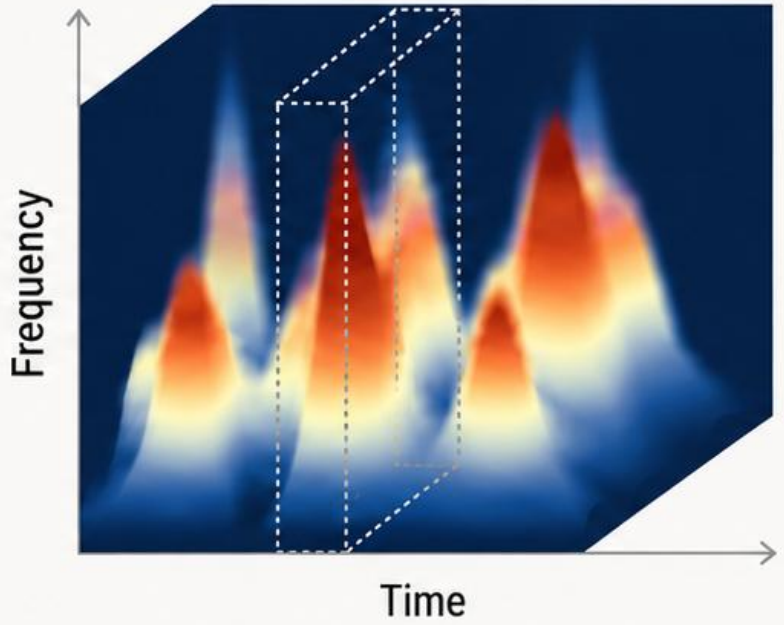


특정 시간 길이의 음성 조각(Frame)이 어떤 주파수 성분을 얼마나(Amplitude) 갖고 있는지 보여주는 단면.

STFT (Short Time Fourier Transform)

Window를 이동시키며 프레임 단위로 분할하여 연속적인 스펙트럼을 추출합니다.

Spectrogram



STFT를 통해 얻은 여러 개의 스펙트럼을 시간 축(Time-axis)에 나열한 결과물. 시간의 흐름에 따른 주파수의 변화를 한눈에 보여줍니다.

Short-Time Fourier Transform

인간의 발성 기관을 모방한 소리의 설계 구조

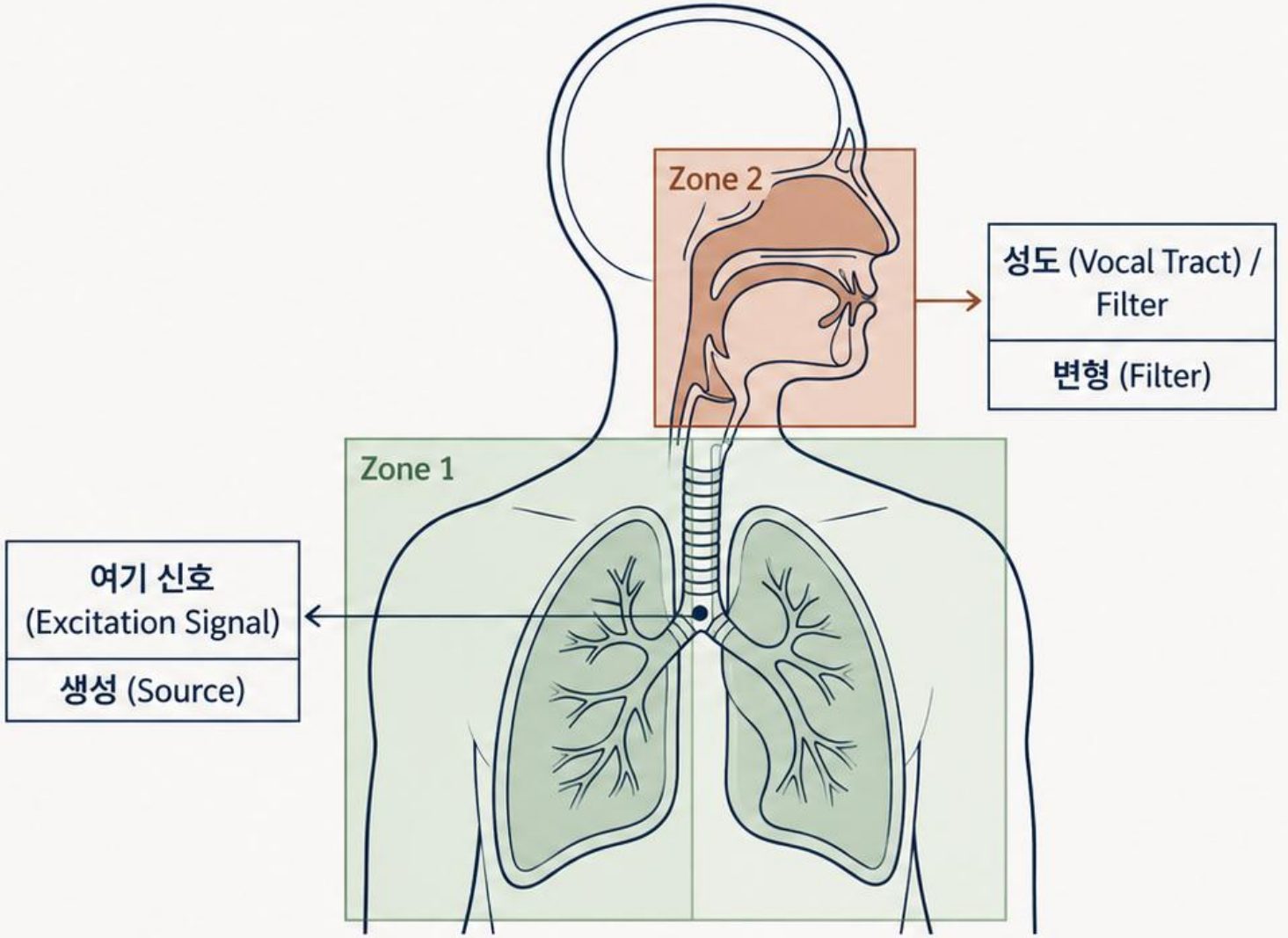
음성 생성은 두 단계의 생물학적 프로세스입니다.

생성 (Source)

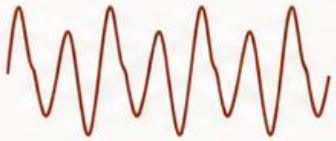
폐에서 나온 공기가 성대를 통과하며 초기 소리인 '여기 신호(Excitation signal)'를 만듭니다.

변형 (Filter)

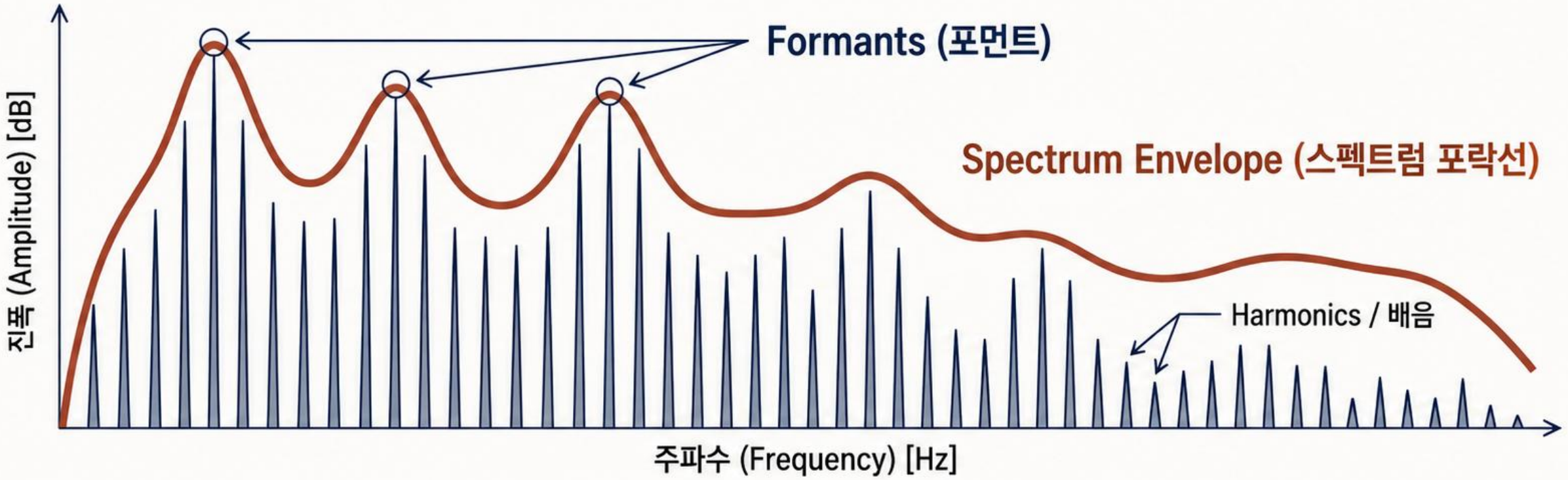
성대를 통과한 소리가 목, 코, 입, 혀 등의 성도를 통과하며 특정 주파수가 증폭되거나 감쇠하여 최종 발음이 결정됩니다.



초기 신호를 결정하는 성대 진동의 유무

	유성음 (Voiced)	무성음 (Unvoiced)
생물학적 특징	발성 시 성대의 진동을 동반함	진동 없이 성대를 고대로 통과함
신호의 성질	**준주기성** (Quasi-periodic) - 완전히 주기적이지 않으나 일정한 패턴 반복 	**비주기성** (Aperiodicity) 
스펙트럼 구조	고유의 기본 주파수(Fundamental frequency)와 배음(Harmonics)들로 구성	다양한 주파수 성분이 고르게 포함된 백색소음(White noise) 형태
분석 / 모델링	배음 구조의 차이가 악기나 사람의 고유한 음색을 생성	정규분포와 같은 확률분포로 쉽게 모델링 가능

발음을 결정하는 공명 주파수와 스펙트럼 포락선



Formant (포먼트)

소리가 성도를 지나며 공명되어 증폭되는 특정 지배적 주파수 대역. 배음과 만나 소리를 선명하게 만드는 필터 역할을 합니다.

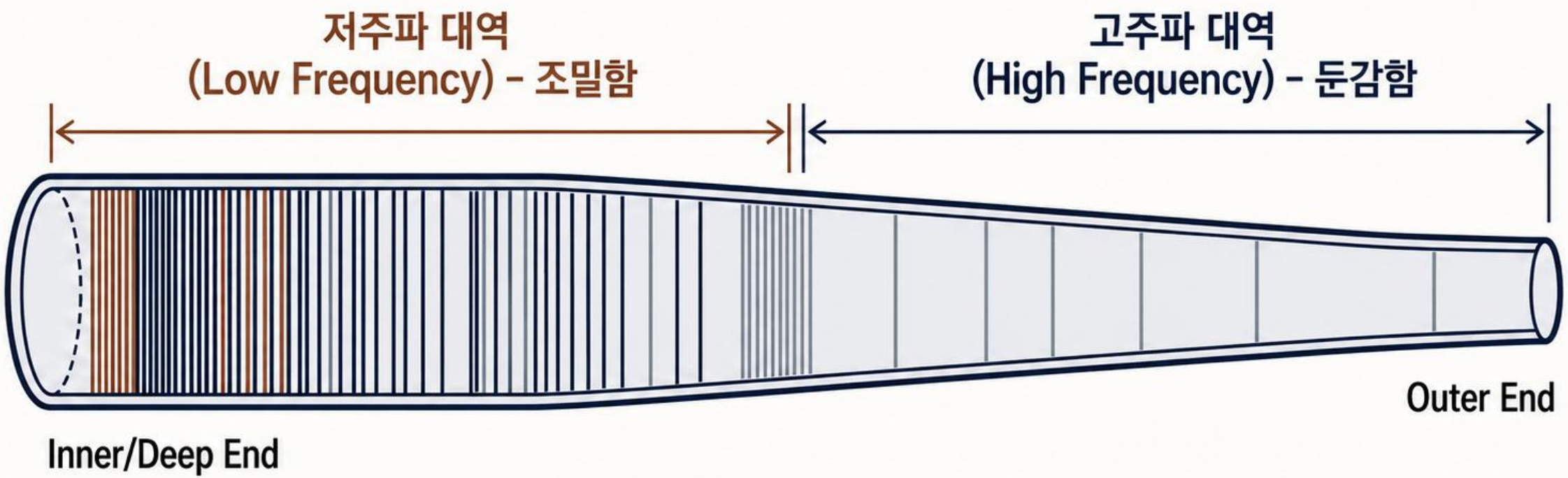
Spectrum Envelope

포먼트들을 연결한 부드러운 곡선.

발음의 결정

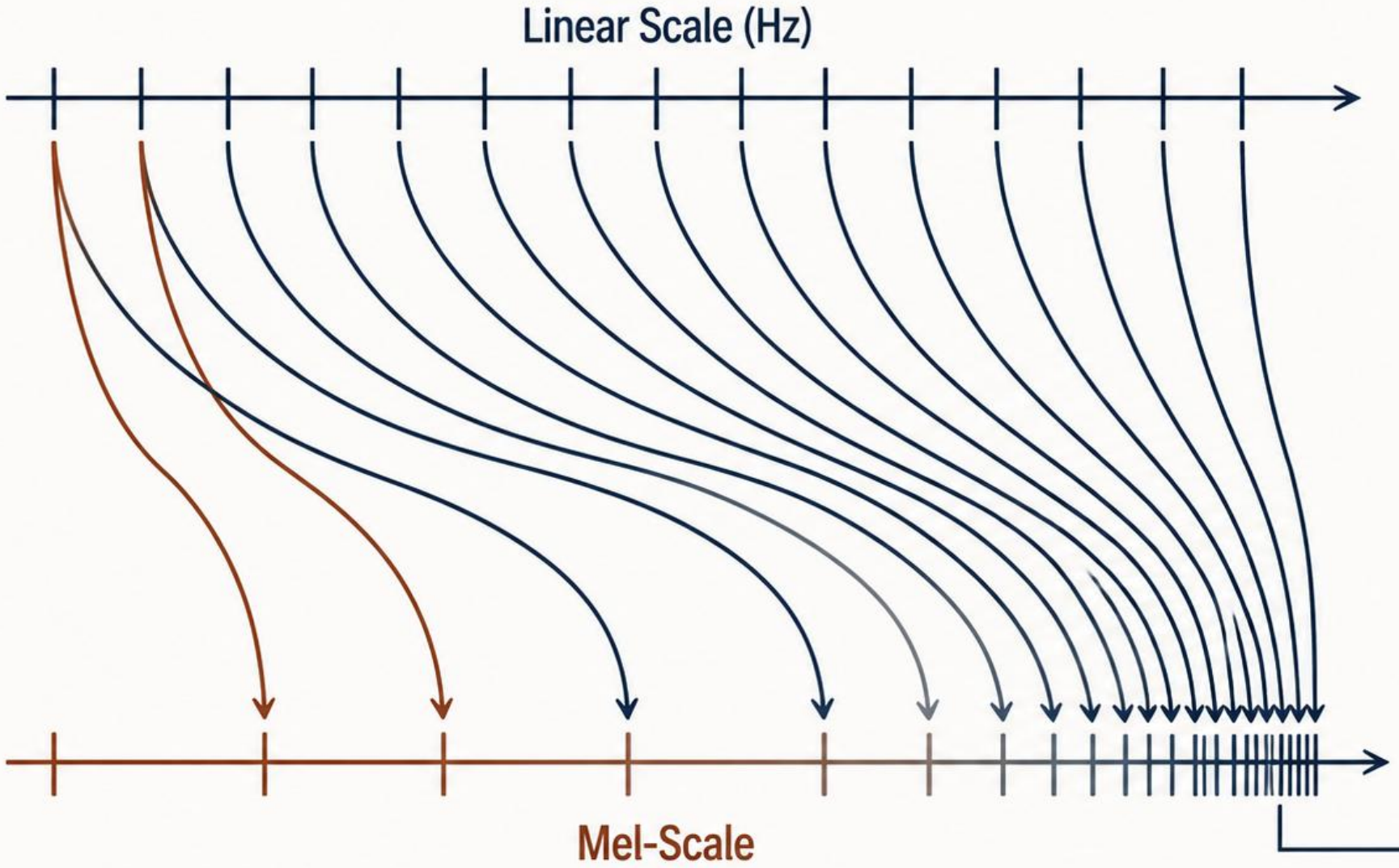
사람마다, 발음마다 성도 구조가 달라지며, 이 Envelope의 형태가 최종적인 '발음의 종류(음소)'를 결정하는 핵심 특징이 됩니다.

중요도에 따라 해상도가 달라지는 인간의 청각 구조



- 고막과 이소골을 거친 진동은 달팽이관의 청각 세포를 자극합니다.
- 인간의 귀는 모든 주파수를 같은 비중으로 인지하지 않습니다.
- 음성 정보의 핵심이 집중된 저주파 대역의 청각 세포는 매우 조밀하게 배치되어 해상도가 높고, 고주파 대역으로 갈수록 둔감해집니다.

청각 세포의 비선형적 배치를 수학으로 번역하다

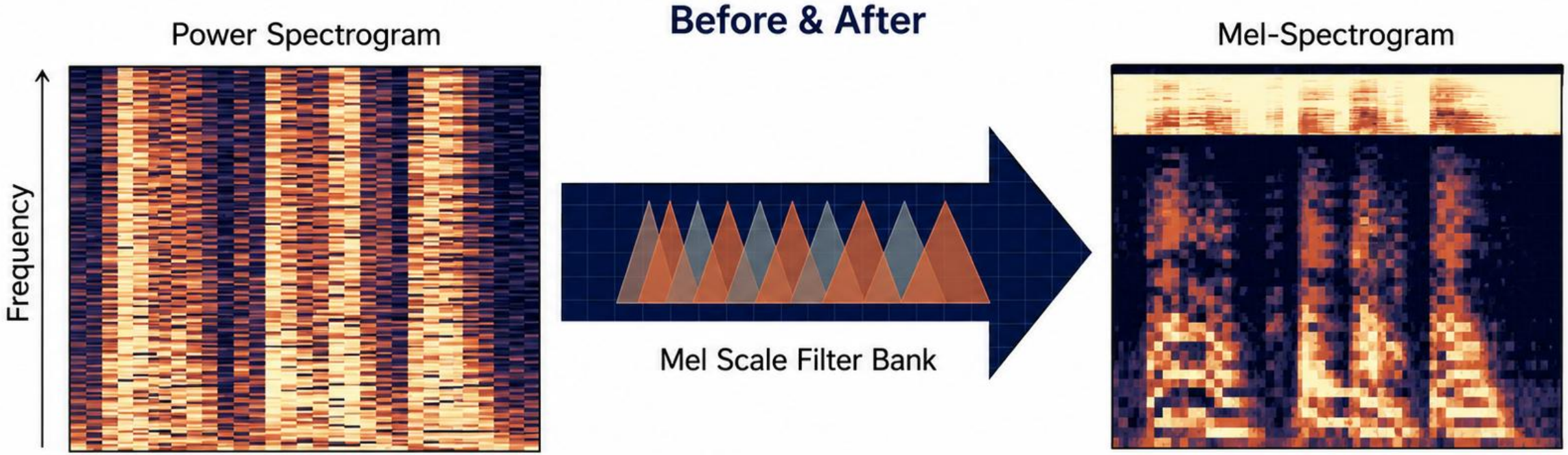


Mel-Scale

인간의 청각 구조를 반영하여 실제 물리적 주파수(Hz)를 수학적으로 왜곡시킨 지표.

기계가 소리를 '인간처럼' 듣제 만들기 위해, 음성 처리 알고리즘은 덜 중요한 고주파수 정보를 압축하고 중요한 저주파수 정보를 확장합니다.

인간의 뇌가 받아들이는 형태의 시각적 데이터



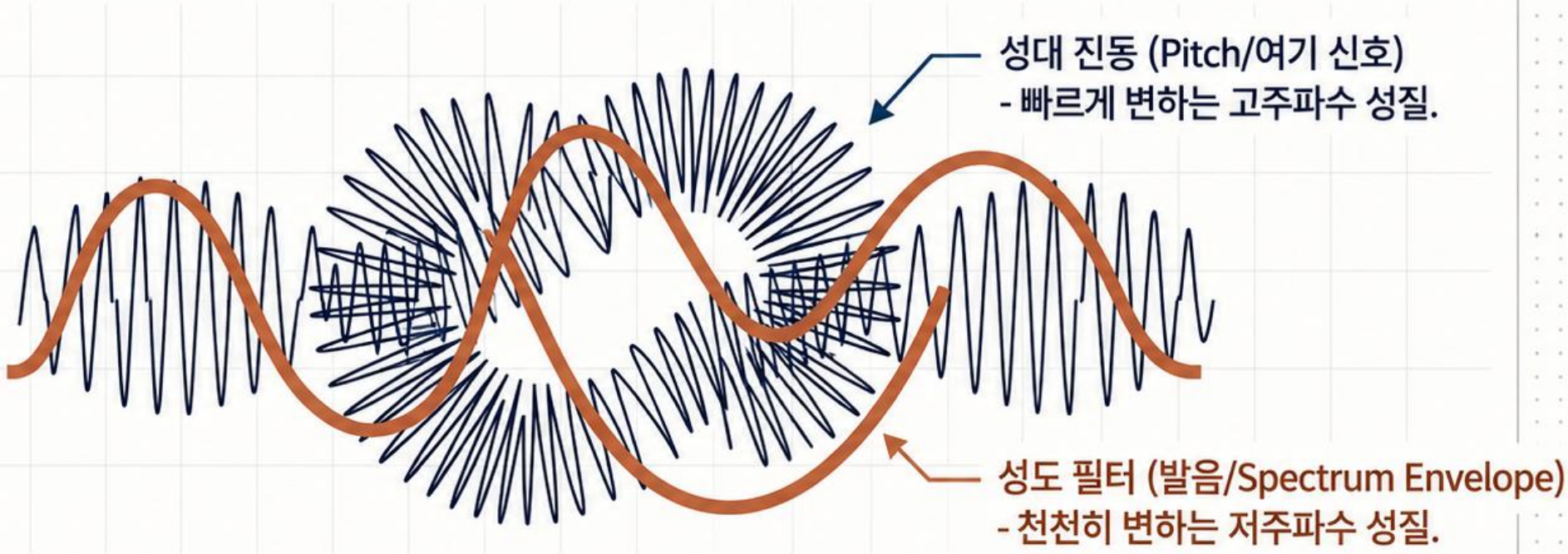
과정

각 스펙트럼의 제곱인 Power Spectrogram에
Mel Scale Filter Bank를 적용하여
차원 수를 줄입니다.

결과 (Mel-Spectrogram)

불필요한 고주파 데이터를 버리고 인간의 귀에 유의미한
정보만 효율적으로 압축해낸 최적화된 음성 특징
데이터입니다.

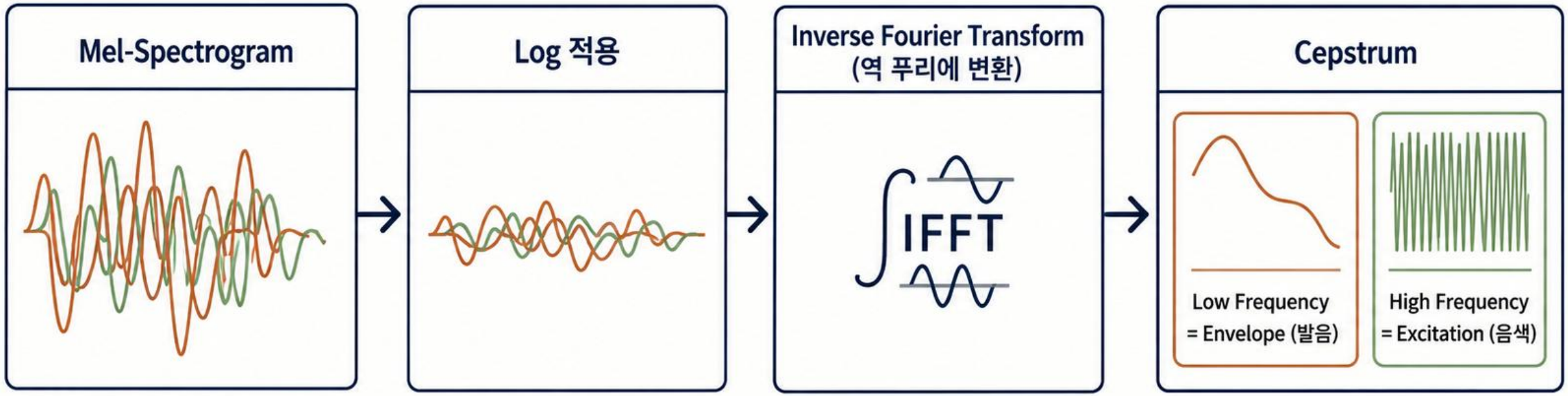
인공지능을 위한 마지막 과제: 정보의 분리



Mel-Spectrogram 상에서는 성대의 진동(음의 높낮이)과 성도의 모양(발음)이 하나로 섞여 있습니다.

음성 인식을 위해서는 화자의 음색이나 음높이에 흔들리지 않고,
순수한 '발음(Spectrum Envelope)' 정보만을 완벽하게 분리해내야 합니다.

켑스트럴 분석: 발음과 음색을 해체하는 수학적 마법



Log의 역할

데이터를 그대로 변환하면 버려지는 정보가 너무 많습니다. Log를 씌워 데이터 스케일을 축소시키고 변동성을 안정화합니다.

Inverse Fourier Transform

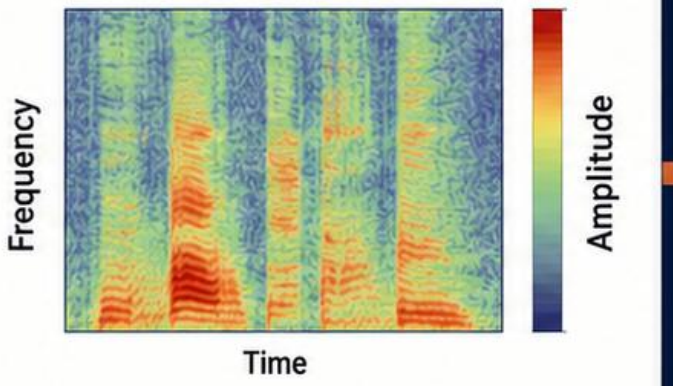
주파수 축에서 다시 한번 푸리에 변환을 수행합니다.

결과

시간 축에서 천천히 변하는 '발음(Envelope)' 정보는 낮은 성분으로, 빠르게 변하는 '여기 신호' 정보는 높은 성분으로 분리되어 원하는 특징만 추출할 수 있게 됩니다.

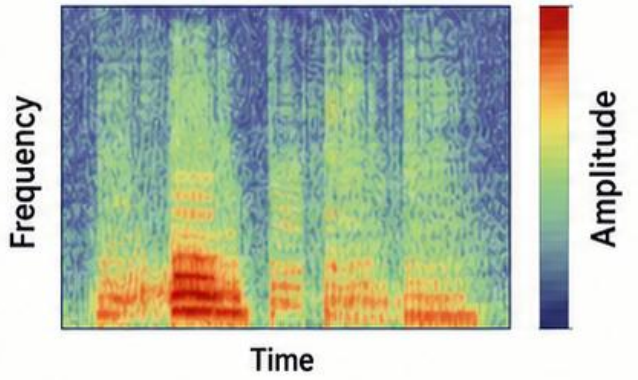
MFCC 추출 파이프라인 완성본

Raw Audio -> [STFT]



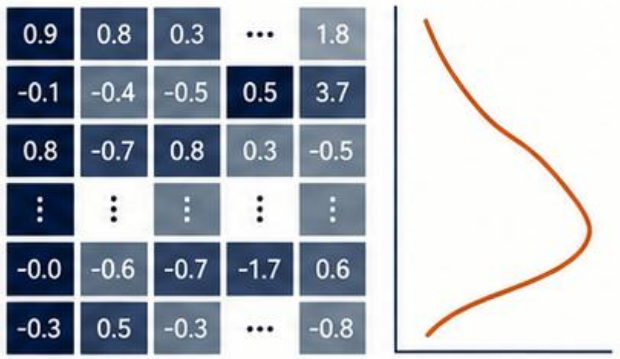
Power Spectrogram

[Mel-Scale Filter Bank]



Mel-Spectrogram

[Log Application + Inverse Fourier Transform]

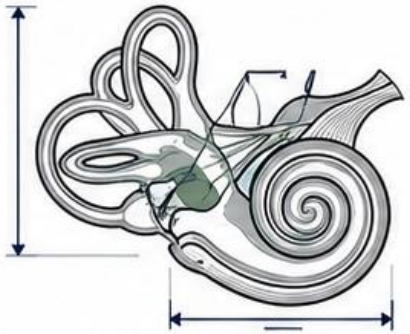


MFCC

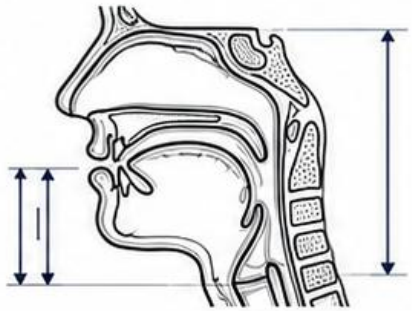
MFCC (Mel-Frequency Cepstral Coefficient): 인간의 청각 구조를 반영하여 특징을 압축하고, 음성 인식에 필수적인 발음 특성(Envelope)만을 정제해낸 궁극의 Feature Vector.

인체 해부학과 알고리즘의 완벽한 대칭성

Human Biology



달팽이관 (Cochlea)



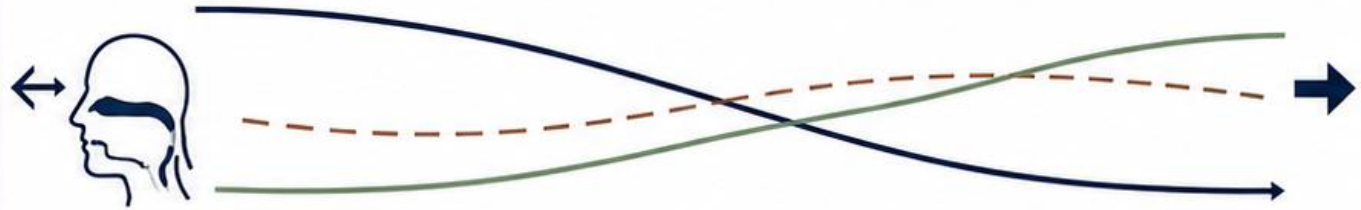
성도 (Vocal Tract)

Ear ↔ Mel-Scale: 비선형적인 인간의
청각 인지 능력을 Filter Bank로 디지털화.

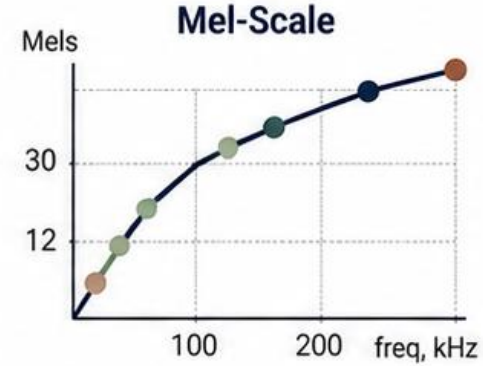


Mel-Scale Filter Bank

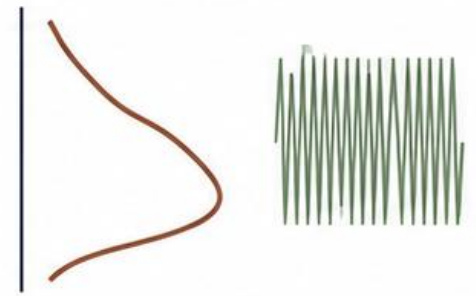
Throat ↔ Cepstral Analysis: 발음을 결정하는
물리적 성도의 형태를 Inverse FT를 통해 수학적으로 분리.



Algorithm

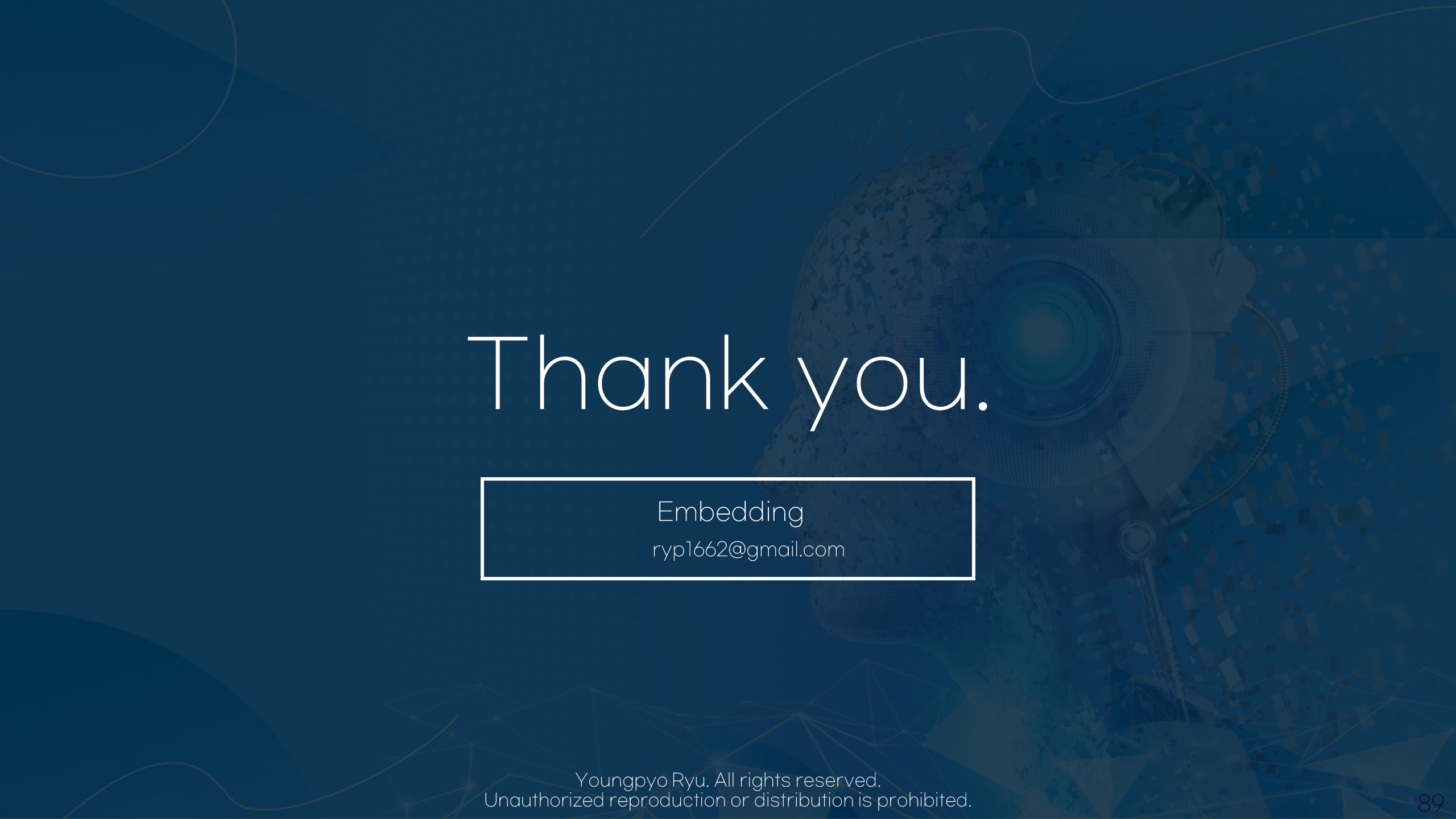


Cepstral Analysis / Envelope



신호 처리는 단순한 임의의 수학이 아닙니다.

MFCC는 기계에게 인간처럼 듣고 이해하는 법을 가르치는 생체 모방 엔지니어링의 결정체입니다.



Youngpyo Ryu. All rights reserved.
Unauthorized reproduction or distribution is prohibited.